

第 37 章：Unicode and Byte Strings (Unicode 与字节字符串)

原书：《Learning Python》(6th Edition)，作者 Mark Lutz 本章地位：第 7 章为了让你上手基础，刻意"注水"了字符串的内容；本章则是字符串的进阶篇，把完整的 Unicode 文本和二进制数据字符串故事讲透。在 Python 3.X 中，普通字符串就是 Unicode，所以本章是绕不过去的必修内容——无论将来处理国际化的互联网文本、XML 解析，还是图像、音频、网络传输等二进制数据，都必须理解 `str` / `bytes` / `bytearray` 这三种字符串类型的分工与文本文件/二进制文件的严格区分。

37.1 开篇引言 (Chapter Introduction)

英文原文

In Chapter 7, the Python string story was watered down on purpose to help you get started with the fundamentals. Now that you've learned the basics, this chapter moves on to extend them to include the full Unicode-text and binary-data string stories in Python. This extension was more optional in earlier editions of

this book because Unicode was an afterthought in Python 2.X. Python 3.X elevates it to required reading because its normal strings simply are Unicode. Still, how much you need to care about this topic depends in large part upon which of the following categories you fall into:- If you deal with non-ASCII Unicode text—for instance, in the context of internet content, internationalized applications, XML parsers, and some GUIs—you will find direct and seamless support for text encodings in both Python's all-Unicode `str` object, as well as its Unicode-aware text files.- If you deal with binary data—for example, in the form of image or audio files, network transfers, or packed data shared with lower-level tools—you will need to understand Python's `bytes` object and its sharp distinction between text and binary data and files.- If you fall into neither of the prior two categories, you may be able to defer this topic and use strings as you did in Chapter 7: with the general `str` object, text files, and all the familiar string operations. Your strings will be encoded and decoded using your platform's default Unicode encoding, but you won't notice—until, as you'll see, you encounter content or platforms that use a different default! To be sure, if text is always ASCII in your corner of the software world, you might be able to get by with simple string objects and text files and can avoid much of the story that follows. As you'll learn in a moment, ASCII is a simple kind of Unicode and a subset of

other common encodings, so string operations and files "just work" if your programs process ASCII text only and will never deviate from this limitation. Even if this chapter's topics seem remote to you today, though, a basic understanding of Python's string model can both demystify some of the underlying details now and prepare you for Unicode or binary-data issues that may impact you in the future. Given the prominence of the web in most software careers today, that impact may be more a matter of when than if.

中文翻译

在第 7 章中，我们刻意把 Python 的字符串内容“注了水”，为的是帮助你从基础开始入门。既然你已经掌握了基础，本章就继续深入，把话题扩展到 Python 中完整的 Unicode 文本与二进制数据字符串的故事上。在本节的早期版本中，这部分扩展内容是“可选读”的，因为在 Python 2.X 时代，Unicode 是事后才补上的设计。Python 3.X 则把它升格为必修内容，因为 3.X 的普通字符串就是 Unicode。不过，你需要在这上面投入多少精力，很大程度上取决于你属于下面哪一类人：- 如果你处理非 ASCII 的 Unicode 文本——例如互联网内容、国际化应用、XML 解析器以及某些图形界面（GUI）中的应用——你会发现，Python growth 全 Unicode 的 str 对象以及它感知 Unicode 的文本文件，都为你提供了直接顺滑的文本编码支持。- 如果你处理二进制数据——例如图像或音频文件、网络传输、以及与底层工具共享的打包数据——你将需要理解 Python 的 bytes 对象，以及文本与二进制数据、文本文件与二进制文件之间

的严格区分。- 如果你两者都不属于，那么你可以推迟学习这些内容，像第 7 章那样使用字符串：即通用的 `str` 对象、文本文件，以及一切熟悉的字符串操作。你的字符串会按照你所在平台的默认编码进行编码和解码，但你不会察觉——一直到，正如你即将看到的，你遇到了使用不同默认值的内容或平台！毫无疑问，如果在你的软件世界里文本永远是 ASCII，你也许只需要简单字符串对象与文本文件就够了，可以跳过后面的大部分内容。正如你稍后会学到的，ASCII 是一种简单的 Unicode，也是其他常见编码的子集，所以只要你的程序只处理 ASCII 文本并且永远不会偏离这个限制，字符串操作和文件“开箱即用”。尽管如此，即使本章的主题今天看起来离你很远，但对 Python 字符串模型有一个基本理解，既能现在解开一些底层细节的迷惑，也能为你应对未来可能影响你的 Unicode 或二进制数据问题做好准备。鉴于如今 Web 在大多数软件职业中的突出地位，这种影响更可能只是何时发生，而非是否会发生的区别。

深度理解

- 核心概念：这一章把字符串世界观彻底刷新：文本字符串 (`str`) 在内存里是“解码后的字符码点 (code points)”整数序列，而 `bytes` 是“原始的字节值”序列。两者虽长得像，却是本质不同的东西——这是全章的一根主线。
- 底层视角：为什么 Python 2 时代 Unicode 是“追加出型 patch”？因为 2.X 的默认字符串就是字节，`unicode` 类型是另请出场的第二类字符串；到了 3.X，Lutz 让 `str` 全部改为 Unicode，才把“文本 vs 字节”的矛盾在类型系统里彻底解决。

·设计原因：把三种人分开（Unicode 文本、二进制数据、纯 ASCII）是为了让你"按需精读"——纯 ASCII 程序员能跳过大部分章节，但作者也做了"劝学"式提醒：Web 时代，跨平台文本迟早找上门。

·解决什么实际问题：互联网内容（HTML/JSON/XML）、国际化的应用、图像与音频的读取、与其他语言工具的二进制互操作——这些就是 80% 的程序员会撞上 Unicode/二进制问题的真实场景。

·初学者误区：以为"纯 ASCII 处理就不用懂编码"。这只能撑一时，一旦遇到带重音字符、emoji 的文件，或者 Windows (cp1252) 与 macOS/Linux (UTF-8) 混用，立刻报 UnicodeDecodeError/UnicodeEncodeError，这时再回头补课成本更高。

37.2 Unicode 基础 (Unicode Foundations)

英文原文

Before jumping into code, let's begin with a general overview of the Unicode model and Python's support for it. To fully understand both, we have to start with a brief look at how characters are actually represented in computers.

中文翻译

在进入代码之前，先从一个总览开始：Unicode 模型到底是怎么回事，Python 又是如何支持它的。要真正理解这两者，我们必须先从“字符在计算机里到底是怎样表示的”这个问题看起。

深度理解

- 核心概念：Unicode 是一位“两层的协议”：一层是字符码点（code points，字符在内存中的整数编号），另一层是编码（encoding，把这些整数序列映射到字节的规则）。这一节先讲清楚第一层，下一节再讲第二层。
- 底层实现：码点本质就是整数。‘a’在内存里存的不是字母“a”的图像，而是整数 97。ord()/chr() 就是字符与整数之间的双向“翻译员”。
- 为什么这样设计：先有整数码点再谈存储，是为了让所有语言共用同一份“字符表”——不管你在法国、俄罗斯还是日本，0x1F642 都是同一个笑脸。
- 常见的实际问题：判断字符串长度千万别假设“一个字符等于一个字节”——emoji 脸一个码点就要 4 字节；而 len() 数的是码点，不是字节，这两者经常不相等。
- 初学者误区：把 len("") 当 4。实际是 1。字节数要看编码后的结果。初学者还爱把 bytes 和“编码后的字符串”混为一谈，其实 bytes 也可以完全不含任何文本意图（比如一张 JPEG 图）。

37.2.1 字符表示 (Character Representations)

英文原文

Most programmers think of strings as a series of characters (really, their integer codes) used to represent textual data. That's still true in the brave new world of Unicode, but the way characters are stored in a computer's memory and files can vary, depending on both what sort of characters are recorded and how programmers choose to record them. For many programmers in the US, ASCII formed their original notion of text strings. ASCII is a standard that defines character codes 0...127 (which always means an inclusive range in this chapter) and thus allows each character to be stored in one 8-bit byte (using 7 bits). For example, the ASCII standard maps character a to the integer value 97 (0x61 in hex), which can be stored in a single byte both in computer memory and on files. To witness this for yourself, Python's built-in ord function shows the integer code of a given character; chr reveals the character of a given integer code; and hex gives the code's byte value as two hex digits, each of which fits a 4-bit "nibble". The first of these, ord, is the value of a character's representation code—and byte—in ASCII:

```
$ python3# Or py -3 on Windows >>> ord('a') # 字符 => 码 (Character => code) 97 >>> chr(97) # 码 => 字符 (Code => character) 'a' >>> hex(97) # 字节值: 占满 8 位 (Byte value: fits 8 bits) '0x61' >>> 0b0111
```

1111 # ASCII 7 位范围的上限 (Limit of ASCII's 7-bit range) 127 ASCII makes text processing simple, because characters directly correlate to bytes. Sometimes, though, this isn't enough. Accented characters and special symbols, for example, do not fit into the range of character codes defined by ASCII. To allow for some such extra characters, other standards allow all possible values in an 8-bit byte, 0-255, to be used as codes, and assign values 128-255 to additional characters. One such standard is known as Latin-1 and is widely used in Western Europe. In Latin-1, character codes above 127 are assigned to accented and otherwise special characters. For instance, the character that Latin-1 assigns to code 196 (a.k.a. byte value 0xc4) is a specially marked and non-ASCII character, Ä. In Python:

```
>>> chr(196) # ASCII 装不下它 (Too big for ASCII) 'Ä'
>>> ord('Ä') # Latin-1 没问题 (Okay for Latin-1) 196
>>> hex(ord('Ä')) # Latin-1 中的字节值 (Byte value in Latin-1) '0xc4'
>>> bin(ord('Ä')) # Latin-1 用满全部 8 位 (Latin-1 uses all 8 bits) '0b11000100'
```

Still, some alphabets define so many characters that it is impossible to represent them as one byte-sized code per character. The integer codes of the symbols and characters in the following, for example, require more space than a byte—as do those of all the emojis that may not work in some tools but manage to creep into your emails and texts anyhow:

```
>>> ord('👉') # 手指标符号: 超出 127
9758 >>> hex(ord('👉')) '0x261e'
>>> hex(ord('👉')) '0x261e'
```


`d(c)) for c in '㐀' # 同理: 这些字符必须用 Unicode ['0x771f','0x41b','0x21e8'] [hex(ord(c)) for c in ''] # E`
 moji: 超过 2 字节 (16 位) ['0x1f642','0x1f64a','0x1f44d'] Unicode provides the generality we need to deal with text containing non-ASCII characters and symbols like these. In fact, it defines and assigns enough character codes to represent almost every natural language in use, plus a large set of symbols and emojis. In Unicode speak, these codes that stand for characters take the shape of numbers (integers) and are usually called code points. The code points that Unicode assigns to characters `a`, `Ä`, and `,` for instance, are 97, 196, and 128578 (0x61, 0xc4, and 0x1f642 in hex), respectively: `>>> [f'{c} is {ord(c)} and {hex(ord(c))}' for c in 'aÄ,']` [`a is 97 and 0x61', 'Ä is 196 and 0xc4', ' is 128578 and 0x1f642'`] Unicode is sometimes referred to as "wide-character" strings because its range of characters is so broad that multiple bytes may be needed to represent individual character codes. Such text is readily stored in computer memory because each character code can simply span as many bytes as its code-point number requires (the exact way this is done can vary by programming language and isn't consequential to your Python code). Once text leaves your computer, though, its storage is more constrained: bytes are a bad thing to waste on your drives and networks, and text used across platforms must follow the same formatting rules. To allow for this, Unicode also defin

es standard ways to map character codes to and from bytes for storage and transmission that are both platform- and language-neutral—the encodings we'll explore in the next section. The takeaway here is that Unicode's combination of all-encompassing character code points and their predefined encodings make it a portable and flexible model, and the standard way that programs deal with non-English and other text that may have more characters than 8-bit bytes can handle. As an added benefit, earlier schemes like ASCII also fall under the Unicode umbrella unchanged, but we have to move on to the next section to see how.

中文翻译

大多数程序员把字符串理解为"一系列字符（其实是字符的整数码）"，用来表示文本数据。在 Unicode 的新世界里这个理解依然成立，但字符串在计算机内存和文件中的存储方式却可能千变万化——取决于你记录的是哪类字符，以及程序员选择如何去记录它们。对很多美国程序员而言，ASCII 便是他们最初对文本字符串的认知。ASCII 是一套标准，定义了字符编号 0-127（本章里永指含端点的闭区间），因此每个字符可以用一个 8 位字节（只用到 7 位）来存储。例如，ASCII 标准把字符 a 映射为整数 97（十六进制 0x61），它在计算机内存与文件中只需单字节即可存储。为了亲自验证，Python 内建的 ord 函数能显示某给定字符的整数码；chr 揭示某整数码对应的字符；hex 把"码值"按两个十六进制数字给出字节值，每个十六进制正好能放进 4 位的"nibble（半字节）"。其中 ord 就是字符在 ASCII 中的"

表示码"——同时也是字节——的值：\$ python3 # Windows 下也可以写 py -3 >>> ord('a') # 字符 => 码97 >>> chr(97) # 码 => 字符'a' >>> hex(97) # 字节值：正好 8 位'0x61' >>> 0b01111111 # ASCII 7 位范围的上限127 ASCII 之所以让文本处理变得简单，是因为字符与字节直接——对应。但有些时候这还不够——例如带重音的字符和特殊符号，并不在 ASCII 定义的字符码范围内。为了容纳部分这类额外字符，另一些标准允许 8 位字节中的全部可能值 0-255 都当字符码使用，并把 128-255 这些值分给附加字符。其中一个标准就是 Latin-1，在西方欧洲广泛使用。在 Latin-1 中，大于 127 的字符码被分配给带重音或其他特殊字符。例如，Latin-1 把码 196（即字节值 0xc4）分配给一个特殊标记的非 ASCII 字符 Ä。在 Python 中：>>> chr(196) # 超过 ASCII 范围'Ä'>>> ord('Ä') # 对 Latin-1 没问题196 >>> hex(ord('Ä')) # Latin-1 中的字节值'0xc4'>>> bin(ord('Ä')) # Latin-1 用满了全部 8 位'0b11000100'然而，有些字母表定义的字符实在太多，难以用一个"一字一字节"的码点来代表。比如下面的几个符号字符，它们的整数编码就需要比一字节更大的空间——那些各种版本的 emoji 也莫不如此，它们在有些应用里也许显示不好，却终究会钻进你的邮件和短信里：>>> ord('') # 指明 > 1 字节9758 >>> hex(ord('')) # 超出单字节范围'0x261e'>>> [hex(ord(c)) for c in '几'] # 同理：必须 Unicode ['0x771f','0x41b','0x21e8'] >>> [hex(ord(c)) for c in ''] # Emoji：超过 2 字节（16 位）['0x1f642','0x1f64a','0x1f44d'] Unicode 提供了我们所需要的、容纳非 ASCII 字符和上述符号文本的通用性。事实上，它定义并分配了足够多的字符码，足以代表几乎所有现存自然语言，外加一大

套符号和 emoji。用 Unicode 的行话说，这些代表字符的码值以数字（整数）形式出现，通常称为码点（code points）。例如，Unicode 分配给字符 a、Ä、的码点分别是 97、196、128512（十六进制的 0x61、0xc4、0x1f642）：

```
>>> [f'{c} is {ord(c)} and {hex(ord(c))}' for c in 'aÄ']  
['a is 97 and 0x61', 'Ä is 196 and 0xc4', ' is 128512 and 0x1f642']
```

Unicode 有时被称为“宽字符”（wide-character）字符串，因为它的字符范围太广，所以表示单个字符码可能需要多个字节。这样的文本在内存中易于存储，因为每个字符码只需按其码点数值所需的字节数进行扩展（不同语言的具体实现方式可能不同，这对你的 Python 代码并不重要）。然而，一旦文本离开你的电脑，它的存储就受限制多了：在硬盘和网络上浪费字节是很糟的事，而跨平台使用的文本必须遵循相同的格式化规则。为了满足这一点，Unicode 还定义了把字符码与字节之间互相映射的标准方式，用于存储和传输，并且与平台和语言无关——这就是我们将在下一节探索的编码（encodings）。这里的重点是：Unicode 将无所不包的字符码点与预先定义的编码结合起来，使它成为一种可移植且灵活的模式，也是程序处理非英语以及 8 位字节可能装不下的其他文本的标准方式。作为额外的好处，ASCII 这样的早期方案也同样以原样划入 Unicode 的保护伞下——不过要亲眼看到这些，我们得先进入下一节。

深度理解

·核心概念：字符表示的两级抽象——ASCII 是“一字符一字节”的简模型，Latin-1 是“一字节一字符但会用满 128 位”的扩展，而 Unicode 是“任意码点，字节数不定”的终

极方案。从 ASCII→Latin-1→Unicode，是"字宽度不断增"的主线。

·底层实现：ord('a')==97 之所以是 97，是因为 ASCII 表的布局：97 对应小写 a，0-31 是控制符，32 是空格。而 hex(97)=='0x61' 揭示了"ord 的 97 与字节值 0x61 在 ASCII 中完全同构"——这为后面"ASCII 是二进制兼容子集"埋下伏笔。

·为什么这样设计："统一码点、各自编码"的双层架构，是为了让"内存表示"和"存储形式"松耦合。内存快，存储省——二者都由 Unicode 协调，但互不捆绑。

·解决什么实际问题：把"纽约键盘不够打字"这个全体程序员的痛，交给一套包含百万余码点的表去解决。无论你是只用英文字母，还是西班牙文、汉字、emoji，都能有统一的编号。

·初学者误区：看到 bin(ord('Ä'))=='0b11000100' 就以为"Latin-1 就是多字节编码"。恰好相反，Latin-1 仍是单字节；0xc4=196 是一个字节的值。真正的多字节方案要到 UTF-8/16/32 才出现。

2.2.2 字符编码 (Character Encodings)

英文原文

One of the keys to understanding how Unicode works lies in the way its integer character codes (a.k.a. code points) are mapped to their encoded forms for efficient storage or transfer. Code points in memory are j

ust integers of arbitrary size, but storage and transfer, by nature, impose constraints on time, space, and interoperability worthy of extra formatting. In the Unicode world, we say that characters are translated to and from raw bytes using an encoding—the rules for translating a Unicode-text string into a sequence of bytes and extracting the same string from its sequence of bytes. More procedurally, this translation back and forth between bytes and strings is captured in two terms (the first of which doubles as a noun and verb, confusingly!):

- Encoding is the process of translating a string of characters into its raw-bytes form, per any desired encoding that's broad enough to store the string's characters.
- Decoding is the process of translating a string of raw bytes into its character-string form, per the encoding originally used to create the bytes string.

As we've seen, Unicode defines both character code points and a set of standard encodings. For some of the encodings it defines, the translation process is trivial—ASCII and Latin-1, for instance, map each character to a single byte, so little work is required to encode and decode if characters are the same bytes in memory too. You can view this for yourself with the `encode` method available on all Python text strings, which simply returns the bytes used to encode the string. The following means that the ASCII character `a` occupies just one byte when encoded per the ASCII encoding:

```
>>> len('a'.encode('ASCII')) # ASCII 的'a' 编码后仅 1 字节1
```

For other encodings, the mapping can be more complex and yield multiple bytes per character. The widely used UTF-8 encoding, for example, allows more characters to be represented by employing a variable-number-of-bytes scheme that's both general and economical. In fact, because UTF-8 can handle any Unicode code point whatsoever, it's become a de facto standard of sorts for text. In UTF-8, character codes less than 128 are represented as a single byte; codes between 128 and 0x7ff (2047) are turned into two bytes, each byte having a value between 128 and 255; and codes above 0x7ff are turned into three- or four-byte sequences having values between >128. This keeps simple ASCII strings compact, sidesteps byte-ordering issues, and avoids null (zero) bytes that can cause problems for C libraries and networking. In Python:

```
>>> len('a'.encode('UTF-8')) # ASCII: 编码为 1 字节1
```

```
>>> len('Ä'.encode('UTF-8')) # 非 ASCII: 编码为 2 字节2
```

```
>>> len(".".encode('UTF-8')) # emoji: 编码为 4 字节4
```

Despite such details, it's important to note that ASCII is a subset of both Latin-1 and UTF-8. This is true because these encodings encode ASCII characters to bytes the same way ASCII does. That, in turn, makes these encodings backward-compatible with existing ASC

II data: every character string encoded per ASCII is also valid according to the Latin-1 and UTF-8 encodings, and every ASCII file is a valid Latin-1 and UTF-8 file. Technically, the ASCII encoding is a 7-bit subset of the other two: it's binary-compatible with all character codes less than 128. Latin-1 and UTF-8 simply allow for additional characters: Latin-1 for characters mapped to values 128-255 within a byte, and UTF-8 for those they may require multiple bytes. The converse is not true, however: UTF-8 and Latin-1 text is not compatible with the ASCII encoding unless its text's code-point values are all less than 128; otherwise, encoding or decoding per ASCII fails. In Python again, the mapping is easy to observe. Per the following, an ASCII character encodes to the same single byte in ASCII, UTF-8, and Latin-1 encodings, but non-ASCII characters do not, and require more general encodings than ASCII to encode at all (the b'...' here is the Python bytes object, which will soon play a leading role in this chapter):

```
>>>'a'.encode('ASCII') # ASCII 编码为同一个字节b'a'
>>>'a'.encode('UTF-8') b'a'
>>>'a'.encode('Latin-1') b'a'

>>>'Ä'.encode('UTF-8') # 但非 ASCII 需要更多字节b'\xc3\x84'

>>>"".encode('UTF-8') # 并向更通用的编码求助b'\xf0\x9f\x99\x82'
```



```
>>> ''.encode('ASCII') # 纯 ASCII 编不了这个符号UnicodeEncodeError:'ascii' codec can't encode character'\U0001f642'...
```

```
>>> ''.encode('Latin-1') # Latin-1 也一样装不下UnicodeEncodeError:'latin-1' codec can't encode character'\U0001f642'...
```

Other encodings support richer character sets in other ways. For instance, UTF-16 and UTF-32 use a fixed and larger 2 and 4 bytes per character, respectively, the former with a special surrogate-pair protocol for codes too large for 2 bytes. Both of these, along with UTF-8, may also permit or require a BOM (Byte Order Marker) preamble at the start of encoded text, which can designate byte order and encoding type, may be present in encoded text stored in files or memory, and is automatically handled for text-mode files. We'll skip further details here for space (watch for the BOM to recur at the end of this chapter, along with the related thorny topic of Unicode normalization), but keep in mind that all of these—ASCII, Latin-1, UTF-8, and others—are simply alternative Unicode encodings that yield the same Unicode code-point text when decoded. The net effect ensures that text is portable across all the tools that use it in return for modest translation costs:

- When decoded, a character code point may or may not occupy multiple bytes in memory, depending on language implementation. Some recent Pythons, for example, use a variable-length scheme to s

tore decoded text with 1, 2, or 4 bytes per character, depending on string content. Earlier Pythons instead store each character in a fixed 2 or 4 bytes, depending on compilation settings. - When encoded, the format of a character code point is wholly determined by the standard Unicode encoding applied. This format is the same regardless of which programming language creates or processes the text, making it ideal for storage and transmission—especially in the diverse realm of the internet. Your own file system storage, or that kind of format is often less ideal for programs to use, though, hence usually decoded upon loading. To Python programmers, an encoding is specified as a string containing the encoding's name. Python comes with roughly 100 different encodings out of the box; see the codecs module in the Python Library Reference for a list. Importing module encodings and asking for help(encodings) shows you many too. Some encodings are implemented in Python, and some in C, and many have multiple names. As a wide sample, latin-1, iso88591, and 8859 are notably all synonyms of the same encoding, Latin-1. We'll revisit encodings later when we study Unicode coding techniques. For another take on the Unicode backstory, see the Python standard manual at python.org. It includes a Unicode HOWTO section, which provides additional minutiae that we will skip here to focus on the fundamentals.

中文翻译

理解 Unicode 运作方式的关键之一，在于它的整数码点是如何被映射成编码形式以便高效存储或传输的。内存中的码点只是任意长度的整数，但存储和传输天然的施加了它们的时间、空间、互操作性方面的约束，因此这些对象需要额外的格式化步骤。在 Unicode 的世界里，我们说是用一种编码把字符翻译成原始字节以及从原始字节还原字符串的规则。更流程化地说，字节与字符串之间来回翻译由两个术语定义（第一个术语又当名词又当动词，有点容易混淆！）：

- 编码（Encoding）：把一个字符字符串翻译为其原始字节形态的过程，只要任选一种足够宽、能容纳该字符串字符的编码方案。
- 译码（Decoding）：把一个原始字节字符串翻译成其字符字符串形式的过程，用的就是当初生成该字节字符串时所用的编码。

正如我们所见，Unicode 不仅定义了字符码点，还定义了一套标准编码。对于它定义的某些编码而言，转换过程轻而易举——比如 ASCII 和 Latin-1，每个字符只映射成一个字节；假如字符在内存中的字节形式正好相同，那么编码解码都几乎不费功夫。你可以自己观察这个现象：所有 Python 文本字符串上都有 `encode` 方法，它直接返回编码该字符串所用的字节。下面这段表示：ASCII 字符 `a`，按 ASCII 编码就只占一个字节：

```
>>> len('a'.encode('ASCII')) # ASCII 的 a 编码后仅需 1 字节1 而其他编码的映射就可能更为复杂，一个字符对应多个字节。例如广为使用的 UTF-8 编码，通过一种变长字节数的方案，既全面又节约地表达更多字符。事实上，UTF-8 能处理任意一台无 Unicode 码点，所以它已经成了文本处理的事实标准。在 UTF-8 中，小于 128 的字符码用一个字节表示；12
```

8 到 0x7ff (2047) 之间的码点则变为两个字节，每个字节的值都在 128 ~ 255 之间；大于 0x7ff 的码点则变为三字节或四字节的序列，值域同样为 128 ~ 255。这样既让桩筒 ASCII 的字符串保持紧凑，又避开了字节顺序问题，还躲开了可能给 C 库和网络带来麻烦的空字节 (null)。Python 中的实测：

```
>>> len('a'.encode('UTF-8')) # ASCII: 1 个字节1
>>> len('Ä'.encode('UTF-8')) # 非 ASCII: 2 个字节2
>>> len(".".encode('UTF-8')) # emoji: 4 个字节4
```

不过有一件重要的事需要明确：ASCII 是 Latin-1 和 UTF-8 两者的子集。原因在于，这两种编码在存储 ASCII 字符时采用的字节与 ASCII 完全一致。于是它们对既有 ASCII 数据天然向后兼容：所有用 ASCII 编码得到字节的字符串，按 Latin-1 或 UTF-8 也统统合法；ASCII 文件也是合法的 Latin-1 和 UTF-8 文件。严格来说，ASCII 编码是另外两者的 7 位子集：它与所有小于 128 的字符码完全二进制兼容。而 Latin-1 和 UTF-8 只是额外多出了字符：Latin-1 用一字节内的 128 ~ 255 容纳附加字符，UTF-8 则允许这些字符可能占据多字节。但反过来却不成立：UTF-8 和 Latin-1 文本在与 ASCII 编码互通时是兼容的，除非它的所有码点值都小于 128；否则按 ASCII 编码或解码就会失败。用 Python 再现，这个映射一望便知。下面代码表明：同样是 ASCII 字符，用 ASCII、UTF-8、Latin-1 编码都会得到同一个单字节值；但非 ASCII 字符就不是这样，它们至少需要比 ASCII 更宽的编码（这里的 b'...' 就是 Python 的 bytes 对象，它很快就要在本章扮演主角）：

```
>>> 'a'.encode('ASCII') # 编码为同一个字节b'a'
>>> 'a'.encode('UTF-8')
b'a'
>>> 'a'.encode('Latin-1')
b'a'
>>> 'Ä'.encode('UTF-8') # 非 ASCII 却需要更多字节b'\xc3\x84'
>>> ".encod
```

`e('UTF-8') # 而且需要更包容的编码b'\xf0\x9f\x99\x82'>
>>".encode('ASCII')> # ASCII 编不了它UnicodeEncodeError:'ascii' codec can't encode character'\U0001f642'...>>>".encode('Latin-1')> # Latin-1 也编不了UnicodeEncodeError:'latin-1' codec can't encode character'\U0001f642'...`另一些编码用别的方式支持更丰富的字符集。比如 UTF-16 和 UTF-32 每个字符分别固定占用 2 字节和 4 字节，其中前者针对超过 2 字节的码点有一个专门的代理对 (surrogate pair) 协议。这两者再加上 UTF-8，都允许或要求在编码文本开头带上 BOM (Byte Order Marker, 字节序标记) 头，该标记可用于指定字节序和编码类型，可能出现在存储于文件或内存中的编码文本里，文本模式的文件会自动处理它。为了节约篇幅，这里略过更多细节 (请留意 BOM 会在本章结尾再次出现，同场的还有 Unicode 规范化这个棘手的主题)，但请记住：ASCII、Latin-1、UTF-8 以及其它编码，统统是 Unicode 的可选编码，解码后得到的都是同一套 Unicode 码点文本。其效果是：文本只要付出少许转换开销，就能跨所有使用它的工具无障碍流动。- 解码后，字符码点在内存中既可能占用多字节也可能不占多，具体看语言实现。例如近期的某些 Python 版本，会根据字符串内容在 1、2、4 字节之间变长存储解码后的文本；旧版 Python 则视编译配置而定，每个字符固定占 2 或 4 字节。- 编码后，字符码点的格式则完全由所用 Unicode 标准编码决定。这套格式与“创建或处理该文本的编程语言”无关，因此极其适合存储与传输——特别是在五花八门的互联网环境中。不过这种格式对程序本身往往并不友好，所以平时加载进来就会先解码。对 Python 程序员而言，编码用一串字符串形式的编码名即可指定。Python 原

生自带大约 100 种编码，请参见 Python 库参考中 codecs 模块的列表。导入 encodings 模块再向 help(encodings) 请教也同样能列出很多编码名。有的编码用 Python 实现，有的用 C 实现，而且很多编码有多个别名——比如 latin-1、iso88591 和 8859 都是同一个 Latin-1 编码的别名。我们会在本章后面研究 Unicode 编码技巧时再次遇到编码。想了解 Unicode 背景的另一视角，可参阅 python.org 的 Python 标准手册。它含一节"Unicode HOWTO"，讲了更多这里略过的细节，这里只专注抓住核心。

深度理解

·核心概念：编码（encoding）与译码（decoding）是双向相反的转换。编码：str（码点）→ 二进制字节序列；译码：bytes（原始字节）→ 码点文本。两者都是"翻译"，但方向相反。

·底层实现：UTF-8 的变长规则极其优雅：码点 <128 1 字节，且最高位为 0；128~2047 2 字节，首字节 110xxxxx、续字节 10xxxxxx；更高则 3 或 4 字节。这套"前导字节 + 续字节"的结构保证了字节流可以无缝拼接、无歧义，且永不出现空字节——这就是它成为"事实标准"的底层原因。

·设计原因：ASCII 是 Latin-1/UTF-8 都是子集，这是向后兼容的巧妙选择：老数据无需转换即可在新编码里照样读。C 库与网络传输还要求不能有 0x00 空字节，UTF-8 的设计正好绕开该雷区。

·解决什么实际问题：让"中文、日文、带重音的欧洲语言、emoji"都进一套编码体系，且与 C 语言互操作时不出空

字节；反过来让老 ASCII 数据零迁移成本。它也是 JSON、XML、HTTP 等网络协议的地基。

·初学者误区：把 `len()` 与字节数混为一谈。`len('Ä')` 是 1（码点个数），`len('Ä'.encode('utf-8'))` 是 2（字节数）。还有，解码时以为“加宽就没问题”——其实 `.encode('ASCII')` 和 `.encode('latin-1')` 都抛 `UnicodeEncodeError`，只有确能容纳该码点（如 UTF-8/UTF-16）的编码才行。

代码分析（b'...' 出现与 encode 的初见）

```
len('a'.encode('ASCII'))    # 1 ASCII
len('Ä'.encode('UTF-8'))    # 2 ASCII
len('ø'.encode('UTF-8'))    # 4
b'\xc3\x84'                # 'Ä'   UTF-8
b'\xf0\x9f\x99\x82'        # 'ø'   UTF-8 4
```

解释器如何处理：`.encode()` 方法会去查 Fake codec 表中对应编码名的编码器，把 `str` 对象里每个码点按规则重排成字节序列，返回 `bytes`；每次调用都会在堆上新建一个独立的 `bytes` 对象。`'\xc3\x84'` 这种 `b` 前缀字面量经编译器解析为一个 `bytes` 对象，显示时按每个字节的值决定用 ASCII 字符还是十六进制转义打印。

设计思想：把“内存表示”与“入库表示”彻底分开——字符串进内存不用编码，出内存（存文件、发网络）才编码；这正是现代 Python 单字模型稳定跨平台的根本。

37.3 引入 Python 字符串模型 (Introducing the String Types)

英文原文

At a more concrete level, the Python language provides several built-in string types to represent content in your scripts—both textual data—integer code-point values of decoded Unicode characters in memory—as well as binary data—raw byte values, including text that is in encoded form. All told, Python's string story in this chapter rests on three string object types: - `str` —for representing Unicode text (decoded code points) - `bytes` —for representing binary data (including encoded text) - `bytearray` —a mutable flavor of the `bytes` type All three types support similar operations but have very different roles and cannot normally be mixed deliberately. Moreover, files and other content tools bake in this text/binary distinction too and use specific string types in different roles. The next sections introduce the salient points of this model.

中文翻译

在更具体的层面上，Python 语言提供了多个内建的字符串类型，用来在脚本里表示内容——既包括文本数据（内存中已解码的 Unicode 字符的整数码点），也包括二进制数据（原始字节值，包括处于已编码形态的文本）。本章的字符串故事，将围绕 Python 的三种字符串对象类型展开： -

str——用于表示 Unicode 文本（已解码的码点） - bytes
——用于表示二进制数据（包括未解码的内容） - bytearray
y——bytes 类型的可变变体这三种类型支持的运算集合很接近，但角色差异巨大，因而通常不可混用。此外，文件等其它内容工具上也固化着文本/二进制的区分，在不同位置使用特定的字符串类型。下面几节介绍这个模型的要点。

深度理解

- 核心概念：识别 Python 字符串领域三兄弟——str（可读文本）、bytes（原始字节）、bytearray（可变字节）。抓住了这三个类型的定位，许多"为什么这里能用 str、那里必须 bytes"的疑惑就能瞬间解开。
- 底层实现：三者在 CPython 中都是 PyUnicodeObject/PyBytesObject/PyByteArrayObject，只是存储粒度不同：str 存的是码点整数（现代 CPython 按内容紧凑压缩 1/2/4 字节），bytes/bytearray 存 8 位字节数值。
- 设计原因：把"文本"和"字节"拆开，是为了对容易出错的东西进行捆绑校验：比如把图像数据当成文本打印、拼接 str 和 bytes 这些错误，在 Python 3 里都是静默失败变为了类型错误。
- 解决什么实际问题：一套 API 既能服务纯文本处理又能服务网络/文件二进制传输，同时仍能"不可混合"的强约束保证你不会把两点都搞混。
- 初学者误区：以为 str 支持的功能 bytes 一定也支持。其实 bytes 没有 format 方法、不支持 f-string，str 上 encode 是编码、而 bytes 上 decode 是解码，方向相反。

用 `dir()` 对比一下，差异一目了然。

代码分析（用 `dir` 对比 `str` 与 `bytes` 的差异）

本章原书中用以下代码演示如何用 `dir` 找出 `str` 与 `bytes` 的属性差异：

```
sorted(set(dir('abc')) - set(dir(b'abc')))  
# ['casefold', 'encode', 'format', 'encode', 'format_map', 'isdeci...  
  
sorted(set(dir(b'abc')) - set(dir('abc')))  
# ['__buffer__', '__bytes__', 'decode', 'fromhex', 'hex']
```

解释器如何执行：`dir(obj)` 返回对象所有可访问属性名单，`set(...)` 去重，集合差集 - 求"独有"。结果即 `str` 独有属性与 `bytes` 独有属性。这两行代码没有产生新的字符串对象，只是元数据处理。

设计思想：差集结果恰恰揭示了"文本专属"（`encode/format/casefold` 等）与"字节专属"（`decode/hex/fromhex` 等）两条路线的边界，是学习过程中自查记忆的好工具。

37.4 The `str` Object (`str` 对象)

英文原文

First up, the basic `str` type (e.g., `'text'`) is for decoded Unicode text. It's formally defined as an immutable sequence of characters—meaning code points that are not necessarily bytes. Its content may contain both simple text, such as ASCII, whose encoded and decoded

ed forms might yield one byte per character, as well as richer Unicode text, whose encoded and decoded forms may both incur multiple bytes per character. In memory, a `str` is just an ordered collection of Unicode code-point integers, which print as their glyphs—visual representations that may vary from host to host—but in the end are just code points. When transferred to and from files, a `str` is automatically encoded to and decoded from a sequence of bytes, using either the host platform's default or a provided encoding name. `str` objects themselves, though, have no notion of an encoding; they are just character code points.

中文翻译

首先要讲的是基础 `str` 类型（例如 `'text'`），它专门负责已解码的 Unicode 文本。严格定义上讲，它是不可变的字符序列——也就是“码点”序列，而这些码点不一定是字节。它的内容既可以包含简单的文本，比如 ASCII（编码与解码形态每字符都是一个字节），也可以是更丰富的 Unicode 文本（编码与解码形态都可能每字符占多个字节）。在内存中，`str` 只是一段有序的 Unicode 码点整数集合，打印时它们以字形（glyph）显示——字形是随主机而异的可视化呈现——但本质上就只是码点。在与文件往返时，`str` 会自动按主机平台默认或指定的编码名，编码成字节序列或从字节序列解码。而 `str` 对象本身对编码毫无概念；它们只是字符码点。

深度理解

- 核心概念：str 是"解码后"的码点序列，逻辑同 code point，不管它看起来占几个字节。
- 底层实现：CPython 3.x 的 str 内部有很多种紧凑表示：全 ASCII 时每个字符只用 1 字节（省内存），出现 128 以上字符则自动切换到 2 字节甚至 4 字节布局（PEP 393 正则化）。
- 为什么这样设计：让 str 在"打印时像字符、内存中像整数序列"之间自如切换，同时保持字符串的不可变性（hash 稳定，可用作字典键）。
- 解决什么：你要在代码里"说人话"（用中文字符写字符串字面量）时，str 随时托底；文本文件读到的就是 str，不必关心内存占几字节。
- 初学者误区：len() 对 str 返回"字符数"而不"字节数"。这也导致了在 Windows 下 print 一堆中文结果和颜文字时出现"宽字符"困惑——其实 len 永远数的是码点。

37.5 The bytes Object (bytes 对象)

英文原文

While str handles Unicode text with ease, many programs need to process raw binary bytes that do not fall under any Unicode encoding—as well as the bytes used to store bytes when they are encoded. Image files, packed binary data you'd process with the struct module,

dule, JPEG data, and so on, all fall under a purely binary-data view. To accommodate this, the bytes type supports processing of truly binary data. bytes is just raw bytes, not Unicode-text characters, though its content may include the bytes of a still-encoded text, which always has an encoding of its own. The bytes type is formally defined as an immutable sequence of 8-bit integers, its content represents byte values, and it supports nearly all the expression operators and methods of the str type — string methods, sequence operations, and even re pattern-object matching operations (formatting works with bytes—it too was floated in the last half of 3.X's life). A bytes object is a sequence of small integers, each in the range 0...255 by design: indexing a bytes returns an int, slicing a bytes returns another bytes, and running list on one returns a list of ints, not characters. However, many operations that presuppose characters (e.g., the isalpha method) assume the bytes are encoded per ASCII. And for display, bytes items whose values fall in the ASCII 0...127 character-code range print as ASCII-character glyphs rather than integers or hex escapes—a convenience that can further blur the line between text and binary bytes.

中文翻译

虽然 str 将 Unicode 文本处理得心应手，但许多程序需要处理的是不含任何 Unicode 编码的原始二进制字节——以

及用于存储文本的已编码字节。图像文件、用 struct 模块处理的打包二进制数据、以及 JPEG 等，全部落入纯二进制数据的范畴。为了容纳这些场景，bytes 类型支持真正二进制数据的处理。bytes 就是原始字节，不是 Unicode 文本字符；不过其内容也可以包含"仍处于已编码状态"的文本的字节，后者总有一个隐含的编码。bytes 类型正式定义是不可变的 8 位整数序列；它的内容表示字节值，并支持几乎和 str 相同的方法集——包括字符串方法、序列操作，甚至 re 模块的模式匹配（格式化也同样适用于 bytes，这一点也是 3.x 后期才加进来的）。bytes 对象本质上是一组小整数序列，每个整数的取值范围天然限定在 0...255：索引 bytes 返回一个 int，切片 bytes 返回另一个 bytes，对 bytes 调用 list 则得到整数而不是字符。即便如此，许多按"字符"假设的算法（例如 isalpha 方法）也把 bytes 内容当作 ASCII 编码来解读。而且显示层面，bytes 项的值若落在 ASCII 0...127 的范围，打印时就以 ASCII 字符字形呈现，而不是整数或十六进制转义——这个便利固然省事，却也在文本与二进制之间又增添了一层模糊。

深度理解

- 核心概念：bytes 是"字节视角"的字符串——看它的元素，你就是真的个小整数（0-255），并不是一个字符。
- 底层实现：bytes 在 CPython 中是一整块连续内存（类似 C 的字符缓冲区），所以它是紧凑、可整体传给 C 扩展和 socket 的高性能对象；.decode() 时 Python 会按所属编码逐字节求出 str。
- 为什么这样设计：把"字节值"与"字符"两件事彻底分开——

—封面确实像，但内容是 "bytes is NOT text"，杜绝了 Python 2 时代把 GB 图像误当文本打印的坑。

·解决什么实际问题：读 JPEG：`open('cat.jpg','rb').read()` → bytes；断点续传、struct 打包、socket 收发、C 扩展的 buffer 传值，都是 bytes 的天下。

·初学者误区：打印 bytes 时看到 'abc' 就以为它是文本，看到 `\xff\xd8` 就以为 "乱码"。两者都只是 bytes 的 print 显示规则（127 内按 ASCII，其余按十六进制转义），跟 "存的是文本还是图像" 毫无关系。

37.6 The bytearray Object (bytearray 对象)

英文原文

Though less commonly used, Python also comes with bytearray, a variant of bytes that is mutable, and so supports in-place changes. The bytearray type provides the usual string operations that str and bytes provide (methods, indexing, concatenation, slicing, and so on), but also has the mutable in-place change operations that lists provide (e.g., append, extend, and assignment to indexes). So, where your bytes can be treated as raw bytes, bytearray adds direct in-place mutability for string data — something long prohibited for str and bytes.

中文翻译

虽然用得较少，Python 还提供了 `bytearray`——它是 `bytes` 的可变变体，因此支持原地修改。`bytearray` 类型提供 `str` 和 `bytes` 所具有的常规字符串操作（方法、索引、拼接、切片等），同时兼有 `list` 的可变就地修改操作（例如 `append`、`extend` 和按索引赋值）。只要你的字符串可当作原始字节来对待，`bytearray` 就为这类字符串数据带来了直接的就地可变能力——而这一点长期以来对 `str` 和 `bytes` 都是禁止的。

深度理解

- 核心概念：`bytearray` 是"可变 bytes"——系统既有 `bytes` 的序列语义，又有 `list` 的就地修改能力。
- 底层实现：CPython 用可变缓冲（`PyByteArrayObject`）存储；构造时若传入 `int n`（例如 `bytearray(10)`），就会生成 `n` 个零字节的缓冲区。
- 为什么这样设计：处理网络缓存、图像像素、加密输出等需要"原地改字节"而尽量少分配的场景，省去了 `str/bytes` 不可变所带来的反复拷贝。
- 解决实际问题：`data = bytearray(udppacket); data[4:6] = b'\x00\x00'` 这种补丁式修改，比"读出→改→重新写"更省内存、更快，尤其适合需要反复修改缓冲区的高吞吐系统。
- 初学者误区：试图 `bytearray('abc')` 直接拿 `str` 构造——会报 `TypeError`，因为 `bytearray` 只接受 `bytes` 或整数序列；也不要写 `C[0]='x'` 这种 `str` 赋给整数格，应写 `C[0]`

= ord('x')。

37.7 文本文件与二进制文件 (Text and Binary Files)

英文原文

Because file I/O is one of the main benefactors of string types, it's also a core Unicode tool. As we've seen, text is really just decoded integer character codes when it's in memory; it's only when text is transferred to and from interfaces like files that Unicode encodings come into play. Truly binary data, by contrast, may have nothing to do with text. Because of this, Python makes a sharp, platform-independent distinction between text and binary files, which you will grapple with when you open a file with the built-in open function:

- When a file is opened in text mode, reading its data automatically decodes it for you and returns a str, and writing a str automatically encodes it before storing it to the file. In both cases, encoding is either the platform's default or a provided encoding argument. Text-mode files also support "universal newline" translation, BOMs, and other encoding-related arguments.
- When a file is opened in binary mode by adding a b to the mode argument in the open call: reading its data does not decode it in any way and returns its raw content as a bytes object. Writing similarly takes a by

tes and stores it unchanged; binary-mode files also accept a bytearray object for writing. Because `str` and `bytes` are sharply the language's way of distinguishing text and binary, you must decide whether your data is text or binary in nature and use `str` or `bytes` objects, respectively, to represent it. In short, the height of the game is that the mode in which you open a file dictates which type of object your script uses to represent its content:- If you are processing image or audio files, packed data created by other tools whose content you must split, or some device's binary streams, chances are you will need `bytes` and binary-mode files. You might choose `bytearray` to mutate the data without copying it in memory.- If you are more processing textual content, such as program text, HTML or JSON text, CSV or XML files, you want `str` and text-mode files. Subtly, the mode-string argument to `open` is its mode and second positional argument becomes pretty crucial here—its content not only specifies a file processing mode (read, write, etc.), but also implies a string object type for your script's content. By adding a `b` (lowercase only, as in `'b'`) to the mode string, you are specifying a binary-mode file and so the result of reading, or the argument you must supply when writing, will be a `bytes` object. Without the `b`, your file is processed in text mode, and you'll use `str` objects to represent its content. To check a few examples: modes `rb`, `wb`, and `rb+` imply `bytes`, but `r`, `w+`, and the mode

rt (the default, meaning read-text) imply str.

中文翻译

由于文件 I/O 是字符串类型的最大受益者之一，它也是 Unicode 的核心工具。正如我们所见，文本在内存里其实就是“已解码的整数字符码”；只有当文本需要传入传出文件等外部接口时，Unicode 编码才粉墨登场。相比之下，真正的二进制数据可能与编码完全无关——甚至跟文本毫无干系。为此，Python 在平台无关的前提下，使内置的 open 函数对文本文件与二进制文件作出了严格区分：- 当以文本模式打开时，读文件会自动进行解码，并返回一个 str；写文件则接受一个 str 并在写到文件前自动编码。两种情形下使用的编码，要么是平台默认值，要么是传给 open 的 encoding 参数。文本模式文件还支持“通用换行符”（universal newline）转换、BOM 处理以及其它编码相关参数。- 当给 open 的模式字符串加上 b 并以二进制模式打开时：读数据完全不解码，直接把原始内容当作 bytes 对象返回；写数据也接受一个 bytes 对象并原样存入文件；二进制模式文件同样接受 bytearray 对象作为要写的内容。正因为语言把 str 与 bytes 区分得明明白白，所以你必须判断自己的数据是“文本型”还是“二进制型”，并分别用 str 或 bytes 对象加以表示。归根结底，你打开文件的模式，决定了你的脚本用哪种对象类型表示文件内容：- 如果你处理的是图像或音频文件、必须拆解其它程序产生的打包数据、或者某些设备的二进制数据流，多半需要用 bytes 配合二进制模式文件；如果还要就地修改数据而不拷贝内存副本，也可以选用 bytearray。- 如果你处理的是“文本性质”的内容，比如程序输出、HTML 或 JSON 消息、CSV 或 XML 文件，那

么应该用 `str` 配上文本模式文件。微妙的地方在于：`open` 的 `mode` 参数（可通过关键字 `mode` 或第二个位置参数给出）在这里变得意义重大——它的内容不仅指定了文件处理模式（读、写等），还暗示了处理内容时使用的字符串对象类型。如果 `mode` 字符串里带一个小写 `b`，那就是二进制模式文件，读取时你将得到 `bytes`，写入时也必须给出 `bytes`。没有 `b` 则是文本模式，内容一律用 `str` 对象。示例：模式 `rb`、`wb`、`rb+` 意味着 `bytes`，而 `r`、`w+` 以及默认的 `r` `t`（文字读）意味着 `str`。

深度理解

·核心概念：`open` 的 `mode` 字符串是“文件类型的分水岭”：有没有字母 `b`，决定了文件是文本文件还是二进制文件，进而决定你用 `str` 还是 `bytes` 跟它打交道。

·底层实现：CPython 在二进制模式下用最原始的 `read()` 返回内存缓冲区，不经过 `codec`；文本模式下则包了一层 `TextIOWrapper`，读写时悄悄调用 `.encode()/decode()` 并做换行符转换。所以二进制文件写 `\r\n` 就真的是 `\r\n` 两个字节，文本文件则会跟平台玩起“换行符翻译”。

·为什么这样设计：把“文件字节”与“程序文本”之间的转换交给 `interpreter`，而不是逼你每行手动 `encode/decode`。文本模式下你会自动拿到可读的 `str`，二进制模式下你拿到的纯净 `bytes`，绝无歧义。

·解决什么问题：不同平台（Windows/Linux/macOS）换行符不一致（`\r\n` vs `\n`）的灾难性随之解决；同时 `NET` 的正则、`struct`、`pickle` 等二进制工具才能在不破坏字节流的条件下处理文件。

·初学者误区：以为"文本文件也可以用二进制读，一样的"。请看示例：在 Windows 上'wb' 写后'rb' 读 (b'abc\r\n') 会带 \r，而'w'/'r' 文本模式读回来是'abc\n'——模式不同，结果字节就不同。

代码分析

```
open('temp.txt', 'w').write('abc\n')      # str →
open('temp.txt', 'r').read()               # 'abc\n' str
open('temp.txt', 'rb').read()              # b'abc\r\n'Windows ...
open('temp.bin', 'wb').write(b'abc\n')     # bytes
open('temp.bin', 'r').read()               # 'abc\n' \n
open('temp.bin', 'rb').read()              # b'abc\n'
```

解释器如何处理：文本模式打包了一个编解码层，读即 decode、写即 encode，并默认执行通用换行符转换（在 Windows 上 \n 读写要换成/接入 \r\n）。二进制模式直接由底层文件句柄读取，无任何转换。设计思想：同一份字节流，读法不同则语义天差地别——这就是"把它变成类型系统"的守护：模式参数直接把对象类型编码进了 API

。

37.8 Using Text Strings（使用文本字符串）

英文原文

Let's step through a few examples that demo the prior section's string types in action. Our primary focus here is on using the types in their text roles (we'll explore binary roles later). Along the way, you'll see how literals, conversions, and non-ASCII text are coded in

中文翻译

我们用几个示例把上一节的字符串类型拉出来"跑"一遍。这里主要关注它们在文中当"文本"用的角色（二进制角色稍后看）。一路你会看到 Python 如何书写字面量、类型转换以及非 ASCII 文本。

深度理解

- 核心概念：文本字符串（str）保存解码后的 Unicode 码点序列，是人类可读文本的默认类型。
- 底层实现：str 在内存中按灵活表示存储码点；与文件/网络交互时通过编码（encoding）与 bytes 互转。
- 设计原因：把“文本”与“原始字节”分开，避免 Python 2 时代隐式混用造成的混乱。
- 实际问题：读写文件、HTTP、数据库时必须明确编码；处理文本用 str，处理协议/加密/图像用 bytes。
- 初学者误区：把 str 当成“带编码的字节”；编码只在边界（文件、套接字）发生，内存中的 str 已是文本。

37.9 字面量与基本属性 (Literals and Basic Properties)

英文原文

Most Python string objects are created when you call

a built-in function like `str` or `bytes`, when you process file created by `open`, or when you code a literal in your script. For the latter, the usual `'...'` makes a `str`; a unique literal form `b'...'` creates a `bytes` object; and `byte array` objects are made either by calling the same-named function or through some other route.

More formally, all the usual string literals we met in Chapter 7 — `'...'`, `"..."`, and triple-quoted blocks — generate a `str`; adding a `b` or `B` before them changes to `bytes`. The `b'...'` (and equivalently `B'...'`) `bytes` literal acts byte-like a raw string in that it is a raw sort of string; in fact, the two prefixes `b` and `r` (as `br'...'`) can be combined to get a raw `bytes` string whose backslashes are literal:

```
>> B = b'code' # 创建 bytes 对象: 8 位字节 >> S = 'hack' # 创建 str 对象: Unicode 字符 >> type(B), type(S)
(<class'bytes'>, <class'str'>) >> B, S # bytes 也打印得像字符序列 (b'code', 'hack') >> B2 = B"" # bytes 前缀
可用在单双三引号上 xxxx yyyy"" >> B2 b'\nxxxx\nyyyy\n' >> b'A\nB\rC', br'A\nB\rC', rb'A\nB\rC' # 与 raw 字符串组合也生效 (b'A\nB\rC', b'A\\nB\\rC', b'A\\nB\\rC')
```

Once you have a string, all the usual operations from before work, but the results are type-specific (`bytes` yields `ints`, `str` yields `characters`), and immutability still applies:

```
>> B, S # 再看一眼类型 (b'code', 'code') >> B[0], S[0]
# 索引: bytes 返回 int, str 返回字符 (99, 'c') >> B[1:], S
```

```
[1:] # 切片：仍是 bytes 或 str (b'ode','ode') >> list(B), list(S) # list() 生整数的是 bytes ([99, 111, 100, 101], ['h', 'a','c','k']) >> B[0] = 'x' # 二者都不可变 TypeError:'bytes' object does not support item assignment >> S[0] = 'x' TypeError:'str' object does not support item assignment
```

Because they apply to more than text and have some unique behaviors, we'll defer bytearray and more about bytes to a dedicated section later.

中文翻译

大多数 Python 字符串对象诞生于三类途径：调用 str 或 bytes 这类内建函数；处理由 open 打开的文件；或者在你的脚本里写字符串字面量。对于后者，普通的'...'生成 str；而独特的特殊字面量形式 b'...'则用来生成 bytes 对象；bytearray 对象则可以调用同名函数、用多种参数来构造。更规范一点说：第 7 章里讲过的所有字符串字面量形式——'...'、"..."、以及三引号块——生成的都是 str；只要在它们前面加一个 b 或 B，就会变成 bytes。b'...'（等价于 B'...'）字节字面量，从精神上讲与我们之前见过的 r'...'原始字符串非常相像；实际上，这两个前缀可以组合 (br'...')，从而得到一个反斜杠字面量的字节串。>>> B = b'abc' # 创建一个 bytes 对象：8 位字节>>> S = 'abc' # 创建一个 str 对象：Unicode 字符>>> type(B), type(S) (<class 'bytes'>, <class 'str'>) >>> B # 整数的序列，却按 ASCII 字符打印b'abc'>>> S # 码点的序列，按字符字形打印'ack'>>> B2 = B"" # bytes 前缀适用于单/双/三引号xxxx yyy "">>> B2 b'\nxxxx\nyyyy\n'>>> b'A\nB\rC', br'

`A\nB\rC', rb'A\nB\rC' # 原始字符串组合也有效(b'A\nB\rC', b'A\\nB\\rC', b'A\\nB\\rC')` 一旦你拥有了一个字符串，第 7 章学过的所有常规运算对它依然通用，但结果会随类型而异（bytes 的是整数、str 的是字符），而且不可变性依旧适用：

```
>>> B = b'code'>>> S = 'hack'>>> type(B), type(S) # 类型检查(<class'bytes'>, <class'str'>)>>> B[0], S[0] # 索引: bytes 给整数, str 给字符(99,'h')>>> B[1:], S[1:] # 切片: 生成新的 bytes 或 str (b'ode','ack')>>> list(B), list(S) # bytes 其实是整数们([99, 111, 100, 101], ['h','a','c','k'])>>> B[0] = 'x' # 二者都不可变TypeError:'bytes' object does not support item assignment>>> S[0] = 'x'TypeError:'str' object does not support item assignment
```

由于 bytearray 不仅用于文本，行为也颇有些独到之处，我们把它连同更多 bytes 的内容，推迟到本章后面对应的小节再展开。提醒（历史遗留）：Python 3.X 为方便从 2.x 迁移，还特意辨识 Python 2.x 的 Unicode 字符串字面量：2.x 的 `u'...'` 在 3.x 里就是 `'...'` str 字面量的同义词。这很通情理，因为 3.x 的 str 本来就是全 Unicode 的。不过在今天的 3.x 世界里，已经再没有理由使用 `u'...'` 字面量（除非你就是喜欢多余前缀），但在野外你可能仍然会在别人代码里碰到它——毕竟 2.x 的生命周期非常长。

深度理解

·核心概念：字面量决定类型：不加前缀是 str，`b'...'/B'...'` 是 bytes，可再组合 `r/br`。`u'...'` 是历史遗留的同义字面量（3.x 中无用）。

- 底层实现：交互式显示 bytes 使用 ASCII 字符来表示 0-255 的整数（127 以下显示字符，否则十六进制转义），这就解释了为什么 `b'code'` 打印起来跟字符串一样，但里面其实是 `[99,111,100,101]`。
 - 为什么这样设计：保持"可读性"与"精确性"之间的平衡。若全部以 `\xff` 形式打印 bytes 很难读；若全部当字符打印又会愚蠢地把字节误当文本，于是折中。
 - 解决什么实际问题：`list(B)` 能看到字节整数值，`chr(B[0])` 又能还原字符，方便调试二进制作数据抽取与判断。
 - 初学者误区：把 `len(bytes)` 当字符数，或在 bytes 元素赋值时写字符串 `'x'`——都必须改成 `ord('x')`。另外 `'h'[0]` 返回 `'h'` 这样单字符 str，而 `b'code'[0]` 返回整数 99。
-

37.10 字符串类型转换 (String Type Conversions)

英文原文

Terminology aside, the first thing you might notice about Python strings is what they cannot do — str and bytes never mix automatically in operations, and they're not automatically converted or coerced one to the other when passed to functions. In fact, a function that expects an argument to be a str may not accept a bytes (and vice versa), and operators are fully strict:

```
>>>'hack' + b'code'
TypeError: can only concatenate s
```

tr (not "bytes") to str

This is easier to understand if you remember that a text string may be radically different in its encoded and decoded forms, and Python has no idea what the bytes content is: whether it's a text string in some parallel encoding (a codec name), or truly binary data (e.g., a JPEG).

Because of this ambiguity, Python basically requires that you either:

- commit to one type or the other, or
- perform manual conversions with the following tools (encoding-optional means optional):

- > S.encode(encoding?) and bytes?Python.Generally.S, encoding) – Encode a str to a new bytes per an encoding
- > B.decode(encoding?) and str?... Encoding/ +) You can translate str to bytes and back in either order, or encode and decode.

The S.encode() and B.decode() methods, and the file open call we'll look at ahead, use either an explicitly passed expression or a default; the methods' default is always UTF-8 (by contrast, open uses a different default, in the locale module, as you'll see).

```
>> S = 'hack'>> S.encode() # str → bytes: 文本编码为字节 b'hack'>> bytes(S, encoding='ascii') # str → bytes 的另一种写法 b'hack'>> B = b'code'>> B.decode() # bytes → str: 字节解码回文本'code'>> str(B, encoding='ascii') # bytes → str 的另一种写法'code'
```

Putting this to work can solve the original type error:

```
>> S = 'hack' >> B = b'code' >> S.encode('ascii') + B
# bytes + bytes: 文本编码后拼接 b'hackcode' >> S + B
.decode('ascii') # str + str: 码点字符串拼接'hackcode'
```

There are a few smaller but important warnings: the encoding argument to bytes is the only thing that's required ('bytes' call), whether the conversion is done to/from named codecs. Also subtle and potentially dangerous: leaving encoding off of a call to str does NOT default to the encoding of S.de...

Whatever str(B) all' actually returns the print string of the bytes object, not the bytes' decoded string— this is because of an older procedural habit called "strangely, bytes already print as characters".

```
>> bytes(S) # 没有编码参数 → 报错 TypeError: string argument without an encoding >> str(B) # str() 不带编码也合法"b'code'" # 但这是打印字符串, 不是转换! >>
print(str(B)) # Python 打印 bytes 按: ... >> len(str(B))
# 7 >> len(str(B, encoding='ascii')) # 4, 真正的解码 4
```

Also in the defaults department, your various default encodings are available in sys and locale, though they're not always what you expect or anything else. The following assumes default settings on this chapter's names; your results may differ!

```
>> import sys, locale >> sys.platform # 底层平台'win32' >> sys.getdefaultencoding() # 编码方法的默认值 (
```

不是 `str()` 的) `'utf-8'>> locale.getpreferredencoding(False)` # `open()` 的默认: 一个 Latin 超集 `'cp1252'`

As shown, the default encoding for file content accessed by `open` currently lives in the `locale` module (a Windows machine's `cp1252`, on most Unix a `UTF8`). The exact defaults, however, may differ from platform to platform, and can even depend on your locale settings, environment variables, and command-line arguments.

For instance, the defaults in this chapter's macOS example host (where `open` falls back to `UTF-8`) differ from the Windows case above:

```
>> import sys, locale >> sys.platform'darwin'>> sys.getdefaultencoding()'utf-8'>> locale.getpreferredencoding(False)'utf-8'
```

Besides such platform differences, text content you receive from a web page or device may use different encodings at all, making the top-level defaults less than useless. Hence, your programs shouldn't rely on any `open` defaults if portability counts — always pass an explicit encoding to `open`.

Finally, note that encoding and decoding are not mere type conversions, they produce data in substantially different formats — and you want to avoid any blurring of the concept, especially for the richer text in the next section.

中文翻译

术语先放一边，最先引起你注意的也许是字符串做不到的事：`str` 与 `bytes` 绝不会自动混合，把它们传给函数时也不会自动互相转换。比如期待 `str` 参数的函数可能不接受 `bytes`（反之亦然），运算符更是严格：`>>>'hack' + b'code'` 引发 `TypeError: can only concatenate str (not "bytes") to str` 这里容易冤枉它：只要想起一个文本字符串的编码形式与解码形式可能完全不同，Python 根本不知道 `bytes` 里装的是什么——一旦 `bytes` 上的内容是编码文本，其编码未知；它也可能就是与文本八字不合的二进制数据（比如加载的音频文件）。正是因为这种歧义，Python 在大多数情况下要求你：要么坚持只用一种类型，要么用下面这些工具做手动的显式转换（其中有的 `term` 可选参数）：

- `S.encode(encoding)` 与 `bytes(S, encoding)`：把 `str` 对象 `S` 按 `encoding` 编码成一个新的 `bytes`；
- `B.decode(encoding)` 与 `str(B, encoding)`：把 `bytes` 对象 `B` 按 `encoding` 解码成一个新的 `str`。

无论 `.encode()/decode()` 方法还是后面要读到的 `open` 调用，都用“显式传入的编码名”或“默认值”。这些方法的默认值永远是 UTF-8（比较而言，`open` 用的是 `locale` 模块里按平台而变的那个值，应尽量避免它）。

```
>>> S = 'hack'>>> S.encode() # str → bytes: 把文本编码成原始字节b'hack'>>> bytes(S, encoding='ascii') # 另一写法: str → bytes b'hack'>>> B = b'code'>>> B.decode() # bytes → str: 把原始字节解码成文本'code'>>> str(B, encoding='ascii') # 另一写法: bytes → str'code'
```

把上面工具组合起来，就解决了最初的类型错误，让 `bytes` 与 `str` 以“已编码文本”或“已解码文本”的身份和睦相处：>

```
>> S, B ('hack', b'code') >>> S.encode('ascii') + B # b
ytes + bytes (已编码) b'hackcode'>>> S + B.decode(
'ascii') # str + str (码点) 'hackcode'转换默认值上还有
几个朴素提醒。首先, bytes 的 encoding 参数不是可选的
, 虽然 S.encode() (与 B.decode()) 里它是可选的。更微妙
的是: 虽然 str 不像 bytes 那样必须带编码, 但省略编码
调用 str(B) 并不代表它"采用默认编码"——由于 Python
的历史原因, 不带编码的 str() 返回的其实是 bytes 对象的
打印字符串, 不是它解码转换后的 str (这通常不是你想要
的!)。 >>> bytes(S) TypeError: string argument wit
hout an encoding >>> str(B) # str() 没有编码也能运行"
b'code'" # 但结果是打印串, 不是转换! >>> len(str(B))
# 这种"转换"有 7 个字符7 >>> len(str(B, encoding='as
cii')) # 带上 encoding 才是真转换4 关于默认值, 你平台
的各类默认编码可经 sys 与 locale 模块取得, 但别把它们
想得太可靠: >>> import sys, locale >>> sys.platform
# 底层平台: Windows'win32'>>> sys.getdefaultenco
ding() # 方法默认值 (不是 str() 的) 'utf-8'>>> locale.g
etpreferredencoding(False) # open() 的默认: Latin-1
的超集'cp1252'如图所示, open 函数的默认文件编码住在
locale 模块里。本章 Windows 示例用的这台 PC 上, 它
是 cp1252——一个在 Latin-1 基础上加了弯引号等字符的
超集。但在其他平台上默认值可能不同, 甚至技术上还依赖
环境变量、命令行参数和主机上的个别设置。例如本章的
macOS 主机上就是另一番景象——和大多数 Unix 平台一
样, 它的 open 默认是 UTF-8: >>> import sys, locale
>>> sys.platform'darwin'>>> sys.getdefaultencodin
g()'utf-8'>>> locale.getpreferredencoding(False)'utf-
```

8'除了这类平台差异，你还得注意：从一个网页读取的文本内容或其它来源的文本，可能用任何一款只要 Unicode 编码，这会让平台默认值就变得毫无意义。因此，只要你的程序在意可移植性（现在或将来），就不该依赖 open 的默认值——互操作很关键时，请总是给 open 传显式的 encoding。我们后面研读 Unicode 文件时会再温习编码默认值，并学会显式传 encode 的 NB 玩法。此外，请记住：编码和解码并不能混同为简单的类型转换——它们产生的是"截然不同种类"的数据：编码返回"把文本串按某 Unicode 方案变换后得到的字节串"；解码返回"撤销该变换后得到的文本串"。从某种意义上讲它算是一种转换，对于 ASCII 这种简单文本映射可能显得陈词滥调，但对下一节那种更丰富的文本而言，不清分这个，Unicode 世界会让你栽不少跟头。

深度理解

·核心概念：str 与 bytes 绝不自动混用；想串起来必须显式 .encode() / .decode()。这两者是"编码文本"与"解码文本"两种可数据间的搬运，而 typeconversion（转换）并不等价。

·默认值的坑（Py 的重要实践）：str(B) 在没有 encoding 参数时不作反向转换，而直接返回 bytes 对象的 repr 打印（如 "b'code'"）；bytes(S) 更严，没有 encoding 会直接报 TypeError。这正是"显式优于隐式"哲学的另一体现。

·底层实现：S.encode() 真正的默认是 UTF-8，写死在方法内部；而 open() 的默认编码是 locale.getpreferred

ncoding(False)，不同平台各不相同（Windows 常为 cp1252）。这就是为什么 Windows 读到 UTF-8 文件会乱码的核心原因——并非文件坏了。

·解决什么实际问题：这个规则保证了当你处理网络响应、HTTP body、JSON 等"字节文本"时不会无意中把 str 拼进去；也让"未知 bytes"不被轻率当成文字处理。

·初学者误区：以为 str(B) 等于 decode。实际要写 B.decode() 或 str(B,'ascii')。必须记得：str() 与 bytes() 的构造默认行为是不同的，str() 无 encoding 时是"打印格式"，bytes(S) 无 encoding 直接异常。

37.11 在 Python 中编写 Unicode 字符串 (Coding Unicode Strings in Python)

英文原文

Encoding and decoding grow more consequential when you start dealing with non-ASCII Unicode text. To code Unicode characters that are difficult to type on your keyboard, Python string literals support the following escape sequences:- hex escapes — \xNN, where two hex nibbles NN specify a character code as a one-byte (8-bit) numeric value.- Unicode escapes — \uNNNN and \UNNNNNNNNN: the first, lowercase form gives 4 hex digits and designates a 2-byte (16-bit) character code; the uppercase form uses 8 hex digits for a 4-byte (32-bit) code. Importantly, in str objects,

all these escapes refer to a character's code points, not to its encoded bytes. By contrast, bytes objects allow the hex escapes, but hex only, and then the value is a byte value—for text, its encoded form, not the decoded code point.

中文翻译

一旦你开始处理非 ASCII 的 Unicode 文本，编码与解码就变得越发举足轻重。为了在键盘上打出那些难以输入的 Unicode 字符，Python 字符串字面量提供两类转义：- 十六进制 (hex escapes) ——`\xNN`：NN 是两个十六进制数字，指定一个“1 字节 (8 位) 数值”的字符码。- Unicode 转义 (Unicode escapes) ——`\uNNNN` 与 `\UNNNNN`：前者 (小写 u) 用 4 位十六进制数字表示 2 字节 (16 位) 的字符码，后者 (大写 U) 用 8 位十六进制数字表示 4 字节 (32 位) 的码。重要：在 str 对象中，这三种转义给出的都是该字符的码点值，而不是它编码后的字节。相比之下，bytes 对象只允许十六进制转义字节值——若内容是文本，那是它的编码形式，而非解码后的码点。

深度理解

- 核心概念：str 里写 `\xNN`、`\uNNNN`、`\UNNNNNNNN` 时，表达的是“码点”；bytes 里写 `\xNN` 表达的是“字节值”。同一串转义字符，宿主类型不同，语义完全不同。
- 底层实现：str 字面量的转义在源码 tokenizer 阶段就被解析成对应整数码点；bytes 字面量的 `\xNN` 直接填入缓冲区字节。

·为什么：因为 bytes 的逻辑单位是字节（0-255），而 str 的逻辑单位是码点（可大至 4 字节）。一个 8 位十六进制 \U0001F642 根本塞不进字节缓冲区。

·解决什么问题：让你不用切换中文拼音/输入法也能从字面量写任何字符，并在代码里直观看到码点的值。

·初学者误区：b'A\u00C4' 想表达 Ä，结果 bytes 会原样保留反斜杠（还带警告），生成 b'A\\u00C4'——因为 bytes 里 Unicode 转义不是转义，而是字面字面。字符串/字节的对立在这里苛严得可爱。

代码分析

```
S = '\xc4\xe8'                # str  \xNN
print(S)                       # 'Äè' 196224
B = b'\xc4\xe8'               # bytes \xNN
print(B)                      # b'\xc4\xe8'
B.decode('latin-1')            # b'\xc4\xe8' Latin-1 -> ...
B'...' + B.decode('utf-8')     # UTF-8
# UnicodeDecodeError: 'utf-8' codec can't decode byte 0xc4 .....
```

解释器如何处理：编译阶段，str 字面量中的 \xNN\uN NNN 由解析器先把“转义序列”转换成整数码点，再存入字符串对象；bytes 字面量中的 \xNN 则直接写进底层字节缓冲区。显示时 str 直接按字形（glyph）输出，bytes 对非 127 则按 \xNN 循环打印。设计思想：谁存的是字节、谁是码点，从字面量这一步步就确立——\u 只有在能放下巨大码点的载体（str）里才是转义。

37.12 源文件的编码字段 (Source-Code Encoding Directives)

英文原文

中文翻译

一旦你需要频繁地在代码字符串中使用非 ASCII 文本，或只要你不需借助转义序列，把 Unicode 源文件保存于编辑器某个非 ASCII 编码下，Python 也可以支持：它默认用 UTF-8 解码源代码文件，但可以用一个"编码声明注释"改变之。声明必须写在第 1 或 2 行，形式通常是：`# -- coding: latin-1` --有了这个注释，Python 就会正确地把文件内容（字符串字面量等等）按照指定编码解码。这大大简化了"想要在代码里直接粘贴中文注释"的日常场景。其他细节：Python 官方文档还提到源文件编码声明与使用者文件类型、BOM 等更多高级主题——这里机器学习与我们不强求。

```
#!/usr/bin/env python # -- coding: utf-8 --注意，#!/s
hebang 行允许出现在编码声明之前。只要两个 # 注释贴
合规则，顺序合法。Example 37-1. source-encoding-la
tin1.py python # -- coding: latin-1 -- # 学校演示：编码
非 ASCII 文本的多种方式 + 源编码 # 若本文件保存为 Lati
n-1 文本，则以下代码一例不差：# 若把上面的 coding
行改成 ASCII 或 UTF-8，就会因 myStr1 的取值(0xc4 #
和 0xe8) 不在范围内而失败。# # 若把文件保存成 UTF-8
，则内容仍可用（UTF-8 是源码默认编码），# 且上面的
coding 行可以省略。ASCII 文本更是它们的共同子集。my
Str1 ='AÄBèC' # 原样嵌入，按源编码myStr2 ='A\xc4B\
```

```

xe8C' # 十六进制码点转义myStr3 ='A\u00c4B\U000000
e8C' # Unicode 短/长转义myStr4 ='A' + chr(0xC4) + 'B'
+ chr(0xE8) + 'C' # 码点拼接import sys, locale print('系
统宿主平台:', sys.platform) print('系统默认编码:', sys.get
defaultencoding()) print('open 默认编码:', locale.getpr
eferredencoding(False)) for aStr in (myStr1, myStr2,
myStr3, myStr4): print(f'{aStr}, 长度={len(aStr)}', end=
',')bytes1 = aStr.encode() # 默认 UTF-8: 重音字符各 2
字节bytes2 = aStr.encode('latin-1') # 显式 Latin-1: 每
字符 1 字节# bytes3 = aStr.encode('ascii') # ASCII 会失
败: 字符不在 0..127print(f'字节长度1={len(bytes1)}, 字
节长度2={len(bytes2)}')运行本脚本 (例如以 Latin-1 编码
保存) :

```

\$ python3 source-encoding-latin1.py

系统宿主平台: darwin 系统默认编码: utf-8 open 默认编
码: UTF-8 ÄBèC, strlen=5, byteslen1=7, byteslen2=5
(四行输出结果相同: strlen 5=码点, byteslen1=7 是 UT
F-8 这里的 7 个字节, byteslen2=5 是 Latin-1 的 5 个字
节) 若你希望使用 UTF-8 编源码, 先把其保存为 UTF-8 (在
编辑器中或用类似下列代码重新保存) : python

```

text = open('source-encoding-latin1.py', encoding='
latin1').read()

```

```

open('source-encoding-utf8.py','w', encoding='utf8').
write(text) 然后把第一行注释改成 # -- coding: UTF-8 --
或者干脆删除。这样两个版本源码文件仅化于那个字面量的
存放形式。macOS 上的 diff 可看出差异 (Windows 操 fc
) :

```

\$ diff source-encoding-latin1.py source-encoding-utf8.py

```
1c1 < # -- coding: Latin-1 -- --- > # -- coding: UTF-8
-- 13c13 < myStr1 ='A?B?C'--- > myStr1 ='AÄBèC' Mixed variable
Unicode note: 变量名规则比文件内容更严格。一行汇总：数字或字母类字符可用于变量名，符号和表情则不行。例如 hÄck 合法，但 hâ? 不合法 (isidentifier() 可查)。即便如此，非 ASCII 变量名会在某些键盘和平台引发麻烦，标准库因此强制只用 ASCII 标识符。总之：慎用。
```

英文原文

Unicode escapes work, but they're a bit tedious when you need to code lots of non-ASCII text in strings. For string literals and other text embedded in your script files, Python uses the UTF-8 encoding by default to read your code's text, but allows you to change this per file—to an arbitrary encoding—with an encoding declaration comment. This is useful when you want to use Unicode characters in your pathbar... In the common case, the special comment appears on either the first or second line: `# -- coding: latin-1 --` see Appendix A on `#!` shebang. When present, Python searches for any text in your code in that encoding, so you can create files in an editor that directly writes such characters, and Python will correctly decode them when reading your code. Example 37-1 demonstrates a source file saved with a Latin-1 encoding declaration... (其

我特意注意到：该节后续还有在 UTF-16/UTF-32 下 str 中 \u 转义的二、三字节情况，以及 bytes 字面量只支持 hex，出现 Unicode escape 会原样保留并报 SyntaxWarning。这些前一节已详聊，此处不再设二级标题重复。

中文翻译

（本节英文见上；源文件编码与 Unicode 字面量相关说明已在前文中文部分覆盖。）

深度理解

- 核心概念：源文件编码声明告诉解释器如何把 .py 文件的字节解码为源字符（默认 UTF-8）。
 - 底层实现：解释器读源文件时识别 coding cookie（如 # -- coding: utf-8 --）或 BOM，再据此解码。
 - 设计原因：让源码能直接包含非 ASCII 标识符与字符串字面量，同时保持文件为普通文本。
 - 实际问题：团队应统一 UTF-8；声明与编辑器实际保存编码不一致会导致语法/字符串错误。
 - 初学者误区：以为字符串字面量的内容编码和源文件编码是两件完全无关的事——字面量文本首先要能被源文件编码正确读入。
-

37.13 Using Byte Strings (使用字节字符串)

英文原文

We'll be able to see strings in action again when we study files ahead. First, though, let's take a brief side trip to dig a bit deeper into the operation sets provided by objects designed for binary data — the bytes type and the bytearray mutable kin. While these types can legally hold encoded text too, their scope is much broader: anything that can be stored as bytes—and that's most digital content—works. As mentioned earlier, Python's bytes type supports sequence operations and most of the same methods as str. Even so, because you cannot mix and match bytes and str without explicit conversions, you'll generally use str objects and text files for text and bytes objects and binary files for bytes (we'll meet files real soon). This makes bytes a crucial tool in many roles and worth a quick demo.

Methods (方法) To see what attributes str has that bytes doesn't (and vice versa), you can always check their dir listings (review: $\text{set}(X) - \text{set}(Y) = \text{items in } X \text{ not in } Y$). This can also be informative about expression operators as attributes (e.g., mod and rmod implement %)—as well as find format among str's extras:


```
>>> sorted(set(dir('abc')) - set(dir(b'abc')))) ['casefold', 'encode', 'format', 'formatmap', 'isdecimal', 'isidentifier', 'isnumeric', 'isprintable']
```

```
>>> sorted(set(dir(b'abc')) - set(dir('abc')))) ['buffer', 'bytes', 'decode', 'fromhex', 'hex']
```

As you can see, str and bytes have nearly identical makeup: their extra attributes are generally the encode/decode combination, plus formatting and unicode-identification methods (bytes has none of these, since it has no idea of Unicode). For example, decode translates a raw bytes into an str, and encode translates an str into its raw bytes. Similarly, hex makes a hex string of bytes, fromhex does the reverse. Most other methods are shared—for instance, find, replace, upper, slicing that or known format. But remember: bytes are immutable, though, and operations return new objects:

```
>>> B = b'code' # b'...' bytes 字面量
>>> B.find(b'od') # 在 bytes 中找子串位置1
```

```
>>> B.replace(b'od', b'X Y') b'cXYe' # 返回新 bytes, 原 B 不变
```

```
>>> B # 验证 B 没变b'code'
```

```
>>> B[0] = 'x' # 不可变 → 报错TypeError:'bytes' object does not support item assignment
```

For more bytes-methods-flavored coverage, see Chapter 7's string coverage; the bytes type does much th

e same work.

中文翻译

想知道 str 有哪些 bytes 没有的属性（反之亦然），可以查它们的 `dir()` 名单（温习：`set(X) - set(Y)` 表示在 X 而不在 Y 的项）。这个方法还能看到运算符对应的属性（例如 % 由 `mod / rmod` 实现），以及定位 str 独有的 `format`：
>>> sorted(set(dir('abc')) - set(dir(b'abc'))) ['casefold', 'encode', 'format', 'formatmap', 'isdecimal', 'isidentifier', 'isnumeric', 'isprintable'] >>> sorted(set(dir(b'abc')) - set(dir('abc'))) ['buffer', 'bytes', 'decode', 'fromhex', 'hex']
可以看到 str 与 bytes 属性几乎相同：差异一般集中在 “encode/decode” 这对逆向方法，以及 add 的几个 Unicode 相关 extra（bytes 对 Unicode 毫无概念，所以没有它们）。除此 types 共享绝大部分方法，比如 `find`、`replace`、切片游标、`upper` 等。但请注意 bytes 不可变，这些 bytes 修改类方法都返回新对象：
>>> B = b'code' >>> B.find(b'od') # 在 bytes 中搜索子串：偏移量1 >>> B.replace(b'od', b'XY') # 替换 → 返回新 bytes b'cXYe' >>> B # 原对象不变（不可变）b'code' >>> B[0] = 'x' # 赋值会抛异常 TypeError: 'bytes' object does not support item assignment
若需要更多 bytes 方法细节，去翻第 7 章的字符串基础即可——bytes 能做的事与 str 大体相同。

深度理解

·核心概念：str 与 bytes 的属性差集告诉你：encode/decode 是互斥的翻译接口（str 只有 encode，bytes 只有 decode），终将拆散“文本与字节”的界线。

- 底层实现：B.find(b'od') 在 bytes 内部是一次 memmem/线性扫描字节流；B.replace() 则先在堆上新建字节缓冲区填好新内容再返回——所以返回的是新对象，B 不变，符合不可变语义。
- 为什么设计：方法引擎共享，就是为了让你的大脑里只需记一套字符串 API (find、replace、split...)，只是输入输出类型由宿主 (str/bytes) 决定。
- 解决什么实际问题：处理 Loca 协议数据、syslog 消息、数据库的原始字节，都能直接复用你已掌握的字符串 API，而无需专门学一套 bytes 专有方法。
- 初学者误区：由于 bytes 也显示 ASCII 字符，初学者常忘 bytes 是 immutable。错误 B[0]='x' 报 TypeError 是常态；若真想改，请转成 bytearray 或先 ord 再造字节。

37.14 序列操作 (Sequence Operations)

英文原文

Beyond methods, all the familiar generic sequence operations — indexing, slicing, concatenation, repetition, membership, and iteration — work as expected on both str and bytes, because they're both sequences. As we've seen, str is a sequence of characters, bytes of byte-size integers, but the sequence operations behave the same.

Note, though, that indexing bytes returns an int (它的字节值), whereas indexing str returns ... well, in the

byte-world you're more likely to want the byte's integer value. As you can see in actions, the bytes result is very clearly integer-y...

```
>> B = b'code'>> B[0], B[-1] # 索引: 返回字节的整数值 (99, 101) >> 'code'[0] # 索引 str 返回单字符 str'c'>> chr(B[0]) # 把该整数码点变成字符'c'>> list(B) # 但本质是整数 bytes [99, 111, 100, 101] >> b'A\x42C\xff\x63' # 含十进制的字节字面量 b'ABC\xff'>> chr(0x63), hex(B[0]) # 都能取 ('c','0x63') >> B[1:], B[:-1] # 切片: 返回 bytes (仍按 ASCII 显示) (b'ode', b'cod') >> len(B) # 长度是字节个数 (不是字符数) 4 >> B + b'lmn' # 拼接: bytes b'code' + b'lmn'>> B * 4 # 重复: bytes b'codecodecodecode'
```

中文翻译

除了方法调用，其它所有通用序列操作——索引、切片、拼接、重复——对 str 和 bytes 都像 list 一样正常工作。本质由于二者都是序列。正如我们所学，str 是码点序列 (code points)、bytes 是字节整数序列，但序列操作的语义完全一致。请注意：索引 bytes 返回的是一个 int (字节的二进制值)；而索引 str 返回字符。bytes 真的是一列 0-255 的 8 位整数，只是显示的时候，出现在 0-127 范围内的字节会显示成它们对应的 ASCII 字符，超出则显示成十六进制转义：>>> B = b'code' # bytes 与 str 都是序列>>> B[0], B[-1] # 索引: 返回字节的整数值(99, 101) >>> 'code'[0] # 索引 str 返回 1-字符的 str'c'>>> chr(B[0]) # 字节值→字符'c'>>> list(B) # 但它真是一列整数 bytes [9

```
9, 111, 100, 101] >>> b'A\x42C\xff\x63' # 0x42 是'B'
, 0x63 是'c'b'ABC\xffc'>>> chr(0x63), hex(B[0]) # 取 i
nt 和 hex 都 ok ('c','0x63') >>> B[1:], B[:-1] # 切片: by
tes (0-127 按 ASCII 显示) (b'ode', b'cod') >>> len(B)
# 长度: 字节的数量 (≠ 字符) 4 >>> B + b'lmn' # 拼接
: bytes b'codeImn'>>> B * 4 # 重复: bytes b'codecode
e codecode'
```

深度理解

·核心概念: bytes 是"int 序列", str 是"code point 序列"。同一套模块式序列操作 ([0]、[1:]、+、*、len...) 在两类型下合法一致的虚函数接口; 到返回值上才露出类型真身。

·底层: 索引 bytes 走 PyBytesGetItem() 把缓冲区那个字节存成新的 int 对象; 切片则走"slice 记法"创建子 .bytes (引用原 buffer + 偏移)。所以 B[1:] 构造出一个新对象而不是改 B。

·显示之迷: 打印 bytes 时竟然呈现出'codeImn' 里 (99, 111, 100, 101) 的字符形态——因为 bytes 的 repr 采用 ASCII 友好方案: 0-127 的字节渲染成字符, 128-255 渲染成 \xNN, 整卷空的显示有助读写但也因此在"字节与文本"之间制造了某种幻觉。

·解决问题: 用 bytes 你才能不抖地处理任意字节流, 不被"编码解码"干扰; 这正是网络、密码学、文件系统元数据的地基。

·误区: 见到 b'\xff\xd8\xff\xe0' 就紧张"是不是坏文件"

——它只是 JPEG 头字节的显示形式，一切正常。约定：
str 才是文本的表示，看到 b'...' 就看字节序列。

37.15 格式化 (Formatting)

英文原文

One notable exception to the same-operations rule for the string types: the % expression formatting works on bytes objects as well (from Python 3.5), but neither the format method nor f-string formatting is available for bytes — an odd bifurcation that seems to remember neither of the format-file flavors it owns:

```
>> b'my %s string' % b'fine'b'my fine string'>>'my {  
} string'.format('fine')'a fine string' # str.format ok >>  
b'my {} string'.format(b'fine') AttributeError:'bytes' ob  
ject has no attribute'format'>> f'my {kind} string' # f-  
string 仅对 str'my fine string'
```

There are arguably sound reasons bytes formatting shouldn't have existed at all — bytes can be about as easily raw bytes as ASCII text, and there's no concept of a format string in bytes world; CF is just bytes. But it worked for the old '%' style, so it's there (and mutable bytes end to leave it around).

The odd thing is that post-operation format and f-string weren't similarly extended. The story: --- The formatting functionality is a historical mountain that has

split.

As of 3.5, % works on bytes; str.format and f'' only work on str; and neither format on a decoded str... Encoding and decoding are not the same as byte-operations; avoid the temptation to do "encode-then-format", which fails if you use a narrow encoding:

```
>> kind = 'AÄBèC'>> f'a {kind} string'.encode('ascii')
UnicodeEncodeError:'ascii' codec can't encode character
```

中文翻译

上述“同一套操作”规则中值得一提的反例：%（旧式）格式化从 Python 3.5 起在 bytes 上也能用了；但 format 方法与 f-string 对 bytes 并不提供——一个相当拧巴的分裂，仿佛忘了格式化本来就是一类工具却分出了多种形态。范例：
>>> b'a %s string' % b'fine' # 3.5+ 的 bytes 也能用 % 格式化
b'a fine string'>>> b'a %s string' % bytes([0xFF, 0xFE]) # 非 ASCII 字节也行
b'a \xff\xfe string'>>> 'a {} string'.format('fine') # 但 .format() 只给 str
'a fine string'>>> b'a {} string'.format(b'fine') # bytes 没有 format 方法
AttributeError:'bytes' object has no attribute 'format'>>> kind = 'fine'>>> f'a {kind} string' # f-string 也仅限 str
'a fine string'>>> bf'a {kind} string' SyntaxError: invalid syntax
在 % 语法之外，bytes 不支持 format / f-string 其实在理——bytes 里没有现行的编码语义（它是任意字节），把 ASCII 塞进一个已 UTF-16 编码的字符

串，或一张装载图像的 JPEG，意义全无。但鉴于 % 一直能凑合，这个不一致就被保留了；而且"先格式后编码"也救不了（取决于编码局限性）：

```
>>> kind='AÄBèC'>>> f'a {kind} string'.encode('ascii') UnicodeEncodeError: 'ascii' codec can't encode character'\xc4' in position 3
```

深度理解

- 核心概念：print 向我们展示了 %（给 bytes 保留）、.format()（仅 str）与 f-string（仅 str）三分天下的历史局面。现代新代码请优先 f-string，无需 bytes 时避免 %。
- 底层：% 格式化实现调用 C 层 PyBytesFormat（bytes 专属），.format() 是 str 子类方法（bytes 上没有实现）；f-string 由语法编译期处理，只可能作用在 str 值上。
- 为何这样分裂：现代 Python（3.5+）意外地只把 % 扩展到 bytes（历史兼容需求，网络协议常用），格式化 was 没有紧随而至。事实上正如文中所说，raw bytes 格式化需求本就是少数。
- 实际应用提示：如果您真想用非 ASCII 构造出 bytes，推荐 f'...'.encode() 之后再入管道，除非你的 bytes 里全是 ASCII。而且不要用 narrow encoding（ascii/ latin-1）去调 .encode('ascii') 除非你确定全是 ASCII——否则一定会 UnicodeEncodeError。
- 易误点：bf"..." 这种过于超额的写法并不合法；bytes 与 f-string 的组合在语法层面就是不存在的（流畅的同学心态可以放平）。

37.16 构造 bytes 的其他方式 (Other Ways to Make bytes)

英文原文

So far we've been making bytes objects with the literal `b'...'` syntax. They can also be constructed by calling `bytes` with a `str` object and an encoding name, by calling `bytes` with a series of integers that represent byte values, or by invoking the `encode` method on a `str`. We saw some of these earlier during conversions, but they're more general than that: `>>> B = b'abc'>>> B = bytes('abc','ascii') # str + 编码 → bytes>>> B = bytes([97, 98, 99]) # 整数序列 → bytes>>> B = 'abc'.encode() # str.encode()>>> S = B.decode() # 或 str(B,'ascii')` Note in particular the last two can be viewed as `str` `bytes` converters, as discussed further in the next section.

中文翻译

迄今为止我们造 `bytes` 一直用 `b'...'` 字面量。其实还可以用三种构造法：`bytes('abc','ascii')` (`str` 加编码名)、`bytes([97, 98, 99])` (字节整数的可迭代序列)、以及`'code'.encode()` (`str` 的方法)。前面的转换小节已见其一，但这些工具远比那种简单转换更通用：`>>> B = b'abc' # 字面量>>> B = bytes('abc','ascii') # str + 编码b'abc'>>> B = bytes([97, 98, 99]) # 整数序列 (0-255) b'abc'>>> B =`

`b'abc'` # 字面量
`>>> B = bytes('abc','ascii') >>> B = 'code'.encode()` # 等同 `bytes()`
`>>> S = B.decode()` # 等同 `str(B,'ascii')`
解析中文提示：你可能惊讶 `bytes(115)` 得到 115 个空字节；`bytes(999)` 则触发 `ValueError` 因为它超过了 0..255 的范围。换句话说：`bytes(n)` 是"分配 n 个零字节"，`bytes([n])` 才是"造一个值为 n 的字节"。

深度理解

·核心概念：`bytes` 构造有四种路径：字面量、`str`+编码、整数序列、`.encode()`。它们殊途同归，最终都产生一模一样的 `bytes` 对象。

·底层：`bytes([1,2,3])` 则由 C 层一次库序列化，逐整数校验 0-255 范围后填入缓冲区；`bytes('abc','ascii')` 走编码器；字节整数的范围不当会提高 `ValueError`，这是保护。

·纠错：`bytes(115)` 不是"bytes 值 115"，而是 115 个 0 字节——这是 Python 一个容易的坑（把 `int` 当长度了）。原谅历史沿着 Py2 的 `bytes(N)` 惯例，Python 3 依旧这样定义。当然是"意外行为"。

·设计：这些多样性让 `bytes` 能轻松来自：给网络库送字节数组、从 C API 取字节缓冲、乃至从整数序列拼出二进制协议帧。

·易错：`bytes(999)` 抛 `ValueError` 报"must be in range(0,256)"，因此保留 0-255 之外要想 `for required` 必须用 `bytes([999])` 这类显式序列，例如 `bytes(range(97,104))` 可以很方便造 `b'abc'`。

代码分析

```
bytes(115)                # 115 char 115
# b'\x00\x00\x00\x00\x00\x00...115 '
bytes([115])              # 115 b's'
bytes(range(97, 104))     # 97..103 → b'abcdefg'
bytes(c + 97 for c in range(7)) #
```

解释器执行：内置 bytes 的签名 bytes(int or iterable[, encoding])——传单个整数被识别为"要分配多少个字节"；传序列/生成器则被识别为"逐项收集字节值"。符号不同，落库语义完全不同。设计思想：双义构造函数出自 C++ 式"重载"思路，Python 靠参数形态区分。务必分清楚 bytes(115) 与 bytes([115])。

37.17 类型混用 (Mixing String Types)

英文原文

When we sampled replace in the prior examples, you may have noticed we passed two bytes objects for the from and to args — str types won't do. In general, Python requires specific types in these method calls and expects manual conversions if you want to convert:

```
>> B = b'code'>> B.replace('od','XY') # 传 str 会报 TypeError
TypeError: a bytes-like object is required, not 'str'>> B.replace(b'od', b'XY') # 换 bytes 就正常 b'cXYe'
>> B.replace(bytes('od'), bytes('XY')) # 没有编码参数的 bytes() 报错
TypeError: string argument without an
```

```
encoding >> B.replace(bytes('od','ascii'), bytes('XY','utf-8')) # 显式转换成功 b'cXYe'
```

The same holds for mixed-type expressions: you should try to keep your text and binary data separate, but if you must mix, usually do so by explicit conversion :

```
>> b'ab' + 'cd' TypeError: can't concat str to bytes >>
b'ab'.decode() + 'cd' # bytes → str 'abcd' >> b'ab' + 'cd'
.encode() # str → bytes b'abcd'
```

中文翻译

前文演示 `replace` 时, 你也许注意到 `from/to` 必须传 `bytes` 对象——`str` 可不行。更一般地说, 函数与方法调用往往要求特定字符串类型; 想打破铁律, 就得手动转型: >>> `B.replace('od','XY')` `TypeError: a bytes-like object is required, not 'str'` >>> `B.replace(b'od', b'XY')` `b'cXYe'` >>> `B.replace(bytes('od'), bytes('XY'))` # 忘了编码参数的 `bytes()` `TypeError: string argument without an encoding` >>> `B.replace(bytes('od','ascii'), bytes('XY','utf-8'))` `b'cXYe'` 能混合的表达式同理: 强烈建议让文本与二进制 "各走各道", 若非得混, 多数情况也要显式转换: >>> `b'ab' + 'cd'` `TypeError: can't concat str to bytes` >>> `b'ab'.decode() + 'cd'` # `bytes`→`str` 再连 `'abcd'` >>> `b'ab' + 'cd'.encode()` # `str`→`bytes` 再连 `b'abcd'`

深度理解

·核心概念：类型安全是这个语言的一等公民——在操作前后时代 Python 坚持"宁可异常不可猜编码"的哲学。违反即 `TypeError/ValueError`。

·底层实现：`+` 与 `replace` 都会先在 C 层检查两个对象的类型标签 (`PyUnicodeCheck/PyBytesCheck`)，一旦发现主、次对象跨类型，立刻以 `TypeError` 拒绝而不是隐式转换（这是一个严格的显式优于隐式的设计。）

·为什么：`bytes` 可能是图片、密文、音频，也可能才是"某编码的文本"，Python 无从判断其默认编码——宁可崩溃也不猜。这对性能与安全性（无歧义）等好处。

·实际指导：协议处理和日志读取中，遇到 `b"..."` 一律先 `.decode('编码')` 再拼接或正则；确保在接口边界一处集中编/解码，这种事常发生在 I/O 层。让界限清晰可控。

·初学者误区：把发生在 `str` 与 `bytes` 之间的 `+` 看小；或者在 `bytes()` 转换时漏掉 `encoding` 参数 (`bytes('od')` 系统报)。→ 编码 = 指名道姓永远用 `bytes('od','ascii')`。

37.18 The bytearray Object (bytearray 对象)

英文原文

`bytearray` is essentially a mutable variant of `bytes`: a mutable sequence of integers in the range 0..255, so i

t supports all the same methods and sequence operations as bytes, plus the in-place mutable operations lists have. Let's take a quick tour of its special flavor.

First off, you can call bytearray as a function passing a bytes (or int) — but notice you must not pass a str unless it plus an encoding:

```
>> C = bytearray(b'abc') # 可变的 bytes >> C bytearray(b'abc') >> C[0] # 索引: 整数 97 >> C[0] = ord('x') # 就地修改入字节 >> C bytearray(b'xbc') >> C[0] = b'x' # 不能赋 str, 也不能赋 bytes ValueError; # TypeError: 'bytes' object cannot be interpreted as an integer >> C[1] = b'Y'[0] # 从 bytes 取整数再赋值 >> C bytearray(b'xYc')
```

""" The bytearray's method list overlaps with the bytes + list mutation ops. Things like append, extend, clear, copy, insert or pop and index-assignment, and slicing assignments all work in-place:

```
>> C = bytearray(b'xYde') >> C.append(b'LMN'[0]) # 追加一个字节 >> C.extend(b'NO') # 扩列一批 >> C + b'!#' # 序列运算照旧 bytearray(b'xYdeLMNO!#') >> C.replace(b'xY', b'co') # 方法照旧 bytearray(b'codeLMNO') >> C * 4 # 乘法照旧 bytearray(b'xYdeLMNOxYdeLMNOxYdeLMNOxYdeLMNO')
```

And you can make a bytes or bytearray from any iterable of ints in the 0..255 range (not just from text), including generators:

```
>> bytes([1, 2, 3, 4]), bytearray([1, 2, 3, 4]) (b'...', bytearray(b'...')) >> bytes([115]) # 一个 int → bytes b's' >> bytes([999]) # 超范围 ValueError: bytes must be in range(0, 256) >> bytes(range(97, 104)) # 生成器也可以 b'abcdefg'
```

In conclusion, bytes, bytearray, and str are "integers" vs "characters" respectively: bytes and bytearray are sequences of 8-bit integers; str is a sequence of decoded code point characters.

中文翻译

bytearray 是 bytes 的可变变体：一个整数取值 0..255 的可变序列。它支持 bytes 同款方法与序列操作，还额外获得 list 的就地修改操作。来，快速看看它的独特味道。

```
>> C = bytearray(b'abc') # 用 bytes (或 int 序列) 构造
>> C bytearray(b'abc') >>> C[0] # 索引也是整数 97
>> C[0] = ord('x') # 就地改字节：必须给整数
>>> C bytearray(b'xbc') >>> C[0] = 'x' # 给 str → ValueError
>>> C[0] = b'x' # 给 bytes → TypeError
TypeError: 'bytes' object cannot be interpreted as an integer
>>> C[1] = b'Y'[0] # 正确姿势：bytes 里取整数
>>> C bytearray(b'xYc')
bytearray 的"方法集合" ≈ bytes 方法 + list 的就地修改方法：append / extend / clear / copy / insert / pop / remove / reverse 和无师自通的 setitem (按索引赋值)、delitem (按索引删除) 等。
```

```
>>> C = bytearray(b'xYde')
>>> C.append(b'LMN'[0]) # 追加 1 字节
>>> C.extend(b'NO') # 整个批量进
>>> C + b'!#' bytearray(b'xYdeLMNO!#')
>>> C.replace(b'xY', b'co') # 方法与
```

bytes 相同 bytearray(b'codeLMNO') >>> C 4 # 重复运算照旧 bytearray(b'xYdeLMNOxYdeLMNOxYdeLMNOxYdeLMNO') 也可以由任意 0..255 的整数序列或生成器构造 bytes/bytearray (不限于文本) : >>> bytes([1, 2, 3, 4]) b'\x01\x02\x03\x04' >>> bytearray([1, 2, 3, 4]) bytearray(b'\x01\x02\x03\x04') # int → bytes: 只能单字节 (0..255) >>> bytes([115]) b's'# bytes → int: 索引出来就是整数 >>> bytes([115])[0] 115 >>> bytes([999]) # 越界整数 → 报错 ValueError: bytes must be in range (0, 256) >>> bytes(115) # 注意这是 115 个 0 字节! b'\x00\x00\x00\x00...' (很长) '>>> bytes(range(97, 104)) , bytes(c + 97 for c in list('abcdef')) (b'abcdef', b'abcdef')

深度理解

·核心概念: bytearray = "可增删改版本的 bytes"。它把 list 的就地修改能力并入字节序列, 是 bytes 与 list(index) 间的桥梁类型。

·底层实现: 背后是一段可扩展的字节缓冲区; append/extend 就地 memmove, 不像 bytes 重建整个对象。若用 bytearray 按字节拼搭协议帧, 反复 insert/delete 只涉及一块内存, 不产生 GC 碎片。

·为何需要: 频繁操作二进制数据时 (例如构造 HTTP 头、图像像素扫描、墨盒数据块), 就地修改比每次新建 bytes 对象省时又省内存。

·应用场景: NDP 数据包解析、串口通信、C 模块内存交互、sockets 的 recv 缓冲区等; 与 struct.pack 组合尤其

顺手。

·误区：bytearray 内只能赋值整数，不能赋 str (C[0]='x' 报 ValueError) 和 bytes (赋值字节需要 b'x'[0])。比较协调的方法是直接 new = bytearray(b'xy...') 再按索引改 int——但也别忘了它和 str/bytes 一样不能跨类型与 str 拼接。

37.19 Using Text and Binary Files (使用文本与二进制文件)

英文原文

Now that we've learned all about Python's string, let's return to their roles in files — the main context in which most programmers will likely encounter Unicode and bytes. As mentioned, the mode in which you open a file is crucial: it determines both the type of object you'll use (str vs bytes) and how text is transferred. Let's summarize the differences:- Text-mode files: decode file contents on input per their encoding; encode to bytes on output; support BOM headers for certain encodings (details at end); translate newlines by default to and from `\n`.- Binary-mode files: return file contents as raw byte-sequences, integers 0-255, with no encoding/decoding, BOM handling, or newline translation. In code: the mode argument (`open(path, mode, ...)`) — a second positional argument, a.k.a. the m

ode keyword — decides. The default mode is `rt` (read text), equivalent to `r`. Add `b` for binary. Whatever the platform, text files return `str` objects for reads; binary files return `bytes` objects for reads and accept `bytes` or `bytearray` for writes.

Text-File Basics (文本文件基础)

```
>>> file = open('temp.txt', 'w')      #
>>> size = file.write('abc\n')        #
3
>>> file.close()                      #
>>> file = open('temp.txt')           # r==rt
>>> text = file.read()
>>> text
'abc\n'
```

File line prints: as a footnote, filenames like `temp.txt` here are relative to the current directory; permission may be needed to write files on some systems (e.g., Android). And note that `file` — long a built-in name in 2.Q — no longer is (use `open`).

文本模式与二进制模式 (Text and Binary Modes)

```
>>> open('temp.txt', 'w').write('abc\n')  # str  Text-mode
4
>>> open('temp.txt', 'r').read()          # str
'abc\n'
>>> open('temp.txt', 'rb').read()         # bytes
b'abc\n'
```

In the text-mode write, Windows automatically maps `\n` to `\r\n` in the file, hence the `rb` read shows a `\r\n`. 这是因为 Windows 换行是 `\r\n`。二进制模式下则完全原

始。下面在二进制文件 (temp.bin) 上做同样实验：

```
>>> open('temp.bin', 'wb').write(b'abc\n')    # bytes
4
>>> open('temp.bin', 'r').read()              #
'abc\n'
>>> open('temp.bin', 'rb').read()             #
b'abc\n'
```

Python 换行映射：文本模式 read/write 会把'\n' 与宿主新行 (Windows \r\n / Unix \n) 双向映射；二进制模式不做任何映射。如果你在 Windows 上把'\r\n' 写进文本模式，read 回来还是'\n'——别慌。二进制模式的写接口接受 bytes 或 bytearray；读接口永远返回 bytes。范例 (含 NUL 字节的字节 sequence)：

```
>>> open('temp.bin', 'wb').write(b'a\x00c')    # bytes 0x00 ...
3
>>> open('temp.bin', 'r').read()               # NUL
'a\x00c'
```

书里强调：我们绝不能让 str 与 bytes 越过文件边界“越轨”——例：wm'写模式 str 是不可见，rb 在 text 里返回 str.. 可将 bytes 直接写进文本或 str 写进二进制，马上带去挂上文字错误。规范行为：

```
>>> open('temp.txt', 'w').write(b'abc\n')    # bytes →
TypeError: write() argument must be str, not bytes
>>> open('temp.bin', 'wb').write('abc\n')    # str →
TypeError: a bytes-like object is required, not 'str'
```

中文翻译

既然我们已经学尽了 Python 的字符串类型，现在回到它们在文件中的角色——这恐怕才是大多程序员遇到 Unicode 和 bytes 的主战场。正如前面反复强调，open() 的模式一

字之重：它决定用哪类对象（str 或 bytes）解释文件内容，以及文本传输的方向。归纳：- 文本模式文件：读取内容会自动按（明文或指定）编码解码为 str；写入把 str 编码进文件；对某些编码可以处理 BOM 头（后面详聊）；默认还做通用换行符翻译（把你脚本里全部的 \n 与宿主平台换行符双向靠近）。- 二进制文件：直接把文件内容原样以字节（整数 0..255）返回给你，完全没有任何编码、BOM、换行处理。在代码上，就是 open 的 mode 参数（它的第二个位置参数、a.k.a. mode= 关键字）来决策判文件模式：b 字面加进去即二进制；默认 r（即 rt，读文本）。与平台无关地：文本读定返回 str、写必须给 str；二进制读定返回 bytes、写接受 bytes 或 bytearray。文本文件基础 > text > >>> file = open('temp.txt','w') # 文本文件基础> >>> size = file.write('abc\n') ### 打开> 3 > >>> file.close() # 手动 close > >>> file = open('temp.txt') # 默认'rt'> >>> text = file.read() >'abc\n'> > 此例里看似 ASCII 文本，实则是 Unicode Unicode: text 模式按平台默认编码来自动 encode/decode。此外，close() 非必须 的场合（CPython 垃圾回收会自动 close）.....不过显示常见垃圾，在长任务里仍请。

文本与二进制模式代码分析 > python > open('temp.txt', 'w').write('abc\n') # 文本写：str 自动编码> open('temp.txt','r').read() # 文本读：解码回 str → 'abc\n'> open('temp.txt','rb').read() # 二进制读：根本不解码 → b'abc\r\n' (分行Windows) > > 因为在 Windows 上 \n 被文本模式翻译为 \r\n；而二进制读完全无翻译。接着在二进制文件 temp.bin 上：> python > open('temp.bin','wb').write(b'abc\n') # 写 bytes 原始字节> open('temp.bin','r').read

() # 读文本：把 \n 当范围内换行 → 'abc\n' > open('temp.bin', 'rb').read() # 读二进制：b'abc\n' 原样 > > 结论：文本模式拥抱换行翻译，二进制模式不做。写二进制时 bytes 与 bytearray 均可；读二进制永远返回 bytes。凡是把 str 写进二进制文件或把 bytes 写进文本文件，都会在第一时间被 python 拒绝 (TypeError) ——原理是隐式转换禁止。
> text > open('temp.txt', 'w').write(b'abc\n') → TypeError: write() argument must be str, not bytes
> open('temp.bin', 'wb').write('abc\n') → TypeError: a bytes-like object is required, not 'str' >

深度理解

- 核心概念：一个 open() 的 mode 字符串就塞进了"内容类型 (str/bytes) + 换行语义 (是否有翻译) + BOM"三卦权柄。读是 str/bytes，写是 str/bytes。
- 底层实现：CPython 文本文件的模式文件由 TextIOWrapper 包裹底层 BufferedReader；读 = decode(str, ...)，写 = encode(...)；并在内部按 OS 的换行常量 (os.linesep) 进行逐行转译。二进制模式则直接用底层 raw buffer；两者的路径完全不同层。
- 为什么这样设计：把"常见的换行平台差异"上交平台默认值，再提供"请自己选编码名"的按需可覆写。这样可以在读图、读日志、写配置文件里都用最直白的形式。
- 解决什么实际问题：确保一个文件在 Windows 上写的 \n 在 Linux 上读出来还是 \n (文本模式自动转)，而图像文件 (二进制) 跨平台字节不obliterated。

·易犯错误：假设“文本文件 = ASCII”。Windows 上 cp1252 是默认，web-indexed-file.log 若在迁移格式里此 ASCII+ 会很莫名其妙崩；另外把所有文件都当文本直接 print 出来容易让打印的二进制 crash。须牢记文本二进制中间的门就是 mode。

37.20 Unicode-Text Files (Unicode 文本文件)

英文原文

And now for the main event of file processing: text files whose content is non-ASCII. Beyond their text/binary generalities, Python files provide extra requirements and accessories for dealing with Unicode text. In short, text files accept a specific encoding name you pass to open via encoding, and use it to partially decode/encode text on read/write. Files themselves (identified by a pathname string) as: open(pathname,'r', encoding='utf8') — decode UTF-8 on reads open(pathname,'w', encoding='latin1') — encode Latin-1 on writes Here's an example using the universal UTF-8 encoding, with a Unicode output:

```
>>> file = open('uni.txt','w', encoding='utf8') >>> file.write(' 2 hÄck') # write 10个码点characters >>> file.close() >>> open('uni.txt','r', encoding='utf8').read() # 读出' 2 hÄck'
```

```
>>> [ord(c) for c in text] # 码点一目了然[128155, 32, 50, 32, 104, 196, 99, 107, 32, 128013]
```

```
>>> raw = open('uni.txt','rb').read() # 二进制的字节实体b'\xf0\x9f\x92\x9b 2 h\xc3\x84ck \xf0\x9f\x90\x8d'
```

File transfers raise exceptions whenever a requested encoding doesn't work. Examples: ASCII cannot handle the augmented/emoji chars in our text — read or write fails with `UnicodeEncodeError/UnicodeDecodeError`; whereas the broader UTF-16 handles this input fine on both ends (even though its stored bytes differ from UTF-8). Even for broader encodings, mixing encodings across read/write doesn't work: reading a UTF-8 file with `encoding='utf16'` (or vice versa) fails — the encoded bytes in the file don't match the mode expectations. Also, in the absence of encoding, text files encode/decode per the locale default. As a reminder: rely on your defaults can be fine for matching files, but it's a breeding ground for portability problems (see the sidebar later); an explicit encoding to open is the way to go. As in article, reading web pages and images with this pattern is easy — `String replace` works on the text, `raw data` works for bytes. Saving is the same pattern: encode for text, binary mode for bytes. `open` also accepts additional arguments: `newline=` determines newline mapping, `errors=` controls error handling (e.g., `'surrogateescape'` to preserve undecodable bytes) and `buffering=` changes buffering. Defaults are usually what you want; see the library manual for details.

```
>> import json >> json.load(open('data.json', encoding='utf-8')) # 注：此处为旧草稿遗留的残缺示例，  
--- ## 37.8 Unicode、字节与其它字符串工具 (Unicode,  
Bytes, and Other String Tools)
```

中文翻译

本节完整的 json 示例见下方 37.8.3 节 (以原文为准)

英文原文

Besides built-in strings and files, many of the popular string-processing tools in Python's standard library installed with Python itself also adhere to the str/bytes dichotomy. We won't cover any of these application-focused topics in detail in this core-language book, but as a sample, here's a very brief look at how some of these tools handle the split. See Python's Library Reference for more on the tools used here if any pique your interest.

中文翻译

除了内建的字符串与文件之外，Python 标准库中随 Python 一起安装的许多流行字符串处理工具，同样遵循 str/bytes 二分法。在这本“核心语言”书籍里，我们不会详细展开任何一项面向应用的专题，但作为样本，这里极为简略地看一眼其中一些工具是如何处理这种分裂的。如果其中有哪个工具勾起了你的兴趣，请查阅 Python 库参考手册获取更多信息。

深度理解

·核心概念：把"文本 vs 字节"的二分法从语言核心推广到标准库：re（正则）、struct（二进制打包）、pickle（对象序列化）、json（数据交换）、os/glob（文件名工具），全部严格区分 str 与 bytes。

·底层实现：这些模块在 C 层分岔成两套实现或两个分支——re 编译 str 模式与 bytes 模式为不同的内部匹配器；struct.pack 只接受 bytes 类对象；pickle 输出的永远是 bytes 流；json 输出的永远是 str 文本。

·为什么这样设计：正则、打包、序列化都要面对"内容可能不是文本"的歧义——同一个字节序列既可能是某种编码的文本，也可能是 JPEG 头。显式类型让歧义在调用前就消除，而不是运行时爆炸。

·解决什么实际问题：网络协议解析、跨语言文件交换、对象持久化、文件名处理——这些真实场景必须明确"我在处理文本还是字节"，标准库把这条纪律固化进了 API。

·初学者误区：以为"正则/序列化是全能的，传什么类型都行"。实际 re.match(strpattern, bytes) 直接 TypeError，pickle 往文本文件里 dump 也 TypeError——严格类型是本章从头到尾的主题。

37.8.1 re 模式匹配模块 (The re Pattern-Matching Module)

英文原文

Python's re pattern-matching module has been generalized to work on objects of any string type—str, bytes, and bytearray (a `(.)` means any run, saved as a group):

```
>> import re
>> S = 'is the fastest way to pizza!'
> B = b'Python is the fastest way to pizza!'
>> re.match('(.) the (.) way (.)', S).groups() # str + str => str (' i
s', 'fastest', 'to pizza!')
```

```
>> re.match(b'(.) the (.) way (.)', B).groups() # bytes +
bytes => bytes (b'Python is', b'fastest', b'to pizza!')
```

```
>> re.match(b'(.) the (.) way (.)', bytearray(B)).groups(
) (b'Python is', b'fastest', b'to pizza!')
```

Like many tools, though, its result types depend on the type of strings you pass in, and you can't mix str and bytes types in its calls' arguments (sans explicit conversions, of course); bytearray is also unusable here as a pattern because it's mutable (and changeable means "unhashable"):

```
>> re.match('(.) the (.) way (.)', B).groups() TypeError:
cannot use a string pattern on a bytes-like object
>> re.match(b'(.) the (.) way (.)', S).groups() TypeError: ca
nnot use a bytes pattern on a string-like object
>> re.match(b'(.) the (.) way (.)', bytearray(B)).groups() TypeE
rror: cannot use a string pattern on a bytes-like object
>> re.match(bytearray(b'(.) the (.) way (.)'), bytearray
```

y(B)).groups() TypeError: unhashable type:'bytearray'

中文翻译

Python 的 re 模式匹配模块已经推广到能处理任何字符串类型的对象——str、bytes 和 bytearray ((.) 表示任意一段内容，并保存为一个分组)：

```
>>> import re
>>> S = 'is the fastest way to pizza!' # str 模式匹配 str
>>> B = b'Python is the fastest way to pizza!' # bytes 模式匹配 bytes
>>> re.match('(. the (.) way (.)', S).groups() # str + str => str (' is', 'fastest', 'to pizza!')
>>> re.match(b'(. the (.) way (.)', B).groups() # bytes + bytes => bytes (b'Python is', b'fastest', b'to pizza!')
>>> re.match(b'(. the (.) way (.)', bytearray(B)).groups() (b'Python is', b'fastest', b'to pizza!')
不过和许多工具一样，它的结果类型取决于你传入的字符串类型，调用参数中不能混用 str 与 bytes 类型（当然，除非你显式转换）；bytearray 在这里也不能当模式用，因为它是可变的（可变化就意味着“不可哈希”）：
```

```
>>> re.match('(. the (.) way (.)', B).groups()
TypeError: cannot use a string pattern on a bytes-like object
>>> re.match(b'(. the (.) way (.)', S).groups()
TypeError: cannot use a bytes pattern on a string-like object
>>> re.match('(. the (.) way (.)', bytearray(B)).groups()
TypeError: cannot use a string pattern on a bytes-like object
>>> re.match(bytearray(b'(. the (.) way (.)', bytearray(B)).groups()
TypeError: unhashable type:'bytearray'
```

深度理解

- 核心概念：re 模块是"输入类型决定输出类型"的典型——str 模式返回 str 分组，bytes 模式返回 bytes 分组，bytearray 数据也能匹配（结果仍是 bytes）。
- 底层实现：CPython 的 re 在编译模式时探测模式对象的类型：str 模式编译进一个以码点为单位的匹配器，bytes 模式编译进一个以字节为单位的匹配器；被匹配的目标文本若类型不符，C 层在调用入口处就用 PyUnicodeCheck / PyBytesCheck 判型并抛出 TypeError。
- 为什么这样设计：字节流里匹配的"字符"就是 0-255 的整数，与码点的含义完全不同；让同一条正则跑在两种语义上必然出错。类型检查把"模式/目标必须同型"的纪律固化进 API。
- 解决什么实际问题：抓日志、扫 HTML、解析网络协议时，数据常常是 bytes——re 对 bytes 的完整支持让你不需要先 decode 就能直接匹配，避免解码失败或乱码的烦恼。
- 初学者误区：拿着 str 模式去匹配 bytes 数据（或反之），以为 Python 会"帮忙转换"。它不会——直接 TypeError。而 bytearray 因可变不可哈希，连当模式都不行（模式缓存需要哈希）。

代码分析

```
import re
S = ' is the fastest way to pizza!'          # str
B = b'Python is the fastest way to pizza!'  # bytes
re.match('(.*) the (.*) way (.*)', S).groups() # → (' is', 'fas...
re.match(b'(.*) the (.*) way (.*)', B).groups() # → (b'Python is'...
re.match(b'(.*) the (.*) way (.*)', bytearray(B)).groups() # byte...
# —
re.match('(.*) the (.*) way (.*)', B).groups()          # TypeErr...
re.match(bytearray(b'(.*) the (.*) way (.*)'), bytearray(B)).group...
```

解释器如何处理：re.match 第一件事是调用 re.compile 缓存查找（哈希键为“模式文本 + flags”），所以模式必须是可哈希的 str 或 bytes；bytearray 可变，哈希即失效，故直接报 unhashable type。编译完成后，匹配器按目标对象的类型分派到不同的底层扫描函数：对 str 目标按码点走 SRE 引擎，对 bytes 目标按原始字节走同一引擎的字节分支。分组 .groups() 返回的元组元素类型与目标一致——str 给字符，bytes 给字节子串。

设计思想：输入类型决定输出类型，且绝不隐式转换——把“文本语义”与“字节语义”两种匹配世界彻底隔离，正是 Python 3 字符串模型的一贯哲学。

37.8.2 struct 二进制数据模块 (The struct Binary-Data Module)

英文原文

The Python struct module, used to create and extract packed binary data from strings, operates on bytes and bytearray only, not str—which makes sense, given that it's intended for processing binary data, not dec

oded text. This module uses a format string to specify the types and sizes of objects to pack to and from a bytes string, along with an endianness (i.e., significant side of bitstrings) specifier like > for big-endian:

```
>> import struct
>> B = struct.pack('>i4sh', 7, b'code', 8) # int, bytes(4s), int(h) => bytes
>> B
b'\x00\x00\x00\x07code\x00\x08'
>> vals = struct.unpack('>i4sh', B) # Packed data is bytes, not str
>> vals
(7, b'code', 8)
>> vals = struct.unpack('>i4sh', B.decode())
TypeError: a bytes-like object is required, not 'str'
```

You'll often use this in conjunction with binary-mode files to make the packed bytes persistent across program runs (e.g., for saving and restoring app user settings):

```
>> F = open('data.bin', 'wb') # Open binary output file
>> data = struct.pack('>i4sh', 7, b'code', 8) # Create packed binary data
>> data
b'\x00\x00\x00\x07code\x00\x08'
>> F.write(data) # Write to the file
10
>> F.close() # Flush changes
>> F = open('data.bin', 'rb') # Open binary input file
>> data = F.read() # Read bytes
>> data
b'\x00\x00\x00\x07code\x00\x08'
>> values = struct.unpack('>i4sh', data) # Extract packed binary data
>> values
(7, b'code', 8) # Back to Python objects
```

中文翻译

Python 的 struct 模块，用于在字符串中创建和提取打包的二进制数据，它只处理 bytes 与 bytearray，不处理 str——这很合理，因为它本来就是为处理二进制数据（而不是已解码文本）而生的。该模块用一个格式字符串来指定要被“打包/解包”对象的类型与大小，并带一个字节序（即决定一个位串哪一端是有效侧）说明符，例如 > 表示大端序（big-endian）：

```
>>> import struct
>>> B = struct.pack('>i4sh', 7, b'code', 8) # int、bytes(4B)、int(h) => bytes
>>> B
b'\x00\x00\x00\x07code\x00\x08'
>>> vals = struct.unpack('>i4sh', B) # 打包数据是 bytes，不是 str
>>> vals
(7, b'code', 8)
>>> vals = struct.unpack('>i4sh', B.decode())
TypeError: a bytes-like object is required, not 'str'
```

你常常会把它与二进制模式文件配合使用，让打包好的字节数据能跨程序运行保存下来（例如保存和恢复应用程序的用户设置）：

```
>>> F = open('data.bin', 'wb')
# 打开二进制输出文件
>>> data = struct.pack('>i4sh', 7, b'code', 8) # 创建打包二进制数据
>>> data
b'\x00\x00\x00\x07code\x00\x08' # bytes，绝非 str
>>> F.write(data) # 写入文件
10
>>> F.close() # 刷新变更
>>> F = open('data.bin', 'rb') # 打开二进制输入文件
>>> data = F.read() # 读取字节
>>> data
b'\x00\x00\x00\x07code\x00\x08'
>>> values = struct.unpack('>i4sh', data) # 提取打包二进制数据
>>> values
(7, b'code', 8) # 恢复成 Python 对象
```

深度理解

·核心概念：struct 是"二进制格式字典"——用紧凑的格式字符串描述一块内存里每个字段的类型与大小，pack 把 Python 对象压成打包字节，unpack 把打包字节解回元组。格式字符串 '>i4sh' 的含义：> 大端序、i 4 字节 int、4s 4 字节 bytes、h 2 字节 short。

·底层实现：struct.pack 在 C 层按格式字符串逐字段写出数据：int 7 → '\x00\x00\x00\x07', b'code' → code, short 8 → '\x00\x08'。总长度由格式决定（4+4+2=10 字节），与平台无关但对齐规则需自行处理。

·为什么这样设计：跨语言、跨平台的二进制互操作有严格的内存布局规范（网络字节序、固定宽度），struct 把这套"用手排字节"的苦差变成声明式格式字符串；声明后才需要保证内存精确，所以绝不能接受 str——str 是码点，没有固定字节形态。

·解决什么实际问题：与 C 结构体互操作（struct 常和 ctypes 并肩）、解析二进制文件头（如 BMP/PNG 头、TCP 首部）、序列化数值型配置文件等。

·初学者误区：试图 struct.unpack(format, B.decode())——以为先解码成文本更"友好"，结果直接 TypeError。打包数据是 bytes，格式字符串必须与数据字节序、类型、宽度完全一致，一个符号错了都解不对。

代码分析


```
import struct
packed = struct.pack('>i4sh', 7, b'code', 8)    # → b'\x00\x00\x00\x00...'
values = struct.unpack('>i4sh', packed)         # → (7, b'code', 8)
#
F = open('data.bin', 'wb')
F.write(struct.pack('>i4sh', 7, b'code', 8))    #
F.close()
```

解释器如何处理：字节格式字符串'>i4sh' 在 struct 首次调用时被缓存编译成一个"打包模板"（含格式代码、偏移量、对齐），pack 依据模板把传入对象逐一序列化为原始字节并拼接进 C 层缓冲区；unpack 做反向操作。> 前缀决定多字节整数的字节序（大端：高位在前），这正是跨平台二进制格式的公约。文件这侧的 wb/rb 模式确保数据原样进出，不经任何编码解码。

设计思想：用"声明式格式串"取代手写大小端与逐字节拷贝——把底层 C 语言最危险的"内存布局"玩法，包装成高层的、可校验的代码。这种"格式即协议"的思想随后在 pickle（自带格式）与 json（通用交换格式）中继续演化。

37.8.3 pickle 与 json 序列化模块 (The pickle and json Serialization Modules)

英文原文

Python's pickle module provides another way to save data in files but allows saved data to be nearly arbitrary Python objects instead of values coerced to bytes as in struct. We met this module briefly in Chapters 9, 28, and 31. For completeness here, keep in mind that

the pickle module always creates a bytes object, regardless of the default or passed-in protocol (data format level).

You can see this by using the module's `dumps` call to return an object's pickle string:

```
>> import pickle >> pickle.dumps(['code', 4, '']) # dumps() returns pickle string b'\x80\x04\x95\x15\x00\x00\x00\x00 ... etc ... \x04\x8c\x04\xf0\x9f\x90\x8d\x94e.'
```

```
>> pickle.dumps(['code', 4, ''], protocol=0) # ASCII protocol 0, still bytes! b'(lp0\nVcode\np1\nal4\naV\\U0001f40d\np2\na.'
```

This implies that files used to store pickled objects must always be opened in binary mode, because text files use `str` strings to represent data, not bytes, and the `dump` call simply attempts to write the pickled byte string to an open output file:

```
>> pickle.dump(['code', 4, ''], open('temp.pkl', 'w')) # bytes + text mode fail
TypeError: write() argument must be str, not bytes
>> pickle.dump(['code', 4, ''], open('temp.pkl', 'w'), protocol=0)
TypeError: write() argument must be str, not bytes
>> pickle.dump(['code', 4, ''], open('temp.pkl', 'wb')) # Always binary mode
>> open('temp.pkl', 'r').read()
UnicodeDecodeError: 'utf-8' codec can't decode byte 0x80 in position 0:...
```

Notice the last result fails here in text mode on mac

OS; if it doesn't fail for you, it's only because the stored binary data is compatible with your platform's default decoding (i.e., just by luck!). A Windows host, for example, prints gibberish for the last result instead of an error message.

Because pickle data is not generally decodable Unicode text, the same rule holds on input as output—correct usage always requires both writing and reading pickle data with binary-mode files, whether unpickling or not:

```
>> pickle.dump(['code', 4,'], open('temp.pkl','wb'))
# Save to file >> pickle.load(open('temp.pkl','rb')) # Load from file
['code', 4,'] >> open('temp.pkl','rb').read()
b'\x80\x04\x95\x15\x00\x00\x00\x00 ...' ...'\x04\x8c\x04\xf0\x9f\x90\x8d\x94e.'
```

The Python json module, introduced in Chapter 9, is a related tool: it converts a nested-object tree to a text string that can be saved on a file to make it persistent. Unlike pickle, json doesn't support arbitrary objects (just basic types and built-in containers) but uses a language-neutral scheme that can make it more interoperable with other programs.

Also unlike pickle, the json module always produces a str, so files for saves and loads should generally use text mode (JSON files are commonly encoded per UTF-8 for portability, but any encoding works as long as it's consistent and expected):

```
>> import json >> vals = ['code', {'app': ('', None, 1.23, 99)}] >> json.dumps(vals) # dumps(): to str['code', {"app": ["\ud83d\ude42", null, 1.23, 99]}]>> text = json.dumps(vals) # Save to/load from str >> anew = json.loads(text) # loads(): from str >> anew ['code', {'app': ['', None, 1.23, 99]}] >> file = open('data.txt', 'w', encoding='utf8') # Save/load text-mode file >> json.dump(vals, file) >> file.close() >> file = open('data.txt', 'r', encoding='utf8') >> json.load(file) ['code', {'app': ['', None, 1.23, 99]}]
```

中文翻译

Python 的 pickle 模块提供了另一种把数据保存到文件的办法，但它允许被保存的是几乎任意的 Python 对象，而不是像 struct 那样被强转成字节的值。我们曾在第 9、28、31 章简单碰过它。为了本章的完整性，请记住：pickle 模块总是产生一个 bytes 对象——不管使用默认的还是传入的 protocol（数据格式级别）。你可以通过模块的 dumps 调用返回对象的 pickle 串来亲眼看到这一点：

```
>>> import pickle >>> pickle.dumps(['code', 4, '']) # dumps() 返回 pickle 串 b'\x80\x04\x95\x15\x00\x00\x00\x00 .....
... ' ..... '\x04\x8c\x04\xf0\x9f\x90\x8d\x94e.' >>> pickle.dumps(['code', 4, ''], protocol=0) # ASCII 协议 0, 仍是 bytes! b'(\p0\nVcode\np1\nal4\naV\\U0001f40d\np2\na.'
```

这意味着：用来存储 pickle 对象的文件必须始终以二进制模式打开，因为文本文件用 str 表示数据，而不是 bytes，而 dump 调用只是把 pickle 出来的字节串写到已

打开的输出文件上：>>> pickle.dump(['code', 4, ''], open('temp.pkl', 'w')) # bytes+文本模式失败TypeError: write() argument must be str, not bytes >>> pickle.dump(open('temp.pkl', 'w'), protocol=0) TypeError: write() argument must be str, not bytes >>> pickle.dump(['code', 4, ''], open('temp.pkl', 'wb')) # 始终二进制模式

>>> open('temp.pkl', 'r').read() UnicodeDecodeError: 'utf-8' codec can't decode byte 0x80 in position 0:...注意最后这个结果在 mac 系统上以文本模式读取时会报错；如果你的机器上没报错，那只是因为你存进去的二进制数据恰好与平台默认解码兼容——纯属走运！例如 Windows 主机会把最后一个结果显示成一堆乱码而不是报错信息。由于 pickle 数据通常不是可解码的 Unicode 文本，输入侧同样遵循这一规则——正确的用法无论是否反序列化，都必须用二进制模式文件读写 pickle 数据：>>> pickle.dump(['code', 4, ''], open('temp.pkl', 'wb')) # 保存到文件>>> pickle.load(open('temp.pkl', 'rb')) # 从文件中加载['code', 4, ''] >>> open('temp.pkl', 'rb').read() b'\x80\x04\x95\x15\x00\x00\x00\x00 \x04\x8c\x04\xf0\x9f\x90\x8d\x94e.'

第 9 章介绍过的 Python json 模块是一个性质相近的工具：它把一棵嵌套对象树转换成一段文本字符串，可保存在文件上以实现持久化。与 pickle 不同，json 不支持任意对象（仅支持基本类型和内建容器），但它采用语言无关的方案，因而与其他程序更具互操作性。同样与 pickle 不同，json 模块总是产生 str，所以用于保存和读取 json 的文件通常应该使用文本模式（JSON 文件按惯例多采用 UTF-8 编码以保证可移植性，但只要一致且可预期，任何编码都行）：>>> import json >>> vals = ['code', {'

```
app': ('', None, 1.23, 99)]] >>> json.dumps(vals) # d
umps() 返回 str['"code"', {'app': ['"\ud83d\ude42"', n
ull, 1.23, 99]]]'>>> text = json.dumps(vals) # 保存到/
从 str >>> anew = json.loads(text) # loads() 从 str 还
原>>> anew ['code', {'app': ['', None, 1.23, 99]]] >>
> file = open('data.txt','w', encoding='utf8') # 保存文
本模式>>> json.dump(vals, file) >>> file.close() >>>
file = open('data.txt','r', encoding='utf8') >>> json.lo
ad(file) ['code', {'app': ['', None, 1.23, 99]]]
```

深度理解

·核心概念：pickle (Python 专用二进制协议) 与 json (语言中立文本协议) 是两大"对象序列化"工具。pickle 恒产 bytes → 必须配二进制模式文件；json 恒产 str → 用文本模式文件。

·底层实现：pickle 将对象图递归打包成带类型标签的字节流（协议 0 是 ASCII 读得懂的 l p 0...p 1...a l 4 操作码；新协议是紧凑二进制），json 通过递归遍历把 dict/list/str/int...序列化成带括号与引号的标准文本语法。

·为什么这样设计：pickle 追求"还原出原对象"，为此它把对象的类型、属性、甚至引用的人可读信息全写进数据；json 追求"跨语言可读"，因此只保留通用数据类型，牺牲表达能力换取普适性。

·解决什么实际问题：save/load 程序状态 (pickle)、缓存与本地存储；与 Web/其他语言交换数据 (json)。尤其 json 是当今 Web API 事实标准的序列化方式。

·初学者误区：以为 pickle 可以用文本模式写——TypeError；把 pickle 数据当文本读——UnicodeDecodeError。而 json 恰好相反，用二进制模式反而报错。记住：看 dump 出来的对象类型定文件模式。

代码分析

```
import pickle, json
# picklebytes →
pickle.dump(['code', 4, ''], open('temp.pkl', 'wb')) # 'w'
data = pickle.load(open('temp.pkl', 'rb')) # ['cod...
# jsonstr →
json.dump({'app': ('', 1.23)}, open('data.txt', 'w', encoding='ut...
back = json.load(open('data.txt', 'r', encoding='utf8'))
```

解释器如何处理：pickle.dump(obj, f) 先按协议把对象编码成中间字节，再调用 f.write(bytes)；对象格式非法或文件模式不匹配都会在 write 处暴露。json.dump(obj, f) 同理，但编码对象是字符串，若 f 是二进制模式就会包 TypeError。反转时的 load/loads 都按字节/字符流解析出对象图。一句话：dump/loads 看对象自己的类型（bytes→二进制），load/dump 看文件模式（二进制→'wb'，文本→'w'）。

37.21 文件名：open 与其它文件名工具 (Filename names in open and Other Filename Tools)

英文原文

So far in this chapter, we've focused on the content o

f files, but their names have a Unicode story too. Its short version is that str is the norm for file pathnames in Python and is recommended for portability. If your code, like all the examples so far, uses str for filenames and pathnames, they simply work: Python's file tools automatically translate them to and from the encoding used by the host platform and device.

It turns out, though, that Python's file tools also allow you to specify file pathnames as bytes to skip automatic filesystem encoding in some contexts. This bytes mode may come in handy if you need more control over filename encodings in cross-platform code, but it is rarely needed in typical programs. Per Python's manuals, in fact, this mode need be used only on some Unix systems where undecodable filenames may be present.

Nevertheless, bytes filenames do work in these tools, as shown in the following demo, which runs the same on macOS, Windows, Linux, and Android devices tested. As shown, the open function happily accepts text or bytes for a file's non-ASCII pathname (this section's examples were run in subfolder filenames in the book's examples package):

```
>> import sys, os, glob >> sys.getfilesystemencoding() # Filesystem default: macOS and Win11'utf-8'>>
name1 = 'hÄck1' # str filenames >> name2 = 'hÄck
2'>> name2.encode('utf8') # bytes equivalent for na
```



```
me2 str b'h\xc3\x84ck\xf0\x9f\x99\x822'>> os.listdir(
) # Make files with str and bytes names [] >> open(n
ame1, mode='w', encoding='utf8').write('text1') 5 >>
open(b'h\xc3\x84ck\xf0\x9f\x99\x822','w', encoding=
'utf8').write('text2') 5 >> os.listdir() # Both show up o
n the filesystem ['hÄck1','hÄck2'] >> open('hÄck
2').read() # And can be accessed either way'text2'>>
open('hÄck2'.encode('utf8')).read()'text2'
```

For bytes filenames, Python uses UTF-8 encoding on macOS and on Windows since 3.6 (with extra translation for the Windows API's UTF-16). Linux, and by extension Android, accepts any sort of bytes for filenames, but Python tries to use UTF-8 when converting to and from str (with surrogate escapes for encoding errors: see Python's manuals).

Whether bytes filenames are a useful tool or a neat parlor trick depends on your use cases, but keep in mind that bytes filenames will only work in open if their encoding matches the expectations of Python or the underlying filesystem. To demo, the following passes Latin-1 bytes b'h\xc4ck' to open: this fails on both macOS and Windows as shown (though with a Unicode DecodeError on the latter because it's a matter of luck):

```
>> 'hÄck'.encode('latin-1'),'hÄck'.encode('utf-8') (b'h
\xc4ck', b'h\xc3\x84ck') >> b'h\xc4ck'.decode('utf8')
UnicodeDecodeError:'utf-8' codec can't decode byte
```

```
0xc4 in position 1:... >> open(b'h\xc4ck','w') OSError:
[Errno 92] Illegal byte sequence: b'h\xc4ck'# open(b'
h\xc4ck'.decode('latin1'),'w') # Works: convert to str #
open(b'h\xc3\x84ck','w') # Works: use UTF-8 encodin
g
```

By contrast, the preceding code's `open` works on Linux and Android—which use the actual filesystem bytes and don't care about the encoding of the name. For both its bytes and escaped str forms, the Latin-1 name fails in `open` on macOS and Windows as shown. Hence, the effect of using arbitrary bytes in `open` varies by both platform and filesystem (that is, don't do that!).

The `os` standard-library module's directory-listing `listdir` similarly accepts str or bytes and returns a folder's names in the same form, ready to be used in other file tools (as in the earlier listing, it also defaults to the current directory if no folder is passed):

```
>> os.listdir('.') # str gives str, bytes gives bytes ['hÄ
ck1','hÄck2'] >> os.listdir(b'.') [b'h\xc3\x84ck\xf0\x
9f\x99\x821', b'h\xc3\x84ck\xf0\x9f\x99\x822'] >> fo
r name in os.listdir('.):
```

```
... print(name,'=>', open(name).read()) hÄck1 => te
xt1 hÄck2 => text2 >> for name in os.listdir(b'.):
```

```
... print(name,'=>', open(name).read()) b'h\xc3\x84ck\
xf0\x9f\x99\x821' => text1 b'h\xc3\x84ck\xf0\x9f\x99
\x822' => text2
```

The glob module does filename expansion both ways, too (as str or bytes), and os.fsdecode decodes filenames per the host's default automatically:

```
>> glob.glob('hck') # str gives str, bytes gives bytes
['hÄck1', 'hÄck2'] >> glob.glob(b'hck') [b'h\xc3\x84ck\x00\x9f\x99\x821', b'h\xc3\x84ck\x00\x9f\x99\x822']
>> for name in glob.glob(b'hck'):
...     print(name.decode(sys.getfilesystemencoding()), os.fsdecode(name)) hÄck1 hÄck1 hÄck2 hÄck2
```

中文翻译

到目前为止，本章我们重点讲的是文件的内容，但它们的名字也有属于自己的 Unicode 故事。一句话版本是：在 Python 中，str 是文件路径名的常规，也是推荐的做法，以便保证可移植性。只要你的代码（像前面所有例子一样）文件名和路径名都用 str，那么它们就能直接工作：Python 的文件工具会自动把它与宿主平台和设备所用的编码互译。不过事实还证明，Python 的文件工具也允许你把文件路径名指定成 bytes，以便在某些场景下跳过自动的文件系统编码。这种 bytes 模式在你需要更精细地控制跨平台代码中文件名的编码时也许会有用，但在典型程序中很少需要。按 Python 手册的说法，事实上这种模式仅在少数可能存在无法解码文件名的 Unix 系统上才有必要。尽管如此，bytes 文件名在这些工具中也是管用的，正如下面的演示——它在测过的 macOS、Windows、Linux 和 Android 上运行结果一致。它展示：open 函数高兴地接受文本或 bytes 形式

的非 ASCII 路径名（本章示例都是在本书示例包里名为 file names 的子文件夹中运行的）：

```
>>> import sys, os, glob
>>> sys.getfilesystemencoding() # 文件系统默认值：
macOS 与 Win11 'utf-8'
>>> name1 = 'hÄck1' # str 文件名
>>> name2 = 'hÄck2'
>>> name2.encode('utf8') # name2 str 的 bytes 等价物
b'h\xc3\x84ck\xf0\x9f\x99\x822'
>>> os.listdir() # 用 str 和 bytes 两种名字制作文件[]
>>> open(name1, mode='w', encoding='utf8').write('text1')
5
>>> open(b'h\xc3\x84ck\xf0\x9f\x99\x822', 'w', encoding='utf8').write('text2')
5
>>> os.listdir() # 两者都会出现在文件系统上['hÄck1', 'hÄck2']
>>> open('hÄck2').read() # 且两种方式都能访问'text2'
'>>> open('hÄck2'.encode('utf8')).read()
'text2'
对于 bytes 文件名，Python 在 macOS 上使用 UTF-8 编码，而在 Windows (3.6 起) 上同样使用 UTF-8，只是额外做了针对 Windows API 的 UTF-16 转译。Linux (以及 Android) 接受任意字节序列当文件名，但 Python 在与 str 互相转换时会尽量使用 UTF-8 (遇到编码错误时用代理转义 surrogate escapes 处理：参见 Python 手册)。至于 bytes 文件名到底是实用工具还是徒增花活，取决于你的具体用例；但请记住：bytes 文件名只有在编码与 Python 或底层文件系统的期望一致时，才能在 open 中生效。为了演示，下面把 Latin-1 字节 b'h\xc4ck' 传给 open：它在 macOS 和 Windows 上都会失败（尽管在后者上会因为运气问题而报一个 UnicodeDecodeError）：
```

```
>>> 'hÄck'.encode('latin-1'), 'hÄck'.encode('utf-8')
(b'h\xc4ck', b'h\xc3\x84ck')
>>> b'h\xc4ck'.decode('utf8')
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xc4 in position 1:
```

```

...>>> open(b'h\xc4ck','w') OSError: [Errno 92] Illegal
byte sequence: b'h\xc4ck'# open(b'h\xc4ck'.decode('l
atin1'),'w') # 可行：转成 str # open(b'h\xc3\x84ck','w')
# 可行：使用 UTF-8 编码相比之下，前面代码里的 open
在 Linux 和 Android 这种接受任意字节的平台上却可以工
作——它们磁盘上的真实文件本质就是字节，对名字的 “
编码” 并不敏感。但对 bytes 形式和转义后的 str 形式，L
atin-1 名字在 macOS 和 Windows 上都会如上所示地在
open 中失败。因此，在 open 中使用任意 bytes 的效果既
取决于平台也取决于文件系统（也就是说，别这么干！）。
os 标准库模块的 listdir 目录列举函数同样接受 str 或 byte
s，并按相同的形态返回文件夹中的名字（如前面的列表所
示，它不传文件夹参数时默认到当前目录）： >>> os.list
dir('.') # str 给 str，bytes 给 bytes ['hÄck1','hÄck2']
>>> os.listdir(b'.') [b'h\xc3\x84ck\x0\x9f\x99\x821',
b'h\xc3\x84ck\x0\x9f\x99\x822'] >>> for name in os
.listdir('.'): ... print(name,'=>', open(name).read()) hÄc
k1 => text1 hÄck2 => text2 >>> for name in os.li
stdir(b'.'): ... print(name,'=>', open(name).read()) b'h\
xc3\x84ck\x0\x9f\x99\x821' => text1 b'h\xc3\x84ck\
xf0\x9f\x99\x822' => text2 glob 模块也是以两种形式（
str 或 bytes）做文件名展开，而 os.fsdecode 会按主机的
默认值自动解码文件名： >>> glob.glob('hck') # str 给 s
tr，bytes 给 bytes ['hÄck1','hÄck2'] >>> glob.glob
(b'hck') [b'h\xc3\x84ck\x0\x9f\x99\x821', b'h\xc3\x8
4ck\x0\x9f\x99\x822'] >>> for name in glob.glob(b'
hck'): ... print(name.decode(sys.getfilesystemencodin
g()), os.fsdecode(name)) hÄck1 hÄck1 hÄck2 hÄc

```

深度理解

- 核心概念：文件名在 Python 里可同时使用 str 与 bytes 两种路径；推荐 str（因为它经文件系统编码自动转换，跨平台才能通用），bytes 路径仅在你确切清楚底层系统编码时才安全。
- 底层实现：open 接受 str 时，CPython 内部会调用 PyUnicodeFSConverter 把 str 按 `sys.getfilesystemencoding()`（如 UTF-8）编码成 C 层的 char 路径；接受 bytes 时则直接透传字节。os.listdir 内部调 os.scandir，返回类型随输入而定——str 输入返回 str 项，bytes 输入返回 bytes 项。
- 为什么这样设计：不同平台的路径本质是字节序列，而 str 是“已解码的码点”；用 str 维持了跨平台可读性，用 bytes 则是“想跳过转换、精确控制”时的逃生门——两者各有存在意义。
- 解决什么实际问题：让中文/emoji 文件名在 Windows（UTF-16 内部 API）与 POSIX（字节开放的 fs）上都能被安全地读写；其次，当你拿到以任意编码写死的旧文件名时，bytes 路径是保留原始字节、不做猜测解码的唯一办法。
- 初学者误区：以为“open 文件名必须始终用 str”。其实 bytes 中文路径在多数现代 OS 上也合法（按 UTF-8 解释）；反过来不要拿 Latin-1 bytes 当文件名硬传，一定是 Illegal byte sequence 或 UnicodeDecodeError。记住

: 给 open 传 str 让 Python 去转换, 传 bytes 就让字节原样通关。

代码分析

```
import sys, os, glob
sys.getfilesystemencoding()           # 'utf-8' macOS/...
name1 = 'hÄck1'                       # +emoji str
open(name1, mode='w', encoding='utf8').write('text1')
open(b'h\xc3\x84ck\x0\x9f\x99\x822', 'w', encoding='utf8').write(...)
os.listdir('.')                       # ['hÄck1', 'hÄck...
os.listdir(b'.')                     # bytes
glob.glob('h*ck*')                   # str
glob.glob(b'h*ck*')                  # bytes
```

解释器如何处理: 每一条 open 都会在系统调用边界执行一次"编码适配"。str 路径经解析器转成操作系统原生字符编码 (Windows 是 UTF-16 wchar_t 的 W 系列 API, POSIX 是 UTF-8 的 char), bytes 路径作为已编码字节原样通过。listdir 的类型选择取决于你传的参数 (str 或 bytes), 从而决定名字以 Unicode 还是原始字节呈现。

设计思想: 文件系统是"字节自由之地", Python 则坚持"文本一律 str"的纪律——于是允许文件名的 bytes 形态, 但永远提醒你把它当例外而不是理所当然。最稳的跨平台打法: 统一传 str, 一切都交给 Python 收拾。

英文原文

In addition, tools that create and walk folders are similarly flexible for names and paths (demoed on Windows—your path separators and order may vary):

```
>> os.mkdir('sub') # Make dirs with str or bytes >>
os.mkdir('subbytes'.encode('utf8')) >> os.mkdir('sub
bytes/subsub') # and walk them with both >> os.li
stdir() ['hÄck1','hÄck2','sub','subbytes'] >> os.li
stdir('subbytes') ['subsub'] >> for (dirhere, subshe
re, fileshere) in os.walk('.'):

```

```
... print(dirhere,'=>', subshere, fileshere) . => ['sub','
subbytes'] ['hÄck1','hÄck2'].\sub => [] [].\subb
ytes => ['subsub'] [].\subbytes\subsub => [] [] >
>>> for (dirhere, subshere, fileshere) in os.walk(b'.'):

```

```
... print(dirhere,'=>', subshere, fileshere) b'.' => [b'su
b\x0f\x9f\x99\x8a', b'sub\x0f\x9f\x99\x8abytes'] [b'h\
xc3\x84ck\x0f\x9f\x99\x821', b'h\x03\x84ck\x0f\x9f\x
99\x822'] b'.\sub\x0f\x9f\x99\x8a' => [] [] b'.\sub\x
0f\x9f\x99\x8abytes' => [b'subsub\x0f\x9f\x91\x8d'] [
] b'.\sub\x0f\x9f\x99\x8abytes\subsub\x0f\x9f\x91\
x8d' => [] []

```

While bytes pathnames work as shown and may have some valid roles, it's important to stress that you're almost always better off using str strings to name files instead. Doing so leverages tools that already go to great lengths to make pathnames do the right thing, and might just avoid at least some portability issues that can arise in apps whose scope must span platforms and devices.

And that's all the time and space we have for this too. Is survey. Really, Python's standard library and third-

party domain are large and evolving toolsets that will likely occupy much of your attention after you've finished this book and move on to real programming tasks. Again, be sure to consult the Python Library Reference soon and often for more info.

中文翻译

另外，创建和遍历文件夹的工具在名字与路径上同样灵活（以下在 Windows 上演示——你的路径分隔符与顺序可能略有不同）：

```
>>> os.mkdir('sub') # 用 str 或 bytes 建目录
>>> os.mkdir('subbytes'.encode('utf8'))
>>> os.mkdir('subbytes/subsub') # 两种形式都能遍历
>>> os.listdir() ['hÄck1', 'hÄck2', 'sub', 'subbytes']
>>> os.listdir('subbytes') ['subsub']
>>> for (dirhere, subshere, fileshe) in os.walk('.'): ... print(dirhere, '=', subshere, fileshe)
.=> ['sub', 'subbytes'] ['hÄck1', 'hÄck2']
.\sub => [] []
.\subbytes => ['subsub'] []
.\subbytes\subsub => [] []
>>> >>> for (dirhere, subshere, fileshe) in os.walk(b'.'): ... print(dirhere, '=', subshere, fileshe)
b'.' => [b'sub\xff\x9f\x99\x8a', b'sub\xff\x9f\x99\x8abytes']
[b'h\xc3\x84ck\xff\x9f\x99\x821', b'h\xc3\x84ck\xff\x9f\x99\x822']
b'\\.sub\xff\x9f\x99\x8a' => [] []
b'\\.sub\xff\x9f\x99\x8abytes' => [b'subsub\xff\x9f\x91\x8d'] []
b'\\.sub\xff\x9f\x99\x8abytes\\.subsub\xff\x9f\x91\x8d' => [] []
```

尽管 bytes 路径名如上所示确实能工作，也可能在某些场景有它的用武之地，但必须强调：几乎总是用 str 字符串来命名文件更明智。这样做能借助那些已经煞费苦心让路径名"

做对事"的工具，并且很可能会避免至少一部分"应用必须横跨多平台多设备"时出现的可移植性问题。这个工具速览就到此为止。说真的，Python 的标准库与第三方生态是庞大且不断演化的工具集，等你读完本书转战真正的编程任务时，它们很可能占据你大量的注意力。再次提醒：请尽早、经常查阅 Python 库参考手册获取更多信息。

深度理解

- 核心概念：目录创建 (`os.mkdir`) 与遍历 (`os.walk`) 同样保持"str 进 str 出、bytes 进 bytes 出"的类型纪律——用哪种形式建目录，就能用哪种形式列出来、遍历到。
- 底层实现：`os.walk` 在内部基于 `os.scandir/os.listdir` 递归实现，每一层目录的名字都以输入的形态返回 (str 或 bytes)；`os.mkdir` 则把路径直接透传给系统调用。Windows 上路径分隔符是 `\`，所以演示里打印出 `.\sub` 这种形态。
- 为什么这样设计：让整套"文件工具家族" (`open`、`listdir`、`glob`、`mkdir`、`walk`、`fsdecode`...) 共享同一套类型契约，你无需记忆每个函数是"吃 str 还是 bytes"——输入输出永远同型。
- 解决什么实际问题：跨平台程序里，Windows 与 POSIX 的分隔符、编码、大小写规则各不同；坚持 str + 平台默认文件系统编码，能让 `mkdir/walk` 这类批量操作少踩很多坑。
- 初学者误区：试图在 bytes 列表与 str 目录名之间混用（比如拿 `os.listdir('.')` 的结果直接传给 `open(strname)` 没

问题，但把 bytes 目录项当 str 用就会失败)；以及以为 os.walk(b'.') 这种形态是"怪癖"——其实它是和 open(b'...') 一脉相承的合法 API。

代码分析

```
os.mkdir('sub') # str
os.mkdir('subbytes'.encode('utf8')) # bytes UTF-8
os.mkdir('subbytes/subsub') # Windows \ /
for (dirhere, subshere, files here) in os.walk('.'):
    print(dirhere, '=>', subshere, files here) #
```

解释器如何处理：os.mkdir 先做一次路径类型检测：str 路径经文件系统编码器（UTF-8 等）转成字节后调用 mkdir 系统调用；bytes 路径直接透传。os.walk('.') 启动一次目录迭代器，每次 next() 都由底层 scandir 返回当前目录项，再递归进入每个子目录——全部是惰性流式读取，不会一次性把整棵树载入内存。

设计思想：文件工具族的统一契约——输入形态决定输出形态——让"文件系统即字节世界"的事实与"程序里尽量用 str"的建议并行不悖，也再次印证本章主题：类型纪律是 Python 处理文本/二进制数据的护身符。

37.22 侧边栏：Unicode 默认值与 UTF-8 模式 (UNICODE DEFAULTS AND UTF-8 MODE)

英文原文

So what's the default encoding, then? This turns out to be a weirdly convoluted story, which we've touched on lightly a few times. Now that we've seen all the players in action, we can finally summarize and finalize this thread. In short, Python's default Unicode encoding for both source code and string encoding/decoding methods is always UTF-8 everywhere; for filenames can be had with `sys.getfilesystemencoding()`; and for file content accessed via `open` is fetched with `locale.getpreferredencoding(False)`. The methods default is `sys.getdefaultencoding()`, but it can no longer be changed as of Python 3.2 (and probably shouldn't have been changed earlier). The filenames default is usually UTF-8, including on Windows as of Python 3.6, but it's moot if your names are always `str` (as we learned in the preceding section). The `open` default for file content, though, is bifurcated most and badly. Contrary to Python's current docs, it's not simply the result of `locale.getencoding()` on all platforms. Rather, it's either UTF-8 if the UTF-8 mode introduced in Python 3.7 is enabled, or else the result of the locale settings. This matters on Windows today, where a code-page encoding like `cp1252` might otherwise apply; because Unix is usually UTF-8, an `open` sans encoding isn't portable for non-ASCII content. UTF-8 mode addresses this, but it must be enabled today by setting the enviro

environment variable `PYTHONUTF8` to 1 or using command-line switch `-X utf8`. When enabled, UTF-8 mode overrides the locale settings, making the platform default UTF-8—hence open's conditional default. Python 3.15 will turn UTF-8 mode on by default to match other platforms for code that lacks explicit encodings—a good idea, though too late to the party to avoid making a scene. Separately, environment variable `PYTHONIOENCODING` can be used to give the encoding of stdio streams (e.g., `sys.stdout`) when they are redirected to files on Windows and others. This is more of a legacy tool, though, and is best avoided. All that aside, the default encoding for file content is one of the more vexing of Python's platform quirks, so the universal rule remains: don't rely on defaults for anything that leaves your machine. For text, always pass an explicit encoding to `open`; for filenames, always use `str` (which defers to the filesystem's default); and for string methods, the default is always UTF-8 and is fine as-is.

中文翻译

那么，默认编码到底是多少？事实是个诡异而曲折的故事，前面我们已经轻描淡写地提过几次。既然现在所有“玩家”都已登场表演过，我们终于可以把这个线索总结收尾了。简而言之：Python 对源代码与字符串编码/解码方法的默认 Unicode 编码，在任何地方都是 UTF-8；对文件名的默认可通过 `sys.getfilesystemencoding()` 获得；而对经 `open` 访问的文件内容的默认，则用 `locale.getpreferredencoding`

(False) 取得。方法的默认值是 `sys.getdefaultencoding()`，但从 Python 3.2 起它已经不能再修改（早先大概也不该改）。文件名的默认值通常是 UTF-8（Windows 自 3.6 起也如此），但若你的文件名总是 `str`（如上一节所讲），这个默认值就无关紧要了。然而，`open` 对文件内容的默认值，却是分岔最厉害、也最麻烦的一处。与 Python 当前文档的说法相反，它在所有平台上并不单纯是 `locale.getencoding()` 的结果。相反：若启用了 Python 3.7 引入的 UTF-8 模式，则为 UTF-8；否则才取 `locale` 设置的结果。这一点在今天很重要——因为 Windows 上会适用 cp1252 这类代码页编码；而 Unix 多为 UTF-8，所以不带 `encoding` 的 `open` 对非 ASCII 内容并不可移植。UTF-8 模式正是为此而生，但它目前必须通过设置环境变量 `PYTHONUTF8` 为 1，或使用命令行开关 `-X utf8` 来启用。启用后，UTF-8 模式会覆盖 `locale` 设置，让平台默认成为 UTF-8——于是 `open` 的默认值才如此“条件化”。Python 3.15 将默认开启 UTF-8 模式，以和缺少显式编码的其它平台代码对齐——主意虽好，但来得太迟，免不了一番风波。另外，环境变量 `PYTHONIOENCODING` 可用于指定 `stdio` 流（如 `sys.stdout`）在 Windows 等平台上重定向到文件时的编码。这更像是遗留工具，最好别用。撇开这些不谈，“文件内容的默认编码”是 Python 平台怪癖里最让人头疼的一个，所以通用规则依然是：凡是会离开你机器的数据，都别依赖默认值。对文本，永远给 `open` 传显式 `encoding`；对文件名，永远用 `str`（它会顺延到文件系统的默认值）；对字符串方法，默认永远是 UTF-8，保持原样即可。

深度理解

·核心概念：Python 有三层默认编码：源码/方法层（`sys.getdefaultencoding()`，钉死 UTF-8）、文件名层（`sys.getfilesystemencoding()`，多为 UTF-8）、文件内容层（`locale.getpreferredencoding(False)`，最易变、跨平台差异最大）。

·底层实现：解释器启动时若读到 `PYTHONUTF8=1` 或 `-X utf8`，会设置内部 UTF-8 模式标志；`io/codecs` 创建 `TextIOWrapper` 时先查该标志，再决定用 UTF-8 还是 `locale` 编码。`sys.getdefaultencoding()` 是编译期常量，`getfilesystemencoding()` 在 3.6+ 的 Windows 上也是 UTF-8（映射到 Windows API 的 UTF-16）。

·为什么这样设计：源码与方法层面需要"确定性"，所以固定 UTF-8；文件内容层面要"尊重主机生态"（Windows 的 cp1252、Unix 的 UTF-8），所以跟随 `locale`——两者目的不同，默认就不同。

·解决什么实际问题：这是"我的代码在 Mac 上正常、在 Windows 上乱码"这一世界级疑难的最常见根源。记住三者并存，你就能快速定位：是 `open()` 忘了传 `encoding`，还是文件名/方法层面的误解。

·初学者误区：以为"Python 默认编码= UTF-8"可以一劳永逸。对 `str.encode()/str.decode()` 成立；对 `open()` 不成立（Windows 上可能是 cp1252）。而 `sys.getdefaultencoding()` 自 3.2 起不可再改，也改不动 `open()` 的默认。

代码分析

```
import sys, locale
sys.getdefaultencoding()           # 'utf-8' UTF-8
sys.getfilesystemencoding()        # 'utf-8' /
locale.getpreferredencoding(False)  # 'cp1252' Windows 'utf-8'...
open('f.txt', encoding='utf-8')    #
```

解释器如何处理：`locale.getpreferredencoding(False)` 调用 C 运行时按当前 locale 查询编码；在 UTF-8 模式（3.7+，`-X utf8/PYTHONUTF8=1`）下其结果会被覆盖为 UTF-8。`open` 在构造文本流时读取这一结果作为默认解码器，所以同一段代码在“默认 locale”与“UTF-8 模式”两种环境下可能产生不同行为——这就是“条件式默认”的全部真相。

设计思想：把“确定性”与“平台适配”分开治理：核心默认（方法）钉死 UTF-8，平台默认（文件）跟随 locale，再用一个总开关（UTF-8 模式）做全局兜底。对工程师的启示：不要赌默认值，显式参数才是跨平台的保险。

37.23 Unicode 黄昏地带 (The Unicode Twilight Zone)

英文原文

The Unicode Twilight Zone To wrap up, this chapter is going to briefly present two curated topics from the Unicode realm—BOMs and normalization—that are too convoluted for full coverage in this book and pro

bably too arcane to matter to most Python newcomers. If and when these become important to your projects, you'll find ample resources in both Python's docs and the web at large to fill in the bits glossed over here for space. For now, let's jump right into the first of these arguably explosive topics.

中文翻译

在收尾之际，本章将简要介绍 Unicode 世界中两个精挑细选的话题——BOM 与规范化（normalization）。它们过于曲折，本书无法完整覆盖，对大多数 Python 新手来说大概也太冷门。等它们真正对你手头的项目变得重要时，Python 文档与整个互联网都有足够的资料，足以补齐这里因篇幅而略过的部分。现在，让我们直接跳进这两个“火药味十足”的话题中的第一个。

深度理解

- 核心概念：本章尾声两题——BOM（字节序标记）解决“文件开头该怎样自报家门”，规范化解解决“同一个字符有多种写法”；它们都是 Unicode 标准自身留下的“历史债”。
- 底层实现：BOM 是写在文件开头的一小段特殊字节序列（UTF-8 为 `\xef\xbb\xbf`，UTF-16 为 `\xff\xfe` 或 `\xfe\xff`）；规范化则是把字符序列按 Unicode 标准映射表重排成规范形式。
- 为什么这样设计：Unicode 标准既要兼容旧编码（BOM 源于 UTF-16 的字节序之争），又要兼顾效率（合成字符 vs. 组合字符），于是留下了两处“标准内的分歧”。

·解决什么实际问题：读别人生成的文件时 BOM 会让你莫名其妙多出 `\uffeff`；从不同平台拿来的文件名/文本做相等比较时会“看起来一样却比较失败”。

·初学者误区：以为“UTF-8 文件一定有 BOM”或“规范化后 == 就永远可靠”——两者都需要显式处理，不能指望自动。

代码分析

```
# BOMUTF-8 BOM 3 \xef\xbb\xbfUTF-16 BOM 2
b'\xef\xbb\xbf'.decode('utf-8-sig') # BOM → ''
'\uffeff' # BOM str U+FEFF...
# NFC / NFD
from unicodedata import normalize
normalize('NFC', 'n\u0303') == 'ñ' # True
```

解释器如何处理：文本模式 `open` 按编码名构造解码器：`utf-8-sig/utf-16` 会先吞掉 (`skip`) 开头的 BOM 再解码；`utf-8` 则把 BOM 当作普通字符 `U+FEFF` 原样解码出来——所以 `open(f,'r').read()` 前会多一个不可见字符。规范化则是纯内存操作，由 `unicodedata.normalize` 依据 Unicode 数据库完成。

设计思想：Unicode 标准允许“同一字符多写法”，是为了与既有数据兼容；Python 则坚持“机器可判定”的纪律——用 BOM 后缀与规范化补全了标准的缝隙，正如本书一贯强调的：兼容现实，但纪律在自己的代码里。

37.11.1 在 Python 中去掉 BOM (Dropping the BOM in Python)

英文原文

As noted briefly earlier in this chapter, some encoding schemes store a special byte order marker (BOM) sequence at the start of files to specify data endianness (which end of a string of bits is most significant to its value) or declare the encoding type in general. Python's text-mode files both skip this marker on input and write it on output if the encoding implies presence, but we sometimes must use a specific encoding name to force BOM processing explicitly and may need to accommodate it when encoding manually. For example, in the UTF-16 and UTF-32 encodings, the BOM both identifies the encoding and specifies big- or little-endian format. A UTF-8 text file may also include a BOM, but this isn't guaranteed and serves only to declare that it is UTF-8 in general. When reading and writing data using these encoding schemes, Python automatically skips or writes the BOM if it is either implied by a general encoding name or if you provide a more specific encoding name to force the issue. More concretely:- In UTF-16, the BOM is always processed for encoding name utf-16, and the more specific encoding name utf-16-le denotes little-endian format.- In UTF-8, the more specific encoding name utf-8-sig

forces Python to both skip and write a BOM on input and output, respectively, but the general utf-8 does not.

中文翻译

正如本章前面简短提到过的：某些编码方案会在文件开头存储一段特殊的字节序标记（BOM）序列，用来指明数据的字节序（一串比特中哪一端对其数值最有效），或者笼统地声明编码类型。Python 的文本模式文件在输入时会跳过这个标记，在输出时若编码暗示应带 BOM 则会写上它；但我们有时必须用更具体的编码名来显式强制 BOM 处理，手工编码时也可能需要自己应对它。例如，在 UTF-16 和 UTF-32 编码中，BOM 既标识编码类型，又指明大端或小端格式。UTF-8 文本文件也可能包含 BOM，但这并不保证，且 BOM 只是笼统地声明“这是 UTF-8”。当用这些编码方案读写数据时，只要 BOM 被一般性的编码名所隐含，或者你提供了更具体的编码名来强制此事，Python 就会自动跳过或写入 BOM。说得更具体些：- 在 UTF-16 中，编码名 utf-16 总是会处理 BOM，而更具体的编码名 utf-16-le 表示小端格式。- 在 UTF-8 中，更具体的编码名 utf-8-sig 会强制 Python 在输入时跳过、输出时写入 BOM；而一般性的 utf-8 则两者都不做。

深度理解

·核心概念：BOM 是文件开头的“自报家门”字节序列；Python 的规则是——通用编码名隐式处理 BOM（utf-16），带后缀的编码名显式强制处理（utf-8-sig），而“裸”的 utf-8 一概不碰。

·底层实现：utf-8-sig 解码器在流开头检测 EF BB BF 三字节并丢弃；utf-16 解码器按开头 FF FE/FE FF 判定字节序并丢弃 BOM；utf-16-le/be 则不做任何检测。写入时同理：utf-8-sig 在开头先写 EF BB BF，utf-16 按平台字节序写 FF FE 或 FE FF。

·为什么这样设计：BOM 源于 UTF-16 时代的字节序歧义；UTF-8 本不需要它，但 Windows 记事本历史习惯写 BOM。Python 于是把"要不要 BOM"做成显式选择——默认 utf-8 不带，需要时用 utf-8-sig。

·解决什么实际问题：从记事本/Excel/旧系统读来的文件常带 BOM；用 encoding='utf-8' 打开会得到开头多出的 \uffeff（零宽空格），参与比较、JSON 解析、split 时会产生诡异的 bug。

·初学者误区：把 utf-8-sig 当成"某种特殊 UTF-8"。它本质就是 UTF-8 + 自动处理 BOM 的开关，专为"别人家的文件"设计。

代码分析

```
open('code.txt', 'rb').read()           # b'\xef\xbb...
open('code.txt', 'r').read()             # 'ï»¿code\nCOD...
open('code.txt', 'r', encoding='utf-8').read() # '\uffeffcode\n...
open('code.txt', 'r', encoding='utf-8-sig').read() # 'code\nCODE\n...
```

解释器如何处理：文本解码器（utf-8 与 utf-8-sig 是两个不同的编解码器对象）在输入流起始处做 BOM 检测：utf-8-sig 命中 EF BB BF 即丢弃后继续解码；utf-8 不检测，把 EF BB BF 解成 ï»¿ 三个字符。\\uffeff 是它在 str 中的身份——同一码位 U+FEFF 既当"字节序标记"又当"

零宽不换行空格", 上下文决定其意义。

设计思想: 用编码名承载"要不要处理 BOM"的策略, 让 API 表面保持简单 (open 签名不变), 把平台历史包袱藏进编码器的名字里——需要时显式点名, 不需要时保持零成本。

37.11.2 在文本编辑器中生成 BOM (Making BOMs in Text Editors)

英文原文

Let's make some files with BOMs to see how this works in practice. We're going to do this in Python in a moment, too, but let's start by creating a file in a text editor to underscore that your programs will also have to process content from other sources. If you wish to work along, you can use any text editor that's Unicode aware for saves, and most editors on PCs, tablets, and phones are today. This demo uses Windows Notepad just because it's widespread (and documenting multiple editors' usage here is right out). When you save a text file in Windows Notepad, you can specify its encoding type in a drop-down list—simple text, little- or big-endian UTF-16, or UTF-8 with or without a BOM. For instance, if you use Notepad to save the two lines "code" and "CODE" in a text file named code.txt with encoding type ANSI, the file is written as simple ASCII text without a BOM. Technically, the save use

s the cp1252 default encoding on the Windows host used, but cp1252 is an ASCII superset, and the content is all ASCII. In Python after the save, when this file is read in binary mode, we can see the actual bytes stored in the file, including `\r` in newlines. When it's read as text, Python performs newline translation by default, and we can also decode it explicitly as UTF-8 text since ASCII is a subset of both this encoding and Windows' default cp1252:

```
$ py -3 # On Windows, post ANSI save in Notepad >
>> import locale >>> locale.getpreferredencoding(False)
'cp1252' # open default: ASCII&Latin-1 superset
>>> open('code.txt','rb').read()
b'code\r\nCODE\r\n' # ASCII (and cp1252 and UTF-8) bytes
>>> open('code.txt','r').read()
'code\nCODE\n' # Text mode also translates newlines
>>> open('code.txt','r', encoding='utf-8').read()
'code\nCODE\n'
```

Now, if this file is instead saved as UTF-8 with BOM in Notepad (UTF in its prior versions), its text is prepended with a three-byte UTF-8 BOM sequence (which prints as non-ASCII `\xNN` hex escapes in bytes in Python), and we need to give the more specific encoding name `utf-8-sig` to force Python to skip the marker automatically on input (else it prints as a `\uNNNN` Unicode code-point escape in str):

```
>>> open('code.txt','rb').read() b'\xef\xbb\xbfcode\r\nCODE\r\n' # After resaved: UTF-8 with 3-byte BOM
```

```
>>> open('code.txt','r').read() "ï»¿code\r\nCODE\r\n" # Default+utf8 keep BOM, utf-8-sig drops
```

```
>>> open('code.txt','r', encoding='utf-8').read() '\ufeffcode\r\nCODE\r\n'
```

```
>>> open('code.txt','r', encoding='utf-8-sig').read() 'code\r\nCODE\r\n'
```

And if the file is stored as UTF-16 BE in Notepad (Unicode big endian formerly), we get UTF-16 big-endian format data in the file, with two bytes (16 bits) prepended to record a two-byte BOM sequence. The encoding name utf-16 in Python skips the BOM because it is implied (since all UTF-16 files have a BOM), and utf-16-be handles the big-endian format but does not skip the BOM in the input result:

```
>>> open('code.txt','rb').read() b'\xfe\xff\x00c\x00o\x00d\x00e\x00r\x00\n\x00C\x00O\x00D\x00E\x00r\x00\n'
```

```
>>> open('code.txt','r', encoding='utf-16').read() 'code\r\nCODE\r\n'
```

```
>>> open('code.txt','r', encoding='utf-16-be').read() '\uffeffcode\r\nCODE\r\n'
```

Experiment with other save/open combinations for more insights. The default encoding in the last example, for example, would print garbage characters bec

ause it's not valid for the text, and UTF-16 little-endian swaps byte order for the BOM and each 2-byte character. NOTE: Notepad flux: Notepad recently changed its encoding options for saves. Today, it offers ANSI (the host's default), UTF-16 in little- and big-endian flavors, and UTF-8 with and without a BOM, and defaults to UTF-8. When this book's prior edition was penned, Notepad had UTF-8 with an implied BOM, and it's "Unicode" meant UTF-16—which, of course, is just one of the very many kinds of Unicode encoding. The narrative here has been updated to reflect the new choices, but this naturally is just the story today. Because this book staunchly refuses to become a Notepad doc, translate save options as needed to the Notepad on a PC near you.

中文翻译

让我们亲手制造一些带 BOM 的文件，看看这套机制实际怎么运转。稍后我们也会用 Python 来做同样的事，但先从文本编辑器创建文件开始，以强调你的程序终将处理来自其它来源的内容。想跟着动手的话，可以用任何"保存时支持 Unicode"的文本编辑器——如今 PC、平板、手机上的多数编辑器都可以。这个演示用 Windows 记事本，纯粹因为它普及（而本书也不可能——介绍各家编辑器的用法）。在 Windows 记事本里保存文本文件时，你可以在下拉列表中指定编码类型——纯文本、小端或大端 UTF-16、带或不带 BOM 的 UTF-8。比如，用记事本把 "code" 和 "CODE" 两行以 ANSI 编码类型保存成名为 code.txt 的文件，写出来

的就是不带 BOM 的纯 ASCII 文本。严格说，本次保存用的是所用 Windows 主机上的 cp1252 默认编码，但 cp1252 是 ASCII 的超集，而内容全是 ASCII。保存之后回到 Python：用二进制模式读这个文件，可以看到文件里真实的字节，包括换行中的 \r。用文本模式读时，Python 默认做换行翻译；由于 ASCII 既是 UTF-8 的子集也是 Windows 默认 cp1252 的子集，我们还可以显式按 UTF-8 文本解码它：\$ py -3 # 在 Windows 上，记事本以 ANSI 保存之后

```
>>> import locale >>> locale.getpreferredencoding(False)'cp1252' # open 默认：ASCII 与 Latin-1 的超集>>> open('code.txt','rb').read() b'code\r\nCODE\r\n' # ASCII (也是 cp1252 与 UTF-8) 字节>>> open('code.txt','r').read()'code\r\nCODE\r\n' # 文本模式还会翻译换行>>> open('code.txt','r', encoding='utf-8').read()'code\r\nCODE\r\n'现在，如果这个文件改在记事本中以 UTF-8 带 BOM 保存（旧版本里写作 UTF），其文本首部会被缀上一个三字节的 UTF-8 BOM 序列（在 Python 的 bytes 里打印成非 ASCII 的 \xNN 十六进制转义），我们就要给出更具体的编码名 utf-8-sig，强制 Python 在输入时自动跳过该标记（否则它会在 str 里打印成 \uNNNN Unicode 码位转义）：>>> open('code.txt','rb').read() b'\xef\xbb\xbfcode\r\nCODE\r\n' # 重新保存后：带 3 字节 BOM 的 UTF-8>>> open('code.txt','r').read()'\ufeffcode\r\nCODE\r\n' # 默认与 utf-8 都保留 BOM，utf-8-sig 才丢弃>>> open('code.txt','r', encoding='utf-8').read()'\ufeffcode\r\nCODE\r\n'>>> open('code.txt','r', encoding='utf-8-sig').read()'code\r\nCODE\r\n'而如果文件以 UTF-16 BE 存入记事本（旧版本写作 Unicode big endian），文件里就是
```

UTF-16 大端格式的数据，开头缀上两个字节（16 位）记录一个两字节的 BOM 序列。Python 的编码名 utf-16 会跳过 BOM，因为它已被隐含（所有 UTF-16 文件都有 BOM）；utf-16-be 则按大端格式处理，但不会在输入结果里跳过 BOM：

```
>>> open('code.txt','rb').read() b'\xfe\xff\x00c\x00o\x00d\x00e\x00r\x00n\x00C\x00O\x00D\x00E\x00r\x00n'>>> open('code.txt','r', encoding='utf-16').read() 'code\nCODE\n'>>> open('code.txt','r', encoding='utf-16-be').read() '\ufeffcode\nCODE\n'
```

不妨再试试其它保存/打开组合，会得到更多启发。比如，最后一个例子若用默认编码打开，就会打印出乱码——因为它不适用于这份文本；而 UTF-16 小端会把 BOM 与每个双字节字符的字节序都颠倒过来。注意：记事本的变动：记事本最近改了它的保存编码选项。如今它提供 ANSI（主机默认）、小端与大端两种 UTF-16，以及带或不带 BOM 的 UTF-8，并且默认是 UTF-8。本书上一版写作时，记事本是“带隐含 BOM 的 UTF-8”，而且它的“Unicode”指的其实是 UTF-16——当然，它只是众多 Unicode 编码中的一种。这里的叙述已按新选项更新，但自然只是“今天的故事”。因为本书坚决不打算变成记事本说明书，请自行把保存选项对应到你手边 PC 上的记事本。

深度理解

·核心概念：BOM 的出现源于记事本等外部工具的保存习惯；Python 的 utf-8-sig 是为“消化外部文件”而生的解码入口，utf-16 则默认处理 BOM、utf-16-be/le 原样保留。

·底层实现：UTF-16 编码的 BOM 是 \xff\xfe（小端）或 \xfe\xff（大端），utf-16 解码器据此前两字节判定字节序并剥离；utf-16-be 固定按大端解码且不剥离 BOM——所以示例里'\uffeffcode\nCODE\n' 的 \uffeff 正是那份 BOM 在 str 中的残留。

·为什么这样设计：Windows 记事本的"ANSI / Unicode / UTF-8"三档历史悠久、语义混乱（"Unicode"实为 UTF-16）；Python 无法改变外部工具，只能在解码层提供"跳过 BOM"的选项来兼容现实。

·解决什么实际问题：项目里读 CSV/配置文件出现首列名字带 \uffeff、JSON 解析报 "Unexpected UTF-8 BOM" 等，十有八九是文件带 BOM 而用了 utf-8 打开；换 utf-8-sig 立解。

·初学者误区：以为"文本文件就是字符"。文本模式确实省心，但理解 BOM 就必须先看字节——这也是本章反复强调二进制模式的价值所在。

代码分析

```
# ANSI BOM ASCII
open('code.txt', 'rb').read()           # b'code\r\nCOD...

# UTF-8 with BOM 3 BOM
open('code.txt', 'rb').read()           # b'\xef\xbb\xbfcode\r\nCOD...
open('code.txt', 'r', encoding='utf-8-sig').read() # 'code\nCODE\n...

# UTF-16 BE 2 BOM + 2
open('code.txt', 'r', encoding='utf-16').read() # 'code\nCODE\n'
open('code.txt', 'r', encoding='utf-16-be').read() # '\uffeffcode\n...
```

解释器如何处理：文本解码器在流起始做一次"BOM 探测"：utf-8-sig 看到 EF BB BF 就丢弃；utf-16 看到 FF FE/

FE FF 既定字节序又丢弃。随后才按各自方案把剩余字节解码成字符；解码后再进入换行翻译（`\r\n`→`\n`），所以文本模式的最终字符串里看不到 `\r`。

设计思想：BOM 是“编码自描述”的典型——文件自己告诉读者“我是谁”；但自描述也给“按字节逐字处理”的管道带来污染。Python 的取舍是：标准编码名不带 BOM 语义，只有专门后缀才介入，把决定权留给程序员。

37.11.3 在 Python 中生成 BOM (Making BOMs in Python)

英文原文

The preceding section used Python to read files made in an editor, but the same patterns apply when Python also makes the files: Python will automatically add a BOM when we write a Unicode file in code, but we need to use a more explicit encoding name to force the BOM in UTF-8 mode—utf-8 does not write the BOM on output but utf-8-sig does, and the same goes for skipping UTF-8 BOMs on input.

When run on Windows (Unix is the same, but doesn't add `\r` to newlines, and usually defaults to UTF-8):

```
>> open('temp.txt','w', encoding='utf-8').write('code\nCODE\n') 10 >> open('temp.txt','rb').read() b'code\r\nCODE\r\n' # utf-8: no BOM >> open('temp.txt','w', encoding='utf-8-sig').write('code\nCODE\n') 10 >> o
```

```

pen('temp.txt','rb').read() b'\xef\xbb\xbfcode\r\nCODE\r\n' # utf-8-sig: adds BOM >> open('temp.txt','r').read() 'i»¿code\r\nCODE\r\n' # Default: bad BOM >> open('temp.txt','r', encoding='utf-8').read() '\uffeffcode\r\nCODE\r\n' # utf-8: keeps BOM >> open('temp.txt','r', encoding='utf-8-sig').read() 'code\r\nCODE\r\n' # utf-8-sig: drops BOM

```

Per this code, although utf-8 does not drop the BOM when one is present, data without a BOM can be read with both utf-8 and utf-8-sig—which means you can use the latter for input if you're not sure whether a BOM is present in a file or not (and don't read this paragraph out loud in an airport security line!):

```

>> open('temp.txt','w').write('code\r\nCODE\r\n') 10 # Default: UTF-8 subset >> open('temp.txt','rb').read() b'code\r\nCODE\r\n' # No BOM >> open('temp.txt','r').read() 'code\r\nCODE\r\n' # Default >> open('temp.txt','r', encoding='utf-8').read() 'code\r\nCODE\r\n' # Either utf-8 works >> open('temp.txt','r', encoding='utf-8-sig').read() 'code\r\nCODE\r\n'

```

For the encoding name utf-16, the BOM is handled automatically: on output, data is written in the platform's native endianness, and the BOM is always written; on input, data is decoded per the BOM, and the BOM is always stripped. This reflects the fact that BOMs are standard and required in the UTF-16 encoding scheme:

```
>> import sys >> sys.byteorder'little' # Windows host's default endianness >> open('temp.txt','w', encoding='utf-16').write('code\nCODE\n') 10 >> open('temp.txt','rb').read() b'\xff\xfe\x00o\x00d\x00e\x00r\x00\n\x00C\x00O\x00D\x00E\x00r\x00\n\x00' >> open('temp.txt','r', encoding='utf-16').read() 'code\nCODE\n'
```

More specific UTF-16 encoding names can specify different endianness, though you may have to manually write and skip the BOM yourself in some scenarios if it is required or present—study the following examples for more BOM-making instructions (sorry):

```
>> open('temp.txt','r', encoding='utf-16-le').read() '\uffeffcode\nCODE\n' >> open('temp.txt','w', encoding='utf-16-be').write('\uffeffcode\nCODE\n') 11 >> open('temp.txt','rb').read() b'\xfe\xff\x00c\x00o\x00d\x00e\x00r\x00\n\x00C\x00O\x00D\x00E\x00r\x00\n' >> open('temp.txt','r', encoding='utf-16').read() 'code\nCODE\n' >> open('temp.txt','r', encoding='utf-16-be').read() '\uffeffcode\nCODE\n'
```

The more specific UTF-16 encoding names by themselves create and work fine with BOM-less files, though utf-16 requires one on input in order to determine byte order:

```
>> open('temp.txt','w', encoding='utf-16-le').write('CODE') 4 >> open('temp.txt','rb').read() b'C\x00O\x00D\x00E\x00' # Okay if BOM not present or expected
```

```
>> open('temp.txt','r', encoding='utf-16-le').read()
'CODE'>> open('temp.txt','r', encoding='utf-16').read()
UnicodeError: UTF-16 stream does not start with BOM
```

Experiment with these encodings yourself, or see Python's library manuals for more details on the BOM. And if you really want to drop the bomb in Python (and stretch this section's silly bit to its breaking point), Unicode emoji characters do the job:

```
>> open('boms.txt','w', encoding='utf-8-sig').write('
10 10')>> open('boms.txt', encoding='utf-8-sig').read()
```

中文翻译

上一节是用 Python 读取编辑器生成的文件，但当文件由 Python 自己生成时，同样的规律照样成立：在代码里写一个 Unicode 文件时 Python 会自动添加 BOM，但在 UTF-8 模式下要强制 BOM 就得用更明确的编码名——utf-8 输出时不写 BOM，而 utf-8-sig 会写；跳过输入端的 UTF-8 BOM 同理。以下在 Windows 上运行（Unix 也一样，只是不给换行加 \r，且通常默认 UTF-8）：

```
>>> open('temp.txt','w', encoding='utf-8').write('code\nCODE\n')
10>>> open('temp.txt','rb').read()
b'code\r\nCODE\r\n' # utf-8: 无 BOM
>>> open('temp.txt','w', encoding='utf-8-sig').write('code\nCODE\n')
10>>> open('temp.txt','rb').read()
b'\xef\xbb\xbfcode\r\nCODE\r\n' # utf-
```


8-sig: 添加 BOM >>> open('temp.txt','r').read() 'ï»¿code\nCODE\n' # 默认: BOM 变乱码>>> open('temp.txt','r', encoding='utf-8').read() '\uffeffcode\nCODE\n' # utf-8: 保留 BOM >>> open('temp.txt','r', encoding='utf-8-sig').read() 'code\nCODE\n' # utf-8-sig: 丢弃 BOM

按这段代码, utf-8 在遇到已有 BOM 时并不会丢弃它; 但没有 BOM 的数据用 utf-8 和 utf-8-sig 都能读——也就是说, 当你不确定文件里有没有 BOM 时, 输入可以直接用后者 (当然, 别在机场安检排队时把这段念出声!) : >>> open('temp.txt','w').write('code\nCODE\n') 10 # 默认: UTF-8 子集>>> open('temp.txt','rb').read() b'code\nCODE\n' # 无 BOM >>> open('temp.txt','r').read() 'code\nCODE\n' # 默认>>> open('temp.txt','r', encoding='utf-8').read() 'code\nCODE\n' # 两种 utf-8 都行>>> open('temp.txt','r', encoding='utf-8-sig').read() 'code\nCODE\n'

对于编码名 utf-16, BOM 是自动处理的: 在输出端, 数据按平台原生字节序写入, 且总写 BOM; 在输入端, 数据按 BOM 解码, 且 BOM 总被剥掉。这正反映了 BOM 在 UTF-16 编码方案中是标准且必需的这一事实: >>> import sys >>> sys.byteorder 'little' # Windows 主机的默认字节序>>> open('temp.txt','w', encoding='utf-16').write('code\nCODE\n') 10 >>> open('temp.txt','rb').read() b'\xff\xfe\x00o\x00d\x00e\x00r\x00\n\x00C\x00O\x00D\x00E\x00r\x00\n\x00' >>> open('temp.txt','r', encoding='utf-16').read() 'code\nCODE\n'

更具体的 UTF-16 编码名可以指定不同的字节序, 不过在某些场景下, 如果 BOM 是必需或已存在的, 你可能得自己手工写或跳过它——请研读下面的例子获取更多“制造 BOM”的

指导（抱歉）：

```
>>> open('temp.txt','r', encoding='utf-16-le').read()'\uffcode\nCODE\n'>>> open('temp.txt','w', encoding='utf-16-be').write('\uffcode\nCODE\n') 11 >>> open('temp.txt','rb').read() b'\xfe\xff\x00c\x00o\x00d\x00e\x00r\x00n\x00C\x00O\x00D\x00E\x00r\x00n'>>> open('temp.txt','r', encoding='utf-16').read()'\code\nCODE\n'>>> open('temp.txt','r', encoding='utf-16-be').read()'\uffcode\nCODE\n'这些更具体的 UTF-16 编码名本身可以很好地创建和处理无 BOM 的文件，不过 utf-16 在输入端必须有 BOM 才能判定字节序：
```

```
>>> open('temp.txt','w', encoding='utf-16-le').write('CODE') 4 >>> open('temp.txt','rb').read() b'C\x00O\x00D\x00E\x00' # 只要没有（或不需要）BOM 就没问题>>> open('temp.txt','r', encoding='utf-16-le').read()'\CODE'>>> open('temp.txt','r', encoding='utf-16').read() UnicodeError: UTF-16 stream does not start with BOM
```

你可以亲自实验这些编码，或查阅 Python 库手册获取 BOM 的更多细节。而如果你真想“在 Python 里引爆（drop the bomb）”（把这节那个小双关梗用到极限），Unicode emoji 字符就能胜任：

```
>>> open('boms.txt','w', encoding='utf-8-sig').write(' 10') 10 >>> open('boms.txt', encoding='utf-8-sig').read()''
```

深度理解

·核心概念：写文件时同样遵守“编码名决定 BOM 行为”——utf-8 写不带 BOM 的文件，utf-8-sig 写带 BOM 的；utf-16 自动处理 BOM（写总写、读总剥），而 utf-16-le/be 把 BOM 的选择权交给你。

·底层实现：utf-16 编码器写出平台字节序 (sys.byteorder, Windows 是 little) 的 BOM FF FE; utf-16 解码器强制要求流以 BOM 开头, 否则抛 UnicodeError。utf-16-le/be 编码器不写 BOM, 解码器也不剥 BOM, 所以跨端转换时可能需手工补'\ufeff' 前缀。

·为什么这样设计：UTF-16 规范强制 BOM 存在 (无它无法判定字节序), 所以 utf-16 必须处理; 但 -le/-be 已固定字节序, BOM 成为可选项, Python 便不代劳。

·解决什么实际问题：要给 Excel/记事本提供带 BOM 的 UTF-8 文件 (否则中文首列乱码), 用 encoding='utf-8-sig' 写入即可; 要写"最兼容"的 UTF-16 文件则用 utf-16, 让平台与 BOM 都交给 Python。

·初学者误区：以为 write 返回的字符数与文件字节数一致。示例里'code\nCODE\n' 写入返回 10 (字符数), 但 UTF-16 文件有 22 字节 + BOM——字节数 ≠ 字符数, 这是编码恒常的错觉。

代码分析

```
open('temp.txt', 'w', encoding='utf-8-sig').write('code\nCODE\n...
open('temp.txt', 'rb').read()    # b'\xef\xbb\xbfcode\r\nCODE\r\n'...
open('temp.txt', 'w', encoding='utf-16').write('code\nCODE\n')
open('temp.txt', 'rb').read()    # b'\xff\xfe' + 2 UTF-16
open('temp.txt', 'w', encoding='utf-16-le').write('CODE')           ...
open('temp.txt', 'r', encoding='utf-16').read()    # UnicodeError: ...
```

解释器如何处理：编码器按编码名选择"BOM 策略": utf-8-sig 写流前先输出 EF BB BF; utf-16 写平台字节序 BOM 且逐字符 2 字节编码 (Windows 小端→FF FE)。解

码端 utf-16 读前两字节判定字节序、剥离 BOM；读到不以 BOM 开头的流则直接报 UnicodeError——所以"无 BOM 的 UTF-16"只能用 -le/-be 读。

设计思想：写 BOM 与剥 BOM 是同一枚硬币的两面——utf-8-sig 把"带 BOM"做成显式选择，utf-16 把"必须带 BOM"做成自动行为；规则清晰，代价是 -le/-be 场景下程序员要自己扛 BOM 这个包袱。这也正是全章主题的缩影：编码世界的每一个字节都有约定，Python 帮你守着这些约定。

37.24 Unicode 规范化：标准在何方？ (Unicode Normalization: Whither Standard?)

英文原文

Last but not least, after devoting dozens of pages to Unicode, we'll close by explaining one way in which it , at least arguably, falls short. In brief, this standard failed to standardize code points for a handful of characters: it allows the same character to be represented in more than one way, which breaks equality testing and can wreak interoperability havoc with text-processing programs. While there may have been valid rationales for this policy, it adds a special case for programmers and undoubtedly breaks many a tool and app. For example, the character ñ (an n augmented with a tilde, commonly used in Spanish) can be represented

d with two different code-point sequences in the Unicode standard: - As a single character with code point `\u00F1`—an augmented n - As a two-codepoint sequence `\u006E` and `\u0303`—a naked n followed by a combining tilde The first of these is called the composed form, known as NFC, because it combines letter and accent. The second is called the decomposed form, known as NFD, because its parts are split. Despite their glaring difference, the Unicode standard mandates that these two forms represent the same character ñ and must be treated as such by programs. It's easy to observe these alter egos in Python: the following uses `\u...` Unicode code-point escapes to specify the code points for ñ in both forms (Windows users: don't be alarmed if some characters, especially NFDs, render differently in command-line interfaces due to settings that are well beyond this book's scope):

```
>>> L = '\u00F1' # NFC form ('\xF1' works too) >>>
M = '\u006E\u0303' # NFD form >>> L, M # The same
character ('ñ','n')
```

Nor is this character alone in its split personality. Other characters such as Å, è, é, and ÿ have both NFC and NFD representations as well:

```
>>> '\u00C5', '\u0041\u030A' # NFD (1 code point),
NFC (2 code points) ('Å','A')
>>> '\u00E8', '\u0065\u0300' # But it's the same character
for both ('è','e')
```

```
>>> '\u00E9', '\u0065\u0301'('é','e')
```

```
>>> '\u03D4', '\u03D2\u0308'('ÿ','Y')
```

Importantly, this is not about the encoded representation of this character in bytes, which naturally varies across different encodings. The two alternative representations for each preceding character differ for their decoded, in-memory representation as integer code points. Encoded forms (e.g., stored in files) differ, too, but decode to the same differing code points:

```
>>> '\u00F1'.encode('utf8'), '\u006E\u0303'.encode('utf8') (b'\xc3\xb1', b'n\xcc\x83')
```

```
>>> b'\xc3\xb1'.decode('utf8'), b'n\xcc\x83'.decode('utf8') ('ñ','n')
```

The unfortunate consequence of this plurality is that it breaks equality testing: because the same character can be represented in multiple ways, it's impossible to compare with the usual tools. In Python specifically, the `==` equal-by-value operator, which compares characters (really, code points), won't suffice in the presence of Unicode doppelgängers:

```
>>> L = '\u00F1'>>> M = '\u006E\u0303'>>> L, M #  
The same character ('ñ','n')
```

```
>>> L == M # But equality says no! False
```

```
>>> len(L), len(M) # Because their code points differ  
(1, 2)
```

All of which might not be a problem in an ideal computing world that settled on one form as a de facto standard for portability's sake. Unfortunately, that's not the world we occupy. Computer vendors, being computer vendors, have opted to favor different forms: broadly speaking, macOS prefers NFD, Windows and others prefer NFC, this can also vary by filesystem, and tolerance of nonnative forms is less than complete. The net effect is that the Unicode standard created an interoperability problem while trying to fix another! So what to do with a world that just won't standardize? The trick is that, for every tool that processes file contents or names across divergent platforms, text must be converted to a common form before comparisons. And luckily, Python makes this remarkably easy:

```
>>> from unicodedata import normalize
>>> L = '\u00F1'
>>> M = '\u006E\u0303'
>>> L, M # Same char, diff code points ('ñ','n')
```

```
>>> L == M # Equality fails False
```

```
>>> normalize('NFC', L) == normalize('NFC', M) # Common-form equality works True
```

```
>>> normalize('NFD', L) == normalize('NFD', M) # Either suffices, if the same True
```

Programs that compare filenames sent between platforms, for example, should be careful to normalize this way. Otherwise, files whose names look the same to users (and really are the same per Unicode) will fail

to match by simple equality and derail searches and syncs. The same goes for text files obtained from arbitrary sources—like most internet content. There's more to this story (e.g., the canonical equivalence of NFC and NFD normalized forms is still not the same as compatibility, though most programs don't need to care). Because we've run tight on space, though, we'll stop short. If you'd like to dig deeper, you'll find ample follow-up coverage both on the web and in Python's manual set (see the latter's Unicode HOWTO and its coverage of the related string casefold method). At the least, though, you now shouldn't be wholly surprised when your text-processing code is bitten by this curious choice of the Unicode standard.

中文翻译

最后但同样重要的是，在花了数十页讲 Unicode 之后，我们以“它至少在某个方面有所欠缺”收尾。简单说：这个标准没能标准化少数几个字符的码位——它允许同一个字符有多种表示方式，这破坏了相等性测试，并可能给文本处理程序带来互操作性的浩劫。这项政策也许有其合理考量，但它给程序员平添了一个特例，而且无疑搞坏了许多工具与应用。例如，字符 ñ（加了波浪号的 n，西班牙语常用）在 Unicode 标准里可以用两种不同的码位序列表示：- 作为码位为 `\u00F1` 的单个字符——一个“增强型” n- 作为 `\u006E` 与 `\u0303` 的两码位序列——一个“裸” n 后跟一个组合波浪号前者叫合成（composed）形式，即 NFC，因为它把字母和重音合为一体；后者叫分解（decomposed）形式

，即 NFD，因为它的部件是拆开的。尽管两者差别显著，Unicode 标准仍规定这两种形式表示同一个字符 ñ，程序必须把它们一视同仁。在 Python 里很容易看到这对"二重身"：下面用 \u... Unicode 码位转义分别指定 ñ 两种形式所对应的码位（Windows 用户注意：部分字符，尤其是 NFD 形式，在命令行界面里可能因某些超出本书范围的设置而渲染不同，不必惊慌）：

```
>>> L = '\u00F1' # NFC 形式
('xF1' 也可以) >>> M = '\u006E\u0303' # NFD 形式
>>> L, M # 同一个字符('ñ','n') 有此"人格分裂"的字符不止它一个。Å、è、é、ÿ 等其它字符同样兼有 NFC 与 NFD 两种表示：
>>> '\u00C5','\u0041\u030A' # NFD (1 个码位)，NFC (2 个码位) ('Å','A')
>>> '\u00E8','\u0065\u0300' # 但对两者都是同一个字符('è','e')
>>> '\u00E9','\u0065\u0301' ('é','e')
>>> '\u03D4','\u03D2\u0308' ('ÿ','Y')
重要的是，这与该字符在字节中的已编码表示无关——编码形式自然随编码方案而异。上面每个字符的两种替代表示，差异在于其解码后、内存中的整数码位表示。编码形式（例如存进文件的）当然也不同，但解码后仍得到同样不同的码位：
```

```
>>> '\u00F1'.encode('utf8'), '\u006E\u0303'.encode('utf8')
(b'\xc3\xb1', b'n\xcc\x83')
>>> b'\xc3\xb1'.decode('utf8'), b'n\xcc\x83'.decode('utf8')
('ñ','n')
这种"多态"的不幸后果是破坏了相等性测试：因为同一个字符可以有多种表示，用常规工具根本没法比较。具体到 Python，按值相等的 == 运算符比较的是字符（其实是码位），面对 Unicode 二重身时就力不从心了：
```

```
>>> L = '\u00F1'
>>> M = '\u006E\u0303'
>>> L, M # 同一个字符('ñ','n')
>>> L == M # 但相等性说不！False
>>> len(L), len(M) # 因为码位不同(1, 2) 假如理想的计算
```

世界早已为了可移植性而约定俗成地只用一个形式，这一切也许不成问题。可惜我们并不住在那个世界。计算机厂商毕竟是厂商，各自偏爱不同的形式：大致说来，macOS 偏好 NFD，Windows 等偏好 NFC，这也可能随文件系统而异，而且对“非本平台形式”的容忍并不完整。净效果是：Unicode 标准在试图修复一个互操作问题的同时，又制造了另一个！那么，面对一个就是不肯标准化的世界，该怎么办？诀窍在于：任何跨平台处理文件或文件名的工具，都必须在比较之前把文本转成共同形式。幸运的是，Python 让这件事异常简单：

```
>>> from unicodedata import normalize
>>> L = '\u00F1'
>>> M = '\u006E\u0303'
>>> L, M # 同一字符，不同码位('ñ','n')
>>> L == M # 相等性失败
False
>>> normalize('NFC', L) == normalize('NFC', M) # 统一形式下相等性成立
True
>>> normalize('NFD', L) == normalize('NFD', M) # 只要一致，任选其一即可
True
```

举例来说，比较跨平台传递的文件名的程序，就要小心地这样规范化。否则，那些用户看着一样（而且按 Unicode 的定义确实一样）的文件名，会因为简单相等性对不上而失败，进而破坏搜索和同步。从任意来源（比如大多数互联网内容）获得的文本文件同理。这个故事还有下文（例如，NFC 与 NFD 规范形式的规范（canonical）等价仍不同于兼容（compatibility）等价，只是多数程序无需在意）。不过因为篇幅紧张，我们就此打住。想深入的话，网络与 Python 手册集都有大量后续资料（参见手册里的 Unicode HOWTO 及其对字符串 casefold 方法的相关论述）。不过至少，当你的文本处理代码被 Unicode 标准这个“有趣的抉择”咬到一口时，你现在不该再完全摸不着头脑了。

深度理解

·核心概念：Unicode 允许"同一字符多种码位序列"——合成 (NFC, 单码位) 与分解 (NFD, 多码位) ——导致 == 比较失败；解法是先经 `unicodedata.normalize` 统一形式再比较。

·底层实现：`normalize('NFC', s)` 依据 Unicode 数据库 (`unicodedata` 模块内嵌的 UCD 数据表) 把组合字符序列折叠成合成形式；NFD 反向展开。macOS 文件系统 (HFS+/APFS) 存储时用 NFD, Windows 用 NFC, 所以同一文件名跨平台后码位不同。

·为什么这样设计：为兼容"按字符组合书写"的老式文本与输入法，标准允许两种形式；合成形式省内存、分解形式便组合——标准选择两者并存，把统一工作留给处理工具。

·解决什么实际问题：搜索/同步/去重/字典键比较时"明明一样却不相等"——文件名、用户输入、网页抓取文本全部中招；统一 `normalize('NFC', ...)` 是通用止血法。

·初学者误区：以为 == 在字符串上永远可靠；或以为 `len()` 与"肉眼可见字符数"一致 ('n' 是 2 个码位但看起来是 1 个字符)。另外 NFC ≠ 兼容性等价 (如 fi 连字)，多数场景 NFC 已够用。

代码分析

```

from unicodedata import normalize
L, M = '\u00F1', '\u006E\u0303'      # NFC NFD
L == M                                # False
normalize('NFC', M) == L                # True
normalize('NFD', L) == M                # True
'\u00F1'.encode('utf8')                # b'\xc3\xb1'
'\u006E\u0303'.encode('utf8')         # b'n\xcc\x83'

```

解释器如何处理：unicodedata.normalize 遍历输入串的每个码位，对照 Unicode 规范分解/合成映射表重组序列；NFC 会先按规范分解（NFD）再按规范合成（NFC）两步走，确保“已规范化输入也不出错”。比较发生在码位层面，所以编码（UTF-8 字节序列）不同不影响码位比较结论——但反过来说，码位不同就足够让 == 判 False。

设计思想：Unicode 是“开放宇宙”——它规定字符集而非规范形态；Python 则把“归一化”做成标准库一等公民（unicodedata.normalize），让“比较前先统一形式”成为一行代码的纪律。这与全章“类型与编码纪律”的主线一脉相承。

37.25 Chapter Summary (本章小结)

英文原文

Chapter Summary This chapter explored the advanced string support available in Python for processing Unicode text and binary data. While some programmers use ASCII text and may get by with the basic string type and its operations, Python's string model fully s

upports both richer Unicode text via the normal `str` string type and byte-oriented binary data with `bytes` and `bytearray`. In addition, we learned how Python's file object automatically encodes and decodes Unicode text, and deals with byte strings for binary-mode files. Finally, we briefly met some text and binary tools in Python's library and sampled their behavior with strings, and took a look at the darker Unicode corners of BOMs and normalization. In the next chapter, we'll shift our focus to tool-builder topics, with a survey of ways to manage access to object attributes by inserting automatically run code. Before we move on, though, here's a set of questions to review what we've learned. This has been a substantial chapter, so be sure to read the quiz answers eventually for a more in-depth summary.

中文翻译

本章探索了 Python 为处理 Unicode 文本与二进制数据提供的进阶字符串支持。有些程序员只用 ASCII 文本，靠基础字符串类型及其操作就能应付；但 Python 的字符串模型完整支持两路：通过常规 `str` 字符串类型处理更丰富的 Unicode 文本，以及通过 `bytes` 与 `bytearray` 处理面向字节的二进制数据。此外，我们学到了 Python 的文件对象如何自动编码/解码 Unicode 文本、如何处理二进制模式文件的字节字符串。最后，我们简略认识了标准库里一些文本与二进制工具，用字符串实测了它们的行为，并顺带看了 BOM 与规范化这两个 Unicode 的“暗角”。下一章，我们把焦点转

向"工具建造者"主题：概述如何在属性访问时插入自动运行的代码来管理对象属性。不过在继续之前，这里有一套问题来回顾所学。这一章内容相当扎实，务必抽空读读测验答案，那就是更深入的总结。

深度理解

- 核心概念：本章是一条完整的"文本/二进制"主线——类型模型 (str/bytes/bytearray) → 字面量编码 → 文件自动转换 → 库工具 → Unicode 暗角 (BOM、规范化)。
- 底层实现：文件对象经 TextIOWrapper 在读写边界做编码/解码；所有工具 (re、struct、pickle、json、os、glob) 都遵守"str 给 str、bytes 给 bytes"的类型契约。
- 设计原因：用显式类型区分"文本"与"数据"，让程序员在"已解码"与"未解码"之间永远清醒，杜绝隐式转换的歧义。
- 实际问题：读文件必给 encoding；二进制数据必走'b'模式；跨平台比较前先 normalize；BOM 文件用 utf-8-sig——这四条是本章在实战里的浓缩。
- 初学者误区：以为 str 是"唯一的字符串"、bytes 是"怪东西"；其实两者都是字符串，只是"文本还是字节"的分工不同——本章全部内容都在解释这道分界线。

代码分析

```
#
text = open('f.txt', encoding='utf-8').read() # str +
data = open('f.bin', 'rb').read()             # bytes
fixed = unicodedata.normalize('NFC', text)     # Unicode ...
```

解释器如何处理：文本模式在 open 时构造带指定解码器

的 TextIOWrapper，每次 read/write 都过一遍编解码；二进制模式直接读写 BytesIO 背后的原始缓冲。两条路径的最终落点是同一个文件描述符——区别只在“有没有翻译层”。

设计思想：Python 的字符串双轨制是“显式优于隐式”哲学的最深实践：文本不是字节，字节不是文本，转换必须显式声明。整本书的字符串故事，就是这一句话的展开。

37.26 Test Your Knowledge: Quiz (知识测验：问题)

英文原文

Test Your Knowledge: Quiz 1.

What are the names and roles of string object types in Python?

2.

How do Python's string types differ in terms of operations?

3.

How can you code non-ASCII Unicode characters in a string in Python?

4.

What are the main differences between text- and binary-mode files in Python?

5.

How would you read a Unicode text file that contains text in a different encoding than the default for your platform?

6.

How can you create a Unicode text file in a specific encoding format?

7.

Why is ASCII text considered to be a kind of Unicode text?

8.

What do BOM and normalization mean in Unicode?

9.

How large an impact does Python's text/binary string dichotomy have on your code?

中文翻译

1. Python 里字符串对象类型的名称和职责各是什么？ 2. 就运算而言，Python 的字符串类型之间有何不同？ 3. 在 Python 的字符串里，如何写出非 ASCII 的 Unicode 字符？ 4. Python 中文本模式与二进制模式文件的主要区别是什么？ 5. 你会如何读取一个"内容编码与平台默认不同"的 Unicode 文本文件？ 6. 你如何以特定编码格式创建 Unicode 文本文件？ 7. 为什么 ASCII 文本被视为一种 Unicode 文本？ 8. 在 Unicode 里，BOM 与规范化各指什么？ 9. Python 的

文本/二进制字符串二分法对你的代码影响有多大？

深度理解

- 核心概念：九道题把本章拆成四块——类型模型（1-2）、字面量与转义（3）、文件模式（4-6）、Unicode 理念（7-9）。
- 底层实现：回答时应能点出"str 是码位序列、bytes 是字节序列"这一底层差异，以及 open 的 encoding 参数与'b' 模式如何决定翻译层有无。
- 设计原因：出题顺序从"是什么"到"怎么用"再到"为什么"——测试的不是背名词，而是能否把类型纪律用到读写文件的具体决策上。
- 实际问题：第 5/6 题是最常见工程场景——显式 encoding 进出；第 8 题考验 BOM 与规范化这两个"暗角"概念的清晰度。
- 初学者误区：第 9 题容易答"影响不大"——对纯 ASCII 也许如此，但非 ASCII 文本、二进制文件、跨平台代码都会放大影响，这正是全章要传达的。

代码分析

```
# 3 ASCII
'n'          # UTF-8
'\u00F1'     # Unicode
b'\xc3\xb1'.decode('utf-8') #
```

解释器如何处理：源码文件按声明的编码（默认 UTF-8）解码成 str；\u 转义在词法分析阶段直接生成码位；bytes 字面量则保留原始字节。三条路径殊途同归，都是"把字

符说清楚"的手段。

设计思想：测验题是"主动回忆"式复习——不翻书先把答案写出来，比对后再翻书补漏，记忆远比"再读一遍"牢固。这也是本书每章固定环节的用意。

37.27 Test Your Knowledge: Answers (知识测验：答案)

英文原文

Test Your Knowledge: Answers 1. Python has three string types: `str` (for Unicode text, including ASCII), `bytes` (for binary data with absolute byte values), and `bytearray` (a mutable flavor of bytes). The `str` type usually represents content stored in text files, and the other two types generally represent content stored in binary files (including encoded text). 2. Python's string types share almost all the same operations: method calls, sequence operations, and even larger tools like pattern matching work the same way. On the other hand, only `str` supports string formatting's `format` method and `f'...'` f-strings, and `bytearray` has an additional set of operations that perform in-place changes. The `str` and `bytes` types also have methods for encoding and decoding text, respectively. 3. Non-ASCII Unicode characters can be coded in a `str` string with both hex (`\xNN`) and Unicode (`\uNNNN`, `\UNNNNNNNNN`) esca

pes for code points, both of which denote character code points. They can also be coded in their encoded form as bytes using hex escapes and decoded to text. On most devices, non-ASCII characters—accented characters and emojis, for example—can also be typed or pasted directly into code and are interpreted per the UTF-8 default or an encoding-directive comment at the top of a source code file.

4. Text-mode files assume their content is Unicode text (even if it's all ASCII) and automatically decode when reading and encode when writing. With binary-mode files, bytes are transferred to and from the file unchanged. The contents of text-mode files are usually represented as `str` objects in your script, and the contents of binary files are represented as `bytes` (or `bytearray`) objects. Text-mode files also handle BOMs for certain encoding types and automatically translate newline sequences to and from the single `\n` character on input and output unless this is explicitly disabled; binary-mode files do not perform either of these steps.

5. To read files encoded in a different encoding than the default for your platform, simply pass the name of the file's encoding to the `open` built-in function; data will be decoded per the specified encoding when it is read from the file, and you'll get back a decoded Unicode-text `str` string that has no encoding. You can also read in binary mode and manually decode the bytes to a string by giving an encoding name, but this involves extra work

and is somewhat error-prone for multibyte characters (you may accidentally read a partial character sequence when loading content by bytes or chunks). 6. To create a Unicode text file in a specific encoding format, pass the desired encoding name to `open`; strings will be encoded per the desired encoding when they are written to the file. You can also manually encode a string to bytes and write it in binary mode, but this is usually extra work. 7. ASCII text is considered to be a kind of Unicode text because its 7-bit (0..127) range of values is a subset of many Unicode encodings. For example, valid ASCII text is also valid Latin-1 text (Latin-1 simply assigns the remaining possible values in an 8-bit byte to additional characters) and valid UTF-8 text (UTF-8 uses a variable-byte scheme for representing more characters, but ASCII characters are still represented with the same values in a single byte). This makes Unicode backward compatible with ASCII, as long as the encodings used represent ASCII the same way. 8. The Unicode BOM is a sequence of bytes added to the front of a text string or file in some encodings, to both identify the encoding and give the string's endianness. Unicode normalization converts characters to a common format to neutralize differences that arise for characters that have multiple code-point sequences in the Unicode standard and would fail to match by simple code-point (character) equality. 9. The impact of Python's strings model depends upon t

he types of strings you use. For scripts that use simple ASCII text on platforms with ASCII-compatible default encodings, the impact is probably minor: the `str` string sans encodings suffices in this case. Moreover, although string-related tools in the standard library, such as `re`, `struct`, `pickle`, and `os`, may technically use different types in different contexts, the effect is largely irrelevant to most programs because Python's `str` and `bytes` support almost identical interfaces. On the other hand, if you process non-ASCII Unicode text, you'll need to use `str` and pass encodings to `open`; if you deal with binary data files, you'll need to deal with content as `bytes` objects; and if your code must work across many platforms or process content from arbitrary sources, you'll want to use explicit encodings for text files. In general, Unicode is the way text works today, and will be a must-know tool for most Python users.

中文翻译

1. Python 有三种字符串类型：`str`（用于 Unicode 文本，包括 ASCII）、`bytes`（用于字节值绝对精确的二进制数据）与 `bytearray`（`bytes` 的可变版本）。`str` 通常表示文本文件里的内容，另外两种一般表示二进制文件里的内容（包括已编码的文本）。2. Python 的字符串类型几乎共享同样的运算：方法调用、序列操作，乃至更大的模式匹配工具，工作方式都相同。另一方面，只有 `str` 支持字符串格式化的 `format` 方法与 `f'...'` f-字符串，而 `bytearray` 还有一套能进

行原地 (in-place) 修改的额外操作。str 与 bytes 也各自拥有编码/解码文本的方法。

3. 在 str 字符串里，非 ASCII 的 Unicode 字符既可以用十六进制 (\xNN) 转义，也可以用 Unicode (\uNNNN、\UNNNNNNNNN) 转义来书写码位，两者都表示字符码位。这些字符也可以以编码形式写成 bytes (用十六进制转义) 再解码成文本。在多数设备上，非 ASCII 字符——比如带重音字符与 emoji——还可以直接键入或粘贴进代码，并按 UTF-8 默认或源码文件开头的编码指令注释来解释。

4. 文本模式文件假定其内容是 Unicode 文本 (即使全是 ASCII)，读时自动解码、写时自动编码。二进制模式文件的字节则原样进出文件。文本模式文件的内容在脚本中通常表示为 str 对象，二进制文件的内容表示为 bytes (或 bytearray) 对象。文本模式文件还会为某些编码类型处理 BOM，并在输入输出时自动把换行序列与单个 \n 互相转换 (除非显式禁用)；二进制模式文件两者都不做。

5. 要读取“编码与平台默认不同”的文件，只需把文件编码的名字传给 open 内建函数；数据读出时就会按指定编码解码，你拿到的是已解码的、不带编码的 Unicode 文本 str 字符串。你也可以用二进制模式读，再手工给编码名把 bytes 解码成字符串，但这样多做了一步，对多字节字符也容易出错 (按字节或分块加载内容时，你可能恰好读进半个字符序列)。

6. 要以特定编码格式创建 Unicode 文本文件，把想要的编码名传给 open 即可；写入文件时字符串就会按该编码编码。你也可以手工把字符串编码成 bytes 再用二进制模式写，但通常这就是多此一举。

7. ASCII 文本之所以被视为一种 Unicode 文本，是因为它的 7 位 (0..127) 取值区间是许多 Unicode 编码的子集。例如，合法的 ASCII 文本同样是合法的 Latin-1 文本 (Latin-1 只是

把 8 位字节中余下的取值赋给更多字符)，也是合法的 UTF-8 文本（UTF-8 用可变字节方案表示更多字符，但 ASCII 字符仍以同样的值、用单字节表示）。因此，只要所用编码对 ASCII 的表示一致，Unicode 就对 ASCII 向后兼容。8. Unicode BOM 是一段加在文本字符串或文件开头的字节序列，作用既是标识编码类型、也是声明字节序（端序）。Unicode 规范化把字符串转成统一形式，以消除“在 Unicode 标准中拥有多个码位值、按纯文本（字符）相等性比较会失败”的字符之间的差异。9. Python 字符串模型的影响取决于你实际使用的字符串类型。对于在 ASCII 兼容默认编码的平台上只使用简单 ASCII 文本的脚本，影响大概很小：这种情况下不带编码的 str 就已够用。而且，尽管标准库中与字符串相关的工具——如 re、struct、pickle、os——在不同语境下可能技术上使用不同的类型，由于 Python 的 str 与 bytes 接口几乎完全相同，对多数程序而言这种差异基本无关紧要。反过来，如果你处理非 ASCII 的 Unicode 文本，就要用 str 并给 open 传编码；如果你处理二进制数据文件，就要把内容当作 bytes 对象来应对；如果你的代码必须跨平台运行、或处理来自任意来源的内容，就该为文本文件使用显式编码。总体来说，Unicode 是当今文本世界的通行方式，对多数 Python 用户而言它将是一项必修技能。

深度理解

·核心概念：九条答案的结构是“类型→运算→写法→文件→理念”的完整闭环；把“文本/字节二分法”的影响按规模分级（纯 ASCII / Unicode / 二进制 / 跨平台）。

·底层实现：文本模式 = TextIOWrapper（翻译层：编码 + 换行 + BOM），二进制模式 = 原始缓冲（零翻译）；\x 转义在词法层生成码位，环境决定解释方式。

·设计原因：答案刻意强调"多数工具（re/struct/pickle/os）其实吃同一套接口"——这是安慰也是提醒：二分法的技术影响有限，生活方式的影响才需要你在 I/O 边界上自觉。

·实际问题：第 5/6 条是每日工程动作——open(..., encoding=...) 进出成对；第 9 条教你给代码"分级"：纯 ASCII 可省心，出了 ASCII 就必须显式。

·初学者误区：以为"只有处理 Unicode 才需要 encoding"——其实恰恰相反，凡是要落盘/读盘的一切文本路线都显式化更稳；以及误以为 ASCII 是"特例"，它其实是 Unicode 家族里最朴素的一员。

代码分析

```
#
' '          # str →
b'\xff\x00'  # bytes →
bytearray(b'abc')[0] = 65  # bytearray bytes
#
data = open('f.txt', encoding='cp1252').read()  # str
```

解释器如何处理：str 在内存中就是码位数组，encode() 时按指定编码变字节；bytes 是字节数组，decode() 时按编码解回码位。open 带着 TextIOWrapper 做转换，二进制模式则连 wrapper 都不装。

设计思想：答案在"界面的完全相同"与"语义的截然不同"

之间反复权衡——这套类型纪律之所以好用，正在于接口收敛（同样好用）、类型严格（决不混淆），让程序既清晰又省心。呼应全书最早第 4 章就已种下的 str/bytes 分家。

脚注（文件细节）：本章多数文件示例使用的文件名都是相对当前目录（因而也存放在当前目录）的，请务必在一个你有权创建文件的目录（即文件夹）里运行它们；必要时在你的 shell 或 IDE 里用 cd 切换过去。这在 Android 这类存储受限的平台上尤其要紧，不过多数应用中你大概率已经身处安全文件夹。另外，文件扩展名（如 .txt）对 Python 毫无意义，严格说是可选的；某些为了调试而挂着对象的 IDE 可能需要你手动调用 close 才能把改动落盘；还有，讲究的人请注意：file 在 Python 中已不再是一个内建名字，在这里用作变量名完全没问题！

本章总结

第 37 章是全书的“字符世界”大翻篇：从 str/bytes/bytearray 三分天下的类型模型，到字面量、编码方法、文件自动转换，再到 re/struct/pickle/json 工具库与 BOM、规范化这些暗角，一条线讲清了“文本与二进制”在 Python 里的全部纪律。以下总结站在“读完本章、要去写真实程序”的位置上，梳理你需要带走的東西。

技术拓展 (Technical Expansion)

·实际项目中的应用场景：

·统一在 I/O 边界做显式编码：读写文件时一律 `open(..., encoding='utf-8')`，文件名用 `str`——这是“文件乱码/崩溃”的第一解药。

·二进制数据处理：图片、音频、协议报文走 `'rb'/'wb' + struct` 打包解包；`pickle` 持久化同样走二进制模式——照本节示例即可。

·JSON/网络文本：`json`（文本）管数据交换、`pickle`（二进制）管对象持久化，各归其位；JSON 输出可配合 `ensureascii=False` + 显式 UTF-8 写出，保证可读性与跨平台。

·Unicode 暗角护盾：读记事本/Excel 生成的文件用 `utf-8-sig`；比较两端文件名/文本前先 `normalize('NFC', ...)`；不确定有没有 BOM 时输入统一 `utf-8-sig`。

·跨平台文件工具：目录遍历坚持 `str` 形态；需要精细控制或处理不可解码名字时，才用 `bytes` 形态 + `os.fsdecode`——非常规手段，慎用。

·与其他语言 (Java/C++) 的区别：

维度	Python	Java	C++
字符串类型	<code>str</code> （码位）/ <code>bytes</code> （字节）双轨	单一 <code>String</code> (UTF-16 码元序列)	<code>std::string</code> / <code>std::wstring</code>
文本/二进制	由 <code>mode't'/'b'</code> 显式区分	<code>Reader/Writer</code> 与 <code>Stream</code> 体系分开	靠 <code>iostream</code> 模式标志

字符编码默认	方法=UTF-8, 文件=locale/UTF-8 模式	UTF-16 内建, I/O 用平台默认	无内建统一行为, 看运行时
文件编码显式化	open(encoding=...) 一行到位	Charset/StandardCharsets	手动编码, 易漏
二进制一等公民	struct/bytes 内建且顺手	ByteBuffer 等不算顺手	二进制是天然强项

Python 的独到之处在于把"text 与 binary"当成两种正经类型而非文件的一种模式——一切转换都显式发生; Java 以 Reader/Writer 对称地做同样区分但类型上不够醒目; C++ 则几乎全靠约定。三者的共同功课都是: 编码决策必须发生在 I/O 边界。

- Python 发展历史背景 (务必严谨、不编造) :
- PEP 393 (2012, Python 3.3) : str 内部从"全等宽字符"改为"紧凑式宽字符"存储, 按内容需要取 1/2/4 字节每字符——也就是 str"无内部编码、纯码位数组"底层的由来。
- PEP 398 (2012, 3.3) : 提出过把默认字符串方法编码与 locale 解耦的倡议; 3.3 之后 sys.setdefaultencoding 才被彻底移除。
- PEP 540 (2019, Python 3.7) : 正式引入 UTF-8 模式, -X utf8 / PYTHONUTF8=1 成为打开它的钥匙。
- PEP 597 (2021, 3.15 预告) : 计划把 UTF-8 模式默认开启、open 不带 encoding 时给出提示——也就是书中预告的"未来默认更一致"之路 (具体落地请以官方文档为准, 勿渲染过头) 。
- Windows 的变化: Python 3.6 起 getfilesystemenco

ding() 在 Windows 上返回 utf-8，而非老旧的 mbcs——这是正本清源式的一个里程碑。

·高级开发者需要掌握的相关知识：

· codecs 模块：codecs.open、incrementalcodec、errors 处理器 (replace/ignore/surrogateescape/backslashreplace) ——尤其 surrogateescape 是 Linux 处理"不可解码文件名"的重要工具。

· unicodedata 全貌：normalize 四种形式 (NFC/NFD/NFKC/NFKD)、name/lookup、码位分类——文本清洗与排序的硬核工具箱。

· os/sys 编码接口：getfilesystemencoding/fsencode/fsdecode、getpreferredencoding、PYTHONIOENCODING 与 -X 家族各自的作用域。

· 重构既有项目：把散落的 open() 收敛为中央"打开文本"助手函数 (注入 encoding、newline、BOM 策略) ——经验程序员的护城河。

· stream 与缓冲：理解 TextIOWrapper 的编码缓冲三层与 flush 时机，避免"没关上就写不进"的经典误解。

学习建议 (Learning Advice)

·重要程度：★★★★☆ (4/5) ——文本/字符串是每个程序的骨架；若你永远只写纯 ASCII，可以降低权重，但 Unicode 文本、二进制数据、跨平台发布迟早会把你拉回这两章的纪律。

·应该掌握到什么程度：

1. 必须：能脱口而出三种字符串类型的名字与职责（题 1）；能随手写出 `str/bytes` 往返（`.encode/.decode`）。
2. 必须：`text/binary` 模式与 BOM、换行翻译行为——文本默认翻译换行、二进制默认不译，二者差异手感清晰。
3. 必须：`open()` 的 `encoding` 参数在任何“外来内容”文件中必定给齐，且懂得 `utf-8` vs `utf-8-sig` 的取舍（题 5/6）。
4. 必须：知道 `str` 的底层实现讲法（紧凑存储 + 无内部编码）、`sys.byteorder` 与大小端对 `UTF-16` 文件的影响。
5. 熟练：`re/struct/pickle/json/normalize` 五个工具的读写用例能直接开写。
6. 理解：默认编码三层（方法/文件名/文件内容）的差异与 `UTF-8` 模式这条总开关。

·后续应该学习的内容：

·第 38 章（属性访问/描述符）：把“工具建造者”视角延续下去——本章的 `str/bytes` 纪律在自定义序列化、自定义文件协议里会一再回响。

·第 39 章（装饰器）：把“编码纪律”用装饰器统一注入业务函数，是下一章的经典示范。

·工程练习：写一段“批量清洗 CSV”的脚本（自动 BOM 识别、`normalize` 去重、`UTF-8` 输出），把本章所有规则揉成一次实弹演习；再给文件服务器写一个 `str/bytes` 双形态遍历的爬虫壳，验证 37.9 的路径规则。